

BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



ĐỒ ÁN TỐT NGHIỆP
**NGHIÊN CỨU PHÁT TRIỂN TRỢ LÝ ẢO PHÁP LUẬT CHO
CHUYỂN ĐỔI SỐ**

Ngành: Công nghệ thông tin
Mã số: 7.48.02.01

Sinh viên thực hiện:

Họ và tên: Vương Văn Duy

Mã sinh viên: CT060411

Người hướng dẫn:

ThS. Trần Đức Thịnh

Khoa Công nghệ thông tin – Đại học Kinh doanh và công nghệ

Hà Nội, 2026

BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



ĐỒ ÁN TỐT NGHIỆP
**NGHIÊN CỨU PHÁT TRIỂN TRỢ LÝ ẢO PHÁP LUẬT CHO
CHUYỂN ĐỔI SỐ**

Ngành: Công nghệ thông tin
Mã số: 7.48.02.01

Sinh viên thực hiện:

Họ và tên: Vương Văn Duy

Mã sinh viên: CT060411

Người hướng dẫn:

ThS. Trần Đức Thịnh

Khoa Công nghệ thông tin – Đại học Kinh doanh và công nghệ

Hà Nội, 2026

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án tốt nghiệp, em đã nhận được rất nhiều sự quan tâm, hướng dẫn và hỗ trợ từ thầy cô, gia đình và bạn bè. Những sự giúp đỡ đó là nguồn động viên quan trọng để em có thể hoàn thành đề tài và hoàn thiện báo cáo này.

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến thầy ThS. Trần Đức Thịnh, giảng viên hướng dẫn đã tận tình định hướng, góp ý và chỉ bảo em trong suốt quá trình nghiên cứu, xây dựng hệ thống và trình bày báo cáo. Những nhận xét của thầy đã giúp em nhìn nhận vấn đề rõ ràng hơn, điều chỉnh hướng tiếp cận phù hợp hơn và hoàn thiện đồ án một cách nghiêm túc.

Em xin chân thành cảm ơn các thầy cô tại Học viện Kỹ thuật mật mã đã truyền đạt cho em những kiến thức nền tảng và chuyên môn trong suốt quá trình học tập. Các kiến thức về lập trình, cơ sở dữ liệu, hệ thống thông tin, trí tuệ nhân tạo và an toàn thông tin là cơ sở quan trọng giúp em triển khai đề tài này.

Em cũng xin gửi lời cảm ơn đến gia đình, những người đã luôn tin tưởng, động viên và tạo điều kiện cho em trong quá trình học tập cũng như trong thời gian thực hiện đồ án tốt nghiệp. Sự quan tâm và ủng hộ của gia đình là động lực lớn giúp em kiên trì hoàn thành công việc.

Bên cạnh đó, em xin cảm ơn bạn bè và những người đã trao đổi, góp ý, hỗ trợ em trong quá trình tìm hiểu tài liệu, thử nghiệm hệ thống và hoàn thiện nội dung báo cáo. Những ý kiến đóng góp đó giúp em có thêm góc nhìn thực tế để cải thiện sản phẩm và cách trình bày.

Mặc dù đã cố gắng nghiên cứu và hoàn thiện đồ án, do kiến thức và kinh nghiệm thực tế còn hạn chế nên báo cáo khó tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của các thầy cô để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn!

LỜI CAM ĐOAN

Tôi xin cam đoan bản đồ án này do tôi tự nghiên cứu dưới sự hướng dẫn của ThS. Trần Đức Thịnh.

Để hoàn thành đồ án này, tôi chỉ sử dụng những tài liệu đã ghi trong mục tài liệu tham khảo, ngoài ra không sử dụng bất cứ tài liệu nào khác mà không được ghi.

Nếu sai, tôi xin chịu mọi hình thức kỷ luật theo quy định của Học viện.

Hà Nội, ngày tháng năm 2026

Sinh viên thực hiện

(Ký tên và ghi rõ họ tên)

Vương Văn Duy

MỤC LỤC

LỜI CẢM ƠN	i
LỜI CAM ĐOAN	ii
MỤC LỤC	iii
DANH MỤC HÌNH VẼ	viii
DANH MỤC BẢNG BIỂU	x
LỜI NÓI ĐẦU	1
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI VÀ CƠ SỞ LÝ THUYẾT	3
1.1. Tổng quan về chatbot	3
1.1.1. Khái niệm chatbot.....	3
1.1.2. Tiềm năng ứng dụng chatbot trong lĩnh vực pháp luật tại Việt Nam	4
1.2. Một số phương pháp xây dựng chatbot tư vấn pháp luật.....	5
1.2.1. Chatbot dựa trên tập luật.....	5
1.2.2. Chatbot dựa trên LLMs và RAG	6
1.2.3. Xây dựng hệ thống chatbot dựa trên AI Agent	8
1.3. Tìm hiểu về mô hình ngôn ngữ lớn.....	8
1.3.1. Khái niệm mô hình ngôn ngữ lớn.....	8
1.3.2. Khái niệm Chat LLMs.....	10
1.4. Tìm hiểu về AI Agent	11
1.4.1. Khái niệm AI Agent.....	11
1.4.2. Các thành phần của AI Agent	12
1.4.3. Tool trong Agent.....	14
1.4.4. Planning trong Agent.....	15
1.4.5. Tự đánh giá và sửa lỗi	16
1.4.6. Tích hợp RAG vào AI Agent - Agentic RAG	18
1.5. Các công nghệ hỗ trợ phát triển hệ thống.....	19
1.5.1. Ngôn ngữ lập trình Python	19
1.5.2. LangGraph Framework	20
1.5.3. FastAPI Framework	24

1.5.4. <i>Next.js</i> 16.....	25
1.5.5. <i>Kotlin Multiplatform</i> và <i>Compose Multiplatform</i> - Ứng dụng di động	25
1.5.6. <i>Hệ thống cơ sở dữ liệu đa mô hình (Polyglot Persistence)</i>	26
1.6. Tổng kết chương 1.....	28
CHƯƠNG 2: PHƯƠNG PHÁP XÂY DỰNG HỆ THỐNG TƯ VẤN PHÁP LUẬT VỚI AGENTIC RAG.....	29
2.1. Kiến trúc tổng quan hệ thống Vietnam Law Chatbot	29
2.1.1. Lý do chọn kiến trúc <i>Microservices</i>	29
2.1.2. Tổng quan kiến trúc bốn lớp.....	30
2.1.3. Thành phần <i>Main Service</i>	32
2.1.4. Thành phần <i>RAG Service</i>	33
2.1.5. Kênh giao tiếp giữa các thành phần	33
2.2. Xây dựng kho dữ liệu vector pháp luật ban đầu	34
2.2.1. Cấu trúc dữ liệu pháp luật đầu vào.....	35
2.2.2. Chuẩn hóa metadata cho từng chunk.....	36
2.2.3. Chiến lược chunking cho văn bản pháp luật	37
2.2.4. Tạo embedding và lưu vào <i>ChromaDB</i>	38
2.2.5. Vai trò của kho vector trong <i>Agentic RAG</i>	39
2.3. Xác định tính chất AI Agent	39
2.3.1. Tính chất môi trường của Agent	39
2.3.2. Đặc tả vai trò và phạm vi Agent.....	40
2.3.3. Nguồn dữ liệu hỗ trợ Agent và hệ thống.....	41
2.3.4. Tập hành động của Agent	42
2.4. Nguồn tri thức và cơ chế truy hồi của Agent.....	42
2.4.1. Tra cứu kho pháp luật nội bộ.....	42
2.4.2. Tìm kiếm nguồn cập nhật bên ngoài	43
2.5. Luồng <i>Agentic RAG</i> có kiểm chứng.....	44
2.5.1. Trạng thái xử lý trong Agent.....	44

2.5.2. Cấu trúc đồ thị tổng quan	45
2.5.3. Bước 1 - Kiểm soát đầu vào.....	46
2.5.4. Bước 2 - Phân tích và tái cấu trúc truy vấn.....	47
2.5.5. Bước 3 - Suy luận và thu thập bằng chứng.....	48
2.5.6. Bước 4 - Kiểm chứng câu trả lời.....	49
2.6. Tối ưu pipeline RAG - truy xuất hai giai đoạn.....	50
2.6.1. Hạn chế của tìm kiếm vector đơn lẻ	50
2.6.2. Xếp hạng lại kết quả truy hồi.....	50
2.6.3. Công thức kết hợp điểm phù hợp	51
2.6.4. Ưu tiên văn bản mới	51
2.6.5. Giải quyết mâu thuẫn pháp lý theo thời gian	52
2.6.6. Ngưỡng tin cậy	53
2.7. Luồng Tư vấn có Hướng dẫn (Guided Consultation)	54
2.7.1. Vấn đề và động lực thiết kế.....	54
2.7.2. Thiết kế không lưu trạng thái phiên	55
2.7.3. Bước 1 - Thu thập ngữ cảnh	55
2.7.4. Bước 2 - Trả lời theo ngữ cảnh đã làm rõ.....	55
2.7.5. Cơ chế trả lời theo dòng sự kiện	56
2.8. Quy trình cập nhật tri thức từ văn bản pháp luật	57
2.8.1. Tổng quan quy trình	57
2.8.2. Trích xuất và cấu trúc hóa văn bản PDF	58
2.8.3. Cơ chế hoàn tác để giữ dữ liệu nhất quán.....	59
2.9. Luồng Chat SSE và Quản lý Hội thoại	60
2.9.1. Cơ chế trả lời theo dòng sự kiện cho chat.....	60
2.9.2. Các trạng thái trong luồng chat.....	61
2.9.3. Quản lý lịch sử hội thoại và câu hỏi gợi ý.....	61
2.10. Phương pháp xây dựng lớp ứng dụng đa nền tảng	63
2.10.1. Mobile App - Kotlin Multiplatform và Compose Multiplatform	63

2.10.2. Mô hình MVI trong Mobile App.....	64
2.10.3. Thiết kế trải nghiệm chat streaming.....	65
2.10.4. Admin Web - Next.js Dashboard cho quản trị dữ liệu	66
2.10.5. WebSocket cho tiến trình xử lý tài liệu	67
2.10.6. Kết nối giữa frontend và backend	68
2.11. Tổng kết chương 2.....	69
CHƯƠNG 3: PHÂN TÍCH, THIẾT KẾ HỆ THỐNG	70
3.1. Yêu cầu của hệ thống.....	70
3.1.1. Yêu cầu về hình thức sản phẩm.....	70
3.1.2. Yêu cầu chức năng.....	71
3.1.3. Yêu cầu phi chức năng.....	72
3.2. Phân tích hệ thống	73
3.2.1. Biểu đồ ca sử dụng tổng quát.....	73
3.2.2. Đặc tả các ca sử dụng.....	73
3.2.3. Biểu đồ tuần tự	84
3.3. Thiết kế hệ thống.....	93
3.3.1. Thiết kế cơ sở dữ liệu.....	93
3.3.2. Thiết kế các API.....	97
3.4. Tổng kết chương 3.....	100
CHƯƠNG 4: TRIỂN KHAI VÀ THỰC NGHIỆM	101
4.1. Môi trường triển khai.....	101
4.1.1. Môi trường phát triển	101
4.1.2. Kiến trúc triển khai.....	101
4.1.3. Biến môi trường và bảo mật.....	102
4.2. Giao diện ứng dụng Mobile	102
4.2.1. Đăng nhập và xác thực	103
4.2.2. Thư viện văn bản pháp luật.....	104
4.2.3. Chat Agentic RAG và ThinkingPanel	106

4.2.4. AI-Powered Search (Tìm kiếm theo ngữ nghĩa)	110
4.2.5. Guided Consultation (Tư vấn có hướng dẫn)	111
4.2.6. Quản lý hội thoại	115
4.3. Giao diện Admin Web	115
4.3.1. Đăng nhập quản trị.....	116
4.3.2. Dashboard tổng quan	116
4.3.3. Quản lý tài liệu	117
4.3.4. Tải lên và theo dõi tiến trình.....	118
4.4. Đánh giá hệ thống.....	119
4.4.1. Thiết kế thực nghiệm.....	119
4.4.2. Các chỉ số đánh giá	120
4.4.3. Kết quả thực nghiệm.....	120
4.4.4. Phân tích tình huống nghiên cứu	122
4.4.5. Đánh giá hiệu năng hệ thống.....	123
4.5. Tổng kết chương 4.....	123
KẾT LUẬN.....	125
TÀI LIỆU THAM KHẢO	127

DANH MỤC HÌNH VẼ

Hình 1.1. Chatbot dựa trên tập luật khi câu hỏi nằm ngoài kịch bản định sẵn	6
Hình 1.2. Mô hình RAG - kết hợp truy xuất tài liệu và tổng hợp câu trả lời	7
Hình 1.3. AI Agent bao gồm nhiều thành phần tư duy, truy vấn và tương tác.....	8
Hình 1.4. Mô hình SWE AI Agent	12
Hình 1.5. Sơ đồ kiến trúc các thành phần của AI Agent	12
Hình 1.6. Mô hình quá trình Planning và Re-planning của Agent.....	16
Hình 1.7. Ví dụ áp dụng ReAct Agent để trả lời câu hỏi.....	18
Hình 1.8. Mô phỏng các Nút và các Cạnh trong LangGraph	21
Hình 1.9. Ví dụ State trong Graph của một Agent về thông tin thời tiết.....	23
Hình 2.1. Kiến trúc tổng quan hệ thống Vietnam Law Chatbot	30
Hình 2.2. Sơ đồ thành phần Main Service	32
Hình 2.3. Sơ đồ thành phần RAG Service	33
Hình 2.4. Phương pháp Agentic RAG gồm bốn bước xử lý.....	45
Hình 2.5. Vòng lặp Re-Act trong quá trình suy luận.....	48
Hình 2.6. Thuật toán phát hiện xung đột văn bản cũ và mới	52
Hình 2.7. Luồng tư vấn có hướng dẫn gồm hai bước	54
Hình 2.8. Luồng cập nhật tri thức pháp luật từ phía quản trị.....	57
Hình 2.9. Luồng trải nghiệm hỏi đáp theo thời gian thực.....	60
Hình 3.1. Biểu đồ ca sử dụng tổng quát của hệ thống Vietnam Law Chatbot	73
Hình 3.2. Ca sử dụng đăng ký, đăng nhập và quản lý tài khoản.....	74
Hình 3.3. Ca sử dụng quản lý danh sách cuộc trò chuyện	76
Hình 3.4. Ca sử dụng gửi câu hỏi pháp luật	77
Hình 3.5. Ca sử dụng tư vấn có hướng dẫn.....	78
Hình 3.6. Ca sử dụng tra cứu thư viện pháp luật.....	80
Hình 3.7. Ca sử dụng tìm kiếm ngữ nghĩa	81
Hình 3.8. Ca sử dụng cập nhật văn bản pháp luật	82
Hình 3.9. Ca sử dụng theo dõi tiến trình xử lý tài liệu	83
Hình 3.10. Biểu đồ tuần tự luồng xác thực và quản lý tài khoản	85
Hình 3.11. Biểu đồ tuần tự luồng quản lý hội thoại	86
Hình 3.12. Biểu đồ tuần tự luồng hỏi đáp pháp luật.....	87
Hình 3.13. Biểu đồ tuần tự luồng tư vấn có hướng dẫn.....	88
Hình 3.14. Biểu đồ tuần tự luồng tra cứu thư viện pháp luật.....	89
Hình 3.15. Biểu đồ tuần tự luồng tìm kiếm ngữ nghĩa	90

Hình 3.16. Biểu đồ tuần tự luồng thông kê và theo dõi tác vụ	91
Hình 3.17. Biểu đồ tuần tự luồng tải lên và xử lý văn bản pháp luật	92
Hình 3.18. Biểu đồ tuần tự luồng quản lý văn bản phía quản trị	93
Hình 3.19. Sơ đồ quan hệ dữ liệu trong PostgreSQL	94
Hình 4.1. Màn hình đăng nhập ứng dụng Mobile	103
Hình 4.2. Màn hình thư viện văn bản pháp luật	104
Hình 4.3. Màn hình chi tiết văn bản pháp luật	105
Hình 4.4. Màn hình Chat với ThinkingPanel đang hiển thị các bước xử lý	106
Hình 4.5. Câu trả lời Agentic RAG	107
Hình 4.6. Câu trả lời Agentic RAG - gợi ý câu hỏi tiếp theo	108
Hình 4.7. Câu trả lời Agentic RAG - danh sách nguồn tham chiếu	109
Hình 4.8. Giao diện tìm kiếm văn bản pháp luật theo ngữ nghĩa	110
Hình 4.9. Guided Consultation - câu hỏi gốc và gợi ý câu hỏi làm rõ ngữ cảnh..	111
Hình 4.10. Guided Consultation - câu hỏi thu thập thông tin chi tiết	112
Hình 4.11. Guided Consultation - hệ thống đang suy nghĩ và tổng hợp	113
Hình 4.12. Guided Consultation - kết quả tư vấn chuyên sâu theo ngữ cảnh.....	114
Hình 4.13. Màn hình danh sách hội thoại	115
Hình 4.14. Trang đăng nhập Admin Web	116
Hình 4.15. Dashboard tổng quan với BarChart và các card thống kê	116
Hình 4.16. Chi tiết tài liệu - danh sách điều khoản đã được trích xuất	117
Hình 4.17. Giao diện tải lên PDF đang trong quá trình xử lý	118
Hình 4.18. Trạng thái hoàn thành sau khi xử lý tài liệu thành công	119

DANH MỤC BẢNG BIỂU

Bảng 2.1. Đặc tả các kênh giao tiếp trong hệ thống	34
Bảng 2.2. Tính chất môi trường của Vietnam Law AI Agent.....	39
Bảng 2.3. Định nghĩa vai trò và giới hạn hoạt động của Agent	40
Bảng 2.4. Nguồn dữ liệu phục vụ Agent và các chức năng tra cứu	41
Bảng 2.5. Trạng thái trung gian trong luồng RAG có kiểm chứng.....	44
Bảng 2.6. Dữ liệu đầu vào và đầu ra của bước kiểm soát đầu vào.....	46
Bảng 2.7. Dữ liệu đầu vào và đầu ra của bước phân tích truy vấn.....	47
Bảng 2.8. Dữ liệu đầu vào và đầu ra của bước suy luận.....	49
Bảng 2.9. Dữ liệu đầu vào và đầu ra của bước kiểm chứng	49
Bảng 2.10. Tham số Year Boost	51
Bảng 2.11. Các ngưỡng tin cậy trong pipeline truy hồi.....	53
Bảng 2.12. Các loại sự kiện trong luồng tư vấn có hướng dẫn	56
Bảng 2.13. Các sự kiện SSE của luồng chat	61
Bảng 2.14. Lý do lựa chọn Kotlin Multiplatform cho Mobile App	64
Bảng 2.15. Các trạng thái xử lý tài liệu.....	68
Bảng 3.1. Yêu cầu về hình thức sản phẩm	70
Bảng 3.2. Yêu cầu chức năng đối với người dùng	71
Bảng 3.3. Yêu cầu chức năng đối với quản trị viên	71
Bảng 3.4. Yêu cầu phi chức năng	72
Bảng 3.5. Đặc tả ca sử dụng đăng ký, đăng nhập và quản lý tài khoản	74
Bảng 3.6. Đặc tả ca sử dụng quản lý danh sách cuộc trò chuyện	76
Bảng 3.7. Đặc tả ca sử dụng gửi câu hỏi pháp luật	77
Bảng 3.8. Đặc tả ca sử dụng tư vấn có hướng dẫn	78
Bảng 3.9. Đặc tả ca sử dụng tra cứu thư viện pháp luật	80
Bảng 3.10. Đặc tả ca sử dụng tìm kiếm ngữ nghĩa.....	81
Bảng 3.11. Đặc tả ca sử dụng cập nhật văn bản pháp luật.....	82
Bảng 3.12. Đặc tả ca sử dụng theo dõi tiến trình xử lý tài liệu.....	83
Bảng 3.13. Bảng users lưu thông tin người dùng	94
Bảng 3.14. Bảng conversations lưu hội thoại.....	95
Bảng 3.15. Bảng messages lưu tin nhắn	95
Bảng 3.16. Bảng document_tasks lưu tiến trình xử lý tài liệu.....	95
Bảng 3.17. Bảng refresh_tokens lưu phiên đăng nhập	96
Bảng 3.18. Collection VietnamLawDB.articles trong MongoDB	96

Bảng 3.19. Collection vietnamese_law trong ChromaDB.....	97
Bảng 3.20. Nhóm API xác thực	97
Bảng 3.21. Nhóm API hội thoại và hỏi đáp	98
Bảng 3.22. Nhóm API tư vấn có hướng dẫn	98
Bảng 3.23. Nhóm API tra cứu pháp luật.....	98
Bảng 3.24. Nhóm API quản trị tài liệu	99
Bảng 3.25. Nhóm API nội bộ của RAG Service	99
Bảng 4.1. Cấu hình môi trường phát triển.....	101
Bảng 4.2. Cấu hình Docker Compose lớp dữ liệu	102
Bảng 4.3. Cấu trúc bộ dữ liệu kiểm thử	119
Bảng 4.4. Bộ chỉ số đánh giá hệ thống.....	120
Bảng 4.5. Kết quả đánh giá hệ thống Agentic RAG.....	121
Bảng 4.6. Chỉ số hiệu năng hệ thống	123

LỜI NÓI ĐẦU

Trong bối cảnh công cuộc chuyển đổi số quốc gia đang diễn ra mạnh mẽ trên mọi lĩnh vực, việc ứng dụng công nghệ thông tin vào quản lý, vận hành và cung cấp dịch vụ hành chính công, pháp luật ngày càng trở nên cấp thiết. Đặc biệt đối với lĩnh vực pháp luật, nơi hệ thống văn bản quy phạm pháp luật của Việt Nam rất đồ sộ, phức tạp và thường xuyên được cập nhật, việc tra cứu và tìm hiểu luật của người dân, doanh nghiệp và thậm chí cả các cán bộ chuyên trách đang gặp không ít khó khăn. Người dùng thông thường gặp rào cản lớn về mặt thuật ngữ chuyên ngành cũng như việc phân mảnh của các thông tin.

Sự bùng nổ của trí tuệ nhân tạo (AI), đặc biệt là các Mô hình ngôn ngữ lớn (Large Language Models - LLMs) đã mở ra hướng đi mới trong việc giải quyết bài toán giao tiếp giữa người và máy. Tuy nhiên, các LLM thông thường dễ gặp phải hiện tượng "ảo giác thông tin" (hallucination) – một điểm chí mạng trong lĩnh vực đòi hỏi tính chính xác cao như pháp luật. Để giải quyết triệt để vấn đề này, kiến trúc Retrieval-Augmented Generation (RAG) và hiện đại hơn là mô hình AI Agent (Agentic RAG) ra đời, kết hợp khả năng suy luận ngôn ngữ linh hoạt của LLM với thông tin được trích xuất từ cơ sở dữ liệu luật pháp chuẩn xác, cập nhật theo thời gian thực.

Nhận thấy tiềm năng to lớn đó cùng mong muốn đóng góp một giải pháp thiết thực cho công tác chuyển đổi số, em đã quyết định chọn đề tài: ***“Nghiên cứu phát triển trợ lý ảo pháp luật cho chuyển đổi số”***.

Mục tiêu chính của đề án là nghiên cứu và xây dựng một hệ thống Chatbot tư vấn pháp luật (Vietnam Law Chatbot) với các chức năng nổi bật như: Chat hỏi đáp dựa trên cơ sở dữ liệu luật Việt Nam; Tư vấn có hướng dẫn (Guided Consultation) giúp làm rõ các bối cảnh cụ thể của người dùng trước khi trả lời; Cung cấp công cụ quản trị giúp tự động hóa quá trình số hóa và cập nhật văn bản pháp luật bằng các kỹ thuật xử lý PDF đa luồng và mô hình AI.

Nội dung của báo cáo được cấu trúc thành 4 chương chính:

Chương 1: Tổng quan đề tài và Cơ sở lý thuyết: Trình bày bối cảnh, thực trạng, các khái niệm về RAG, AI Agent, LLMs và các công nghệ sử dụng trong đề án.

Chương 2: Phương pháp xây dựng hệ thống tư vấn pháp luật với Agentic RAG: Trình bày phương pháp xây dựng hệ thống tư vấn pháp luật, bao gồm kiến

trúc tổng quan, cách thiết kế Agentic RAG, nguồn tri thức, cơ chế truy xuất, luồng tư vấn có hướng dẫn, quy trình cập nhật tri thức và lớp ứng dụng đa nền tảng.

Chương 3: Phân tích, thiết kế hệ thống: Trình bày yêu cầu hệ thống, phân tích các ca sử dụng, mô tả luồng tương tác bằng biểu đồ tuần tự, đồng thời thiết kế cơ sở dữ liệu và các nhóm API chính phục vụ cho hệ thống.

Chương 4: Triển khai và thực nghiệm: Trình bày môi trường triển khai, giao diện ứng dụng Mobile, giao diện Admin Web, quy trình vận hành thực tế và kết quả đánh giá hệ thống thông qua các chỉ số thực nghiệm.

Sau một thời gian nghiên cứu và thực hiện, đồ án đã hoàn thành các mục tiêu cơ bản đề ra. Tuy nhiên, do lĩnh vực Trí tuệ nhân tạo, đặc biệt là chatbot thể hệ mới, là một ngành khoa học phức tạp và đòi hỏi nghiên cứu dài hạn, cùng với khung thời gian thực hiện có hạn, báo cáo chắc chắn không thể tránh khỏi những thiếu sót. Kính mong nhận được sự đóng góp quý báu từ Quý Thầy cô và các bạn để đề tài được hoàn thiện hơn.

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI VÀ CƠ SỞ LÝ THUYẾT

1.1. Tổng quan về chatbot

1.1.1. Khái niệm chatbot

Chatbot hay tác nhân hội thoại là một hệ thống phần mềm được thiết kế để mô phỏng cuộc hội thoại với người dùng thông qua giao diện văn bản hoặc giọng nói. Mục tiêu cốt lõi của chatbot là tự động hoá quy trình giao tiếp, cho phép người dùng truy xuất thông tin hoặc thực hiện tác vụ một cách hiệu quả mà không cần sự can thiệp trực tiếp từ con người. Việc ứng dụng chatbot mang lại nhiều lợi ích đáng kể, bao gồm: nâng cao hiệu quả vận hành, giảm chi phí nhân sự cho các tác vụ lặp lại, cung cấp dịch vụ liên tục (24/7) và cải thiện trải nghiệm người dùng.

Dựa trên công nghệ lõi, chatbot có thể được phân loại thành hai nhóm chính: chatbot truyền thống và chatbot ứng dụng trí tuệ nhân tạo.

Các hệ thống chatbot truyền thống là những hệ thống sử dụng phương pháp tập luật, hoạt động dựa trên một tập hợp các quy tắc và kịch bản được lập trình sẵn. Điểm yếu cố hữu của phương pháp này là sự thiếu linh hoạt. Hệ thống phụ thuộc hoàn toàn vào cơ sở tri thức do người thiết kế xây dựng và chỉ có thể xử lý các câu hỏi tuân thủ đúng kịch bản. Khi người dùng đặt câu hỏi bằng những cấu trúc ngữ pháp hoặc từ vựng khác biệt, chatbot truyền thống thường không thể hiểu và đưa ra phản hồi phù hợp.

Để khắc phục những hạn chế của chatbot truyền thống, các thế hệ chatbot hiện đại tích hợp những thành tựu của trí tuệ nhân tạo, đặc biệt là trong lĩnh vực Xử lý ngôn ngữ tự nhiên (NLP) và các Mô hình ngôn ngữ lớn (LLMs). Thay vì chỉ bám vào các quy tắc cứng, AI chatbot có khả năng "hiểu" ý định đằng sau câu hỏi của người dùng, cho phép chúng xử lý các truy vấn đa dạng và phức tạp hơn, đồng thời tạo ra các phản hồi tự nhiên, linh hoạt.

Một trong những công nghệ tiên tiến được sử dụng để xây dựng AI chatbot hiện nay là nền tảng kiến trúc AI Agent. AI Agent không chỉ dừng lại ở việc trả lời dựa trên tập dữ liệu được huấn luyện trước mà còn có khả năng suy luận và sử dụng các công cụ bên ngoài, cho phép chatbot tương tác với các hệ thống khác như cơ sở dữ liệu, API tìm kiếm web,... Trong phạm vi đề án này, hướng tiếp cận được lựa chọn là xây dựng một AI chatbot tư vấn pháp luật dựa trên kiến trúc AI Agent nhằm khai thác tối đa khả năng tương tác với dữ liệu pháp luật liên quan.

1.1.2. Tiềm năng ứng dụng chatbot trong lĩnh vực pháp luật tại Việt Nam

1.1.2.1. Thực trạng tra cứu pháp luật và nhu cầu ứng dụng chatbot

Hệ thống văn bản quy phạm pháp luật của Việt Nam là một trong những hệ thống pháp luật có quy mô lớn và được cập nhật liên tục tại khu vực Đông Nam Á. Theo dữ liệu thực nghiệm thu thập trong quá trình thực hiện đề tài, hệ thống đã số hoá và nạp vào cơ sở dữ liệu nội bộ tổng cộng khoảng 46.047 văn bản pháp luật với khoảng 528.620 điều luật, trải dài 82 năm từ 1945 đến 2026 và bao quát hơn 62.000 chủ đề pháp lý khác nhau.

- Người dùng thông thường gặp nhiều rào cản khi tự tra cứu pháp luật:
- Truy cập thông tin nhanh chóng 24/7: Người dân và doanh nghiệp cần tra cứu quy định pháp luật mà không phụ thuộc vào giờ làm việc của cơ quan nhà nước hay văn phòng luật sư. Nhu cầu này đặc biệt cao trong các thời điểm cần xử lý gấp thủ tục hành chính, hợp đồng hay tranh chấp.
- Giảm rào cản thuật ngữ chuyên ngành: Ngôn ngữ pháp luật mang tính trang trọng với nhiều thuật ngữ đặc thù như "phạm vi điều chỉnh", "đối tượng áp dụng", "hiệu lực thi hành" - khiến người dân không có nền tảng pháp lý khó tiếp cận trực tiếp. Chatbot AI có thể diễn giải các thuật ngữ này thành ngôn ngữ tự nhiên, dễ hiểu.
- Đối chiếu thông tin phân mảnh: Một quy định cụ thể thường được đề cập rải rác trong nhiều văn bản (luật gốc, nghị định hướng dẫn, thông tư sửa đổi). Chatbot có thể tự động tổng hợp thông tin từ nhiều nguồn và đưa ra câu trả lời đầy đủ, kèm dẫn nguồn rõ ràng.
- Phát hiện xung đột thời gian hiệu lực: Pháp luật Việt Nam thường xuyên được sửa đổi - riêng năm 2025 đã có hàng chục nghìn điều luật được ban hành mới hoặc sửa đổi. Chatbot AI có thể cảnh báo khi hai văn bản mâu thuẫn nhau, giúp người dùng áp dụng đúng quy định đang có hiệu lực.

Các giải pháp hiện có như thuvienphapluat.vn, luatvietnam.vn, vbpl.vn chủ yếu cung cấp công cụ tìm kiếm theo từ khoá và liệt kê văn bản - người dùng vẫn phải tự đọc, tự đối chiếu và tự suy luận. Đây chính là khoảng trống mà hệ thống Vietnam Law Chatbot hướng tới lấp đầy.

1.1.2.2. Lợi ích của chatbot AI đối với các đối tượng sử dụng

Bên cạnh lợi ích đối với người dân và doanh nghiệp, hệ thống chatbot AI tư vấn pháp luật cũng mang lại nhiều lợi ích rõ rệt cho các đối tượng sử dụng khác:

Tiết kiệm thời gian trong việc giải đáp câu hỏi lặp lại: Các cán bộ, công chức thường nhận được nhiều câu hỏi trùng lặp từ người dân như: thời hạn nộp hồ sơ, thủ tục đăng ký, mức phạt vi phạm,... Chatbot có thể thay thế việc trả lời những câu hỏi này một cách nhanh chóng và chính xác, giúp cán bộ tập trung vào các công việc phức tạp và chuyên môn hơn.

Hỗ trợ tra cứu tức thời khi soạn thảo văn bản: Khi được tích hợp với hệ thống quản lý văn bản, chatbot có thể cung cấp thông tin tức thời như các điều luật liên quan, tiền lệ, quy định hiện hành - giúp cán bộ soạn thảo quyết định hành chính nhanh chóng và chính xác hơn.

Cải thiện chất lượng dịch vụ công: Nhờ có chatbot xử lý các yêu cầu cơ bản, cán bộ có thêm thời gian để tương tác sâu hơn với người dân trong các vấn đề phức tạp - góp phần nâng cao chất lượng phục vụ và sự hài lòng của người dân.

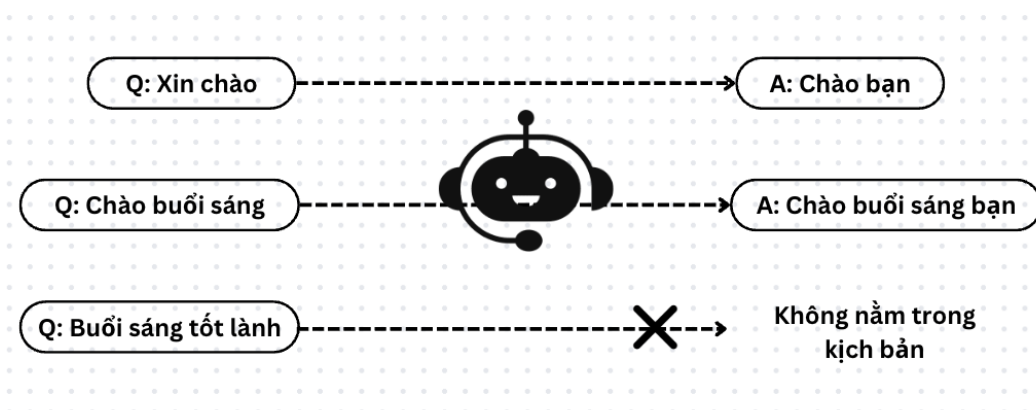
Thúc đẩy chuyển đổi số trong lĩnh vực pháp luật: Xây dựng chatbot pháp luật đúng tinh thần Quyết định 749/QĐ-TTg ngày 03/6/2020 của Thủ tướng Chính phủ về Chương trình Chuyển đổi số quốc gia - đưa dịch vụ pháp lý lên nền tảng số, giúp người dân tiếp cận thông tin pháp luật bình đẳng và thuận tiện hơn.

1.2. Một số phương pháp xây dựng chatbot tư vấn pháp luật

Việc phát triển chatbot tư vấn pháp luật dựa trên nhiều phương pháp luận khác nhau, mỗi phương pháp đều sở hữu những đặc điểm, ưu và nhược điểm riêng biệt, phản ánh sự tiến hoá của ngành khoa học máy tính và xử lý ngôn ngữ tự nhiên. Về cơ bản, các phương pháp xây dựng chatbot hiện nay có thể được phân thành ba nhóm chính: chatbot dựa trên các tập luật, chatbot dựa trên các mô hình LLMs và RAG, chatbot dựa trên AI Agent.

1.2.1. Chatbot dựa trên tập luật

Phương pháp xây dựng chatbot dựa trên tập luật là cách tiếp cận sơ khai và nền tảng nhất. Hoạt động của các chatbot này hoàn toàn phụ thuộc vào một kịch bản và một bộ quy tắc được lập trình sẵn. Các nhà phát triển phải dự đoán các câu hỏi, từ khoá có thể có từ người dùng và xây dựng một cây quyết định logic để đưa ra câu trả lời tương ứng.



Hình 1.1. Chatbot dựa trên tập luật khi câu hỏi nằm ngoài kịch bản định sẵn

Mặc dù phương pháp này đảm bảo tính nhất quán và có thể kiểm soát hoàn toàn luồng hội thoại, nó bộc lộ nhiều hạn chế. Hệ thống thiếu đi sự linh hoạt, không có khả năng xử lý các câu hỏi phức tạp, sai chính tả hoặc các truy vấn không nằm trong kịch bản định sẵn. Với lĩnh vực pháp luật Việt Nam - nơi có hàng trăm nghìn điều luật và vô số cách diễn đạt câu hỏi - phương pháp này hoàn toàn không khả thi.

1.2.2. Chatbot dựa trên LLMs và RAG

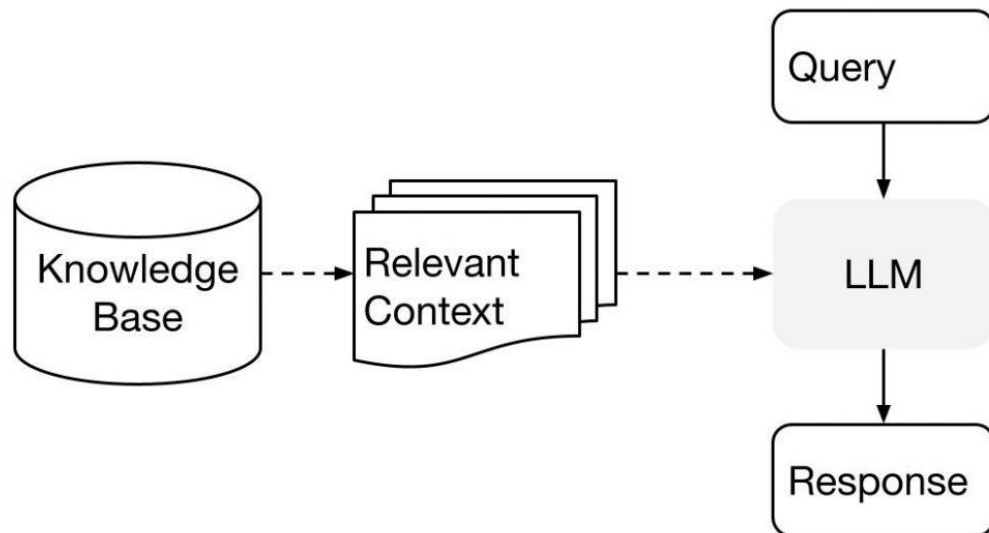
Để khắc phục những giới hạn của cách tiếp cận dựa trên luật, các phương pháp dựa trên học máy đã được nghiên cứu và ứng dụng rộng rãi. Thay vì các quy tắc cứng, chatbot học máy được huấn luyện trên các tập dữ liệu lớn để tự nhận dạng mẫu, hiểu ý định và thực thể trong câu nói của người dùng.

Trong các chatbot này, nổi bật hơn cả là các chatbot với nền tảng là mô hình sinh ngôn ngữ, điển hình là các Mô hình ngôn ngữ lớn, có khả năng tự tạo ra câu trả lời mới hoàn toàn, mang lại các cuộc hội thoại tự nhiên và linh hoạt hơn. Tuy nhiên, khi áp dụng vào lĩnh vực pháp luật, cần phải khắc phục một số vấn đề:

- Non-Specific domain: Các mô hình LLM được đào tạo giải quyết đa dạng các vấn đề, không thể giải quyết chuyên sâu bài toán pháp luật Việt Nam với đặc thù riêng.
- Non-access private datas: Các mô hình LLM không thể hiểu những dữ liệu văn bản pháp luật nội bộ của hệ thống. Đồng thời, vì lý do bảo mật, hệ thống không được phép đào tạo với dữ liệu riêng tư. Tuy nhiên, việc không thể sử dụng dữ liệu nội bộ lại khiến chatbot không có khả năng thấu hiểu và giải quyết các vấn đề pháp lý cụ thể của người dùng.
- Hiện tượng ảo giác (Hallucination): Khi không được đào tạo kiến thức liên quan phần dữ liệu pháp luật nội bộ, các mô hình LLM có xu hướng "bịa" ra

những thông tin này để trả lời - ví dụ, tự đặt ra số hiệu nghị định không tồn tại, tự bịa mức phạt không có căn cứ - dẫn đến hậu quả pháp lý nghiêm trọng cho người dùng.

Để giải quyết những bài toán này, kỹ thuật Truy xuất tạo sinh tăng cường (RAG - Retrieval-Augmented Generation) thường được áp dụng nhằm tìm kiếm và bổ sung thêm ngữ cảnh từ cơ sở dữ liệu pháp luật, cho phép LLM trả lời câu hỏi với những dữ liệu cụ thể, liên quan trực tiếp đến câu hỏi của người dùng.



Hình 1.2. Mô hình RAG - kết hợp truy xuất tài liệu và tổng hợp câu trả lời

Cách tiếp cận này giải quyết triệt để các vấn đề đã nêu ở phần trước:

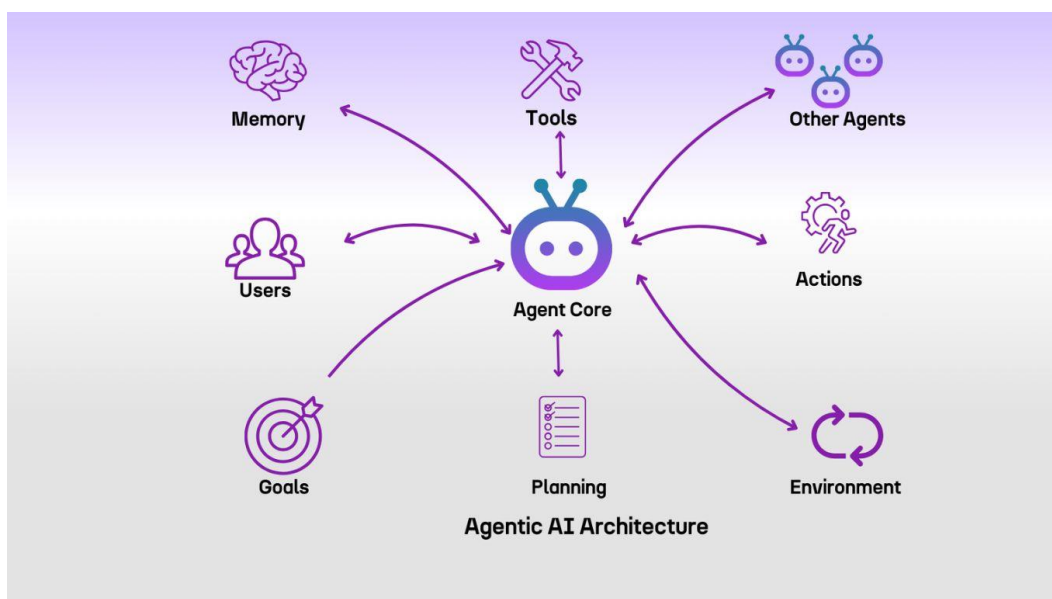
- Vượt qua giới hạn về miền kiến thức và dữ liệu riêng tư: Bằng cách kết nối LLM với cơ sở dữ liệu pháp luật bên ngoài, chatbot có thể truy cập và sử dụng nguồn dữ liệu riêng tư của hệ thống một cách an toàn và hiệu quả mà không cần tái huấn luyện toàn bộ mô hình tốn kém.
- Giảm thiểu hiện tượng ảo giác: Vì LLM được yêu cầu phải trả lời dựa trên thông tin thực tế được cung cấp trong prompt, khả năng "bịa" ra thông tin sai lệch bị triệt tiêu đáng kể. Câu trả lời được "neo" vào điều luật cụ thể trong cơ sở dữ liệu, từ đó tăng cường độ tin cậy và chính xác.

Mặc dù vậy, chatbot dựa trên LLM và RAG cơ bản (Naive RAG) vẫn còn bị hạn chế: chỉ thực hiện một lần tìm kiếm cố định, không xử lý được câu hỏi đa chiều đòi hỏi tổng hợp từ nhiều văn bản, không có cơ chế tự đánh giá chất lượng kết quả, và không phát hiện được xung đột thời gian hiệu lực giữa các văn bản pháp luật.

1.2.3. Xây dựng hệ thống chatbot dựa trên AI Agent

Phương pháp xây dựng chatbot dựa trên AI Agent là một kiến trúc tiên tiến, ra đời như sự kết hợp của những phương pháp xây dựng chatbot khác cùng với khả năng tự suy luận, chủ động tương tác với người dùng.

AI Agent là kiến trúc bao gồm các thành phần suy luận, tạo sinh, thành phần tương tác với các công cụ bên ngoài (cơ sở dữ liệu pháp luật, API tìm kiếm web,...). Nhờ đó, AI Agent có khả năng thấu hiểu câu hỏi người dùng, lên kế hoạch tra cứu, sử dụng công cụ, tự đánh giá và điều chỉnh chiến lược qua nhiều bước - tương tự cách một chuyên gia pháp lý thực thụ làm việc.



Hình 1.3. AI Agent bao gồm nhiều thành phần tư duy, truy vấn và tương tác

Như vậy, mô hình AI Agent không chỉ tận dụng được sức mạnh ngôn ngữ của LLM mà còn khắc phục được những điểm yếu chí mạng của chúng, tạo ra một thể hệ chatbot thông minh, am hiểu sâu sắc về lĩnh vực pháp luật và đáng tin cậy cho người dùng. Sự kết hợp giữa AI Agent và RAG tạo ra Agentic RAG - kiến trúc được lựa chọn cho hệ thống Vietnam Law Chatbot của đề tài này.

1.3. Tìm hiểu về mô hình ngôn ngữ lớn

1.3.1. Khái niệm mô hình ngôn ngữ lớn

Mô hình ngôn ngữ lớn (Large Language Models - LLM) là những mô hình AI được thiết kế để xử lý và tạo sinh ngôn ngữ tự nhiên. Được xây dựng dựa trên kiến trúc mạng nơ-ron sâu, đặc biệt là mô hình Transformer, LLM được đào tạo trên lượng dữ liệu khổng lồ và cho ra mô hình lên tới hàng chục tỷ tham số. Nhờ đó,

LLM có khả năng hiểu ngôn ngữ tự nhiên, trả lời các câu hỏi, dịch văn bản, lập luận logic,...

LLM không chỉ học cách xử lý các mẫu ngôn ngữ phức tạp mà còn nắm bắt được ngữ cảnh, ngữ nghĩa, và cả bối cảnh văn hoá trong nhiều ngôn ngữ khác nhau. Nhờ khả năng tạo sinh ngôn ngữ tự nhiên mạnh mẽ, LLM mở ra tiềm năng vô hạn cho các ứng dụng AI, điển hình trong đó là các ứng dụng chatbot hay Chat LLMs.

Để LLM có thể đưa ra output chính xác, chúng hoạt động dựa trên mạng nơ-ron sâu, thường là kiến trúc Transformer. Quá trình này bao gồm các bước chính:

- Mã hoá đầu vào (Input Encoding / Embedding Layer): Khi nhận văn bản đầu vào, LLM chuyển đổi các từ (tokens) thành các biểu diễn số học dưới dạng vector, gọi là "embedding". Lớp nhúng này nắm bắt ý nghĩa ngữ nghĩa và cú pháp của từ, cho phép mô hình xử lý ngôn ngữ theo phương pháp toán học.
- Xử lý qua các lớp Transformer: Dữ liệu sau khi nhúng đi qua các lớp của kiến trúc Transformer.
- Lớp truyền tiếp (Feedforward Layers): Các lớp này thực hiện các biến đổi phi tuyến tính trên các vector embedding, giúp mô hình học và tạo ra các liên kết phức tạp giữa dữ liệu đầu vào và đầu ra, góp phần tạo ra văn bản phù hợp với ngữ cảnh.
- Cơ chế chú ý (Attention Mechanism): Đây là một thành phần cốt lõi, đặc biệt trong kiến trúc Transformer. Cơ chế này cho phép mô hình tập trung có chọn lọc vào các phần quan trọng nhất của văn bản đầu vào khi thực hiện dự đoán. Bằng cách gán trọng số cho mỗi phần dữ liệu đầu vào, mô hình có thể xác định mức độ liên quan của từng yếu tố, từ đó tạo ra văn bản mạch lạc và bám sát yêu cầu của người dùng. Một số LLM có thể sử dụng Lớp tuần hoàn (Recurrent Layers) để xử lý dữ liệu theo trình tự, duy trì thông tin từ các bước trước đó, rất hữu ích cho việc duy trì tính liên tục trong hội thoại.
- Dự đoán đầu ra (Output Prediction): Dựa trên thông tin đã được xử lý và hiểu biết về ngữ cảnh, LLM sẽ dự đoán từ hoặc chuỗi từ tiếp theo để tạo thành câu trả lời hoặc nội dung.
- Chất lượng của dữ liệu huấn luyện đóng vai trò cực kỳ quan trọng; dữ liệu càng lớn và càng chất lượng thì LLM học ngôn ngữ tự nhiên càng tốt, từ đó đưa ra output chính xác hơn. Quá trình huấn luyện thường bao gồm giai đoạn học không giám sát trên dữ liệu lớn để xây dựng kiến thức nền tảng, sau đó có thể được tinh chỉnh (fine-tuning) cho các tác vụ cụ thể. Trong hệ thống

Vietnam Law Chatbot, temperature được đặt thấp (0.1) để đảm bảo câu trả lời pháp lý nhất quán và hạn chế sự ngẫu nhiên.

1.3.2. Khái niệm Chat LLMs

Chatbot AI hay Chat LLMs là những ứng dụng chat dựa trên LLM, cho phép người dùng tương tác với các mô hình ngôn ngữ lớn dưới dạng các đoạn hội thoại gần gũi. Nhờ cách tương tác tiện dụng, Chatbot AI cho phép người dùng phổ thông dễ dàng sử dụng, đồng thời có thể triển khai rộng rãi trên nhiều loại thiết bị khác nhau. Nhóm hệ thống này ngày càng được ứng dụng trong nhiều lĩnh vực như giáo dục, hỗ trợ khách hàng, tra cứu tri thức và tư vấn nghiệp vụ.

Ưu điểm và hạn chế của Chat LLMs

- **Ưu điểm:**
 - Khả năng giao tiếp tự nhiên: LLM có thể hiểu và tạo ra văn bản giống như con người, giúp tương tác trở nên trực quan và dễ dàng hơn.
 - Truy cập thông tin nhanh chóng: Có thể xử lý và tóm tắt lượng lớn thông tin từ nhiều nguồn khác nhau, cung cấp câu trả lời nhanh cho nhiều loại câu hỏi.
 - Hỗ trợ đa dạng các tác vụ: Viết lách, dịch thuật, tóm tắt văn bản, lập trình, phân tích dữ liệu và nhiều công việc khác.
 - Cá nhân hoá trải nghiệm người dùng: Một số LLM có khả năng học hỏi từ các tương tác trước đó để đưa ra phản hồi phù hợp hơn với từng người dùng.
 - Giúp nâng cao hiệu suất công việc: Bằng cách tự động hóa một số tác vụ và cung cấp thông tin nhanh chóng, LLM giúp người dùng tiết kiệm thời gian và tăng năng suất.
 - Thao tác với đa dạng tài nguyên: LLM có thể thao tác với chữ viết, hình ảnh, âm thanh, video,...
- **Nhược điểm:**
 - Nguy cơ về lạm dụng: LLM có thể bị sử dụng cho các mục đích xấu như tạo tin giả, lừa đảo, tấn công mạng hoặc thao túng dư luận.
 - Sự phụ thuộc quá mức: Người dùng có thể trở nên quá phụ thuộc vào LLM, làm giảm khả năng tư duy độc lập và các kỹ năng giải quyết vấn đề của bản thân.
 - Rủi ro về bảo mật: Việc chia sẻ thông tin nhạy cảm với LLM có thể dẫn đến nguy cơ rò rỉ dữ liệu nếu không được quản lý cẩn thận.

- Hiện tượng ảo giác (Hallucination): LLM có thể "bịa" ra thông tin không chính xác nhưng lại trình bày một cách rất thuyết phục. Điều này đặc biệt nguy hiểm khi người dùng tin tưởng hoàn toàn vào câu trả lời của chúng.

1.4. Tìm hiểu về AI Agent

1.4.1. Khái niệm AI Agent

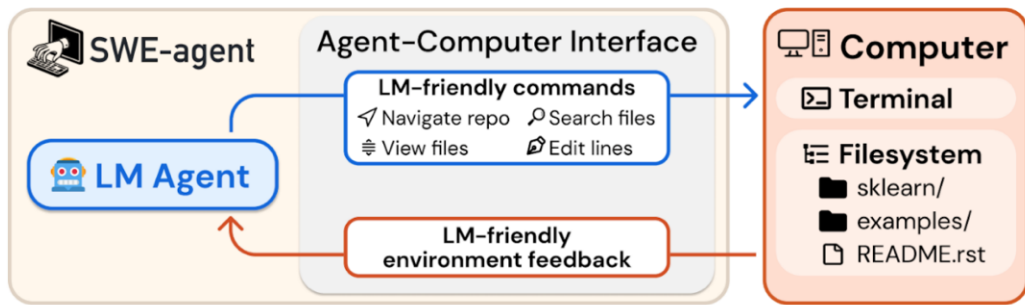
Một tác tử (Agent) là bất cứ thứ gì có khả năng nhận thức được môi trường xung quanh và hành động theo môi trường đó. Cuốn sách Artificial Intelligence: A Modern Approach (1995) định nghĩa một tác tử là bất cứ thứ gì có thể xem là có khả năng **nhận thức (perceive)** môi trường của nó thông qua các **cảm biến (sensor)** và tác động lên môi trường đó thông qua các **bộ truyền động (actuators)**. “Bộ truyền động” là khái niệm chỉ bất kỳ cách thức, công cụ, biện pháp nào giúp Agent tác động lên môi trường xung quanh nó.

Một Agent được đặc trưng bởi môi trường mà nó hoạt động và tập hợp các hành động mà Agent có thể tác động lên môi trường đó. Ví dụ, một Agent được xây dựng để hỗ trợ tra cứu pháp luật thì "hệ thống pháp luật Việt Nam" (bao gồm CSDL nội bộ và web) sẽ là môi trường của Agent và tác động của Agent lên môi trường sẽ là các hành động "tìm kiếm điều luật", "đối chiếu văn bản", "tổng hợp câu trả lời".

Tác tử thông minh (AI Agent) là Agent được xây dựng trên nền tảng các kỹ thuật AI. Môi trường của AI Agent là bối cảnh, yêu cầu, tài liệu,... được đưa vào bởi người dùng hoặc các tài nguyên hệ thống khác. Các tác động của AI Agent lên môi trường được tăng cường bởi các công cụ mà nó có quyền truy cập.

AI Agent đại diện cho sự phát triển đáng kể trong AI, thể hiện sự thay đổi mô hình trong cách các hệ thống thông minh tương tác với môi trường, đưa ra quyết định và đạt được các mục tiêu phức tạp.

Hình sau đây mô tả một AI Agent có tên là SWE-Agent, được xây dựng trên nền tảng mô hình GPT-4. Môi trường của Agent này là máy vi tính với hệ thống tập tin, công cụ dòng lệnh. Tập hành động bao gồm các thao tác lưu trữ, duyệt, xem tập tin.



Hình 1.4. Mô hình SWE AI Agent

Khi áp dụng vào xây dựng chatbot pháp luật, AI Agent cho phép chatbot thấu hiểu ngữ cảnh câu hỏi và đưa ra những hành động cụ thể để thực hiện yêu cầu - tra cứu điều luật liên quan, đối chiếu văn bản cũ mới, xác minh tính chính xác - trước khi trả lời người dùng.

1.4.2. Các thành phần của AI Agent

Kiến trúc của AI Agent đại diện cho sự tích hợp tinh vi của nhiều thành phần cho phép Agent có khả năng nhận thức, tư duy, lên kế hoạch và thực hiện hành động. AI Agent bao gồm các thành phần sau đây.

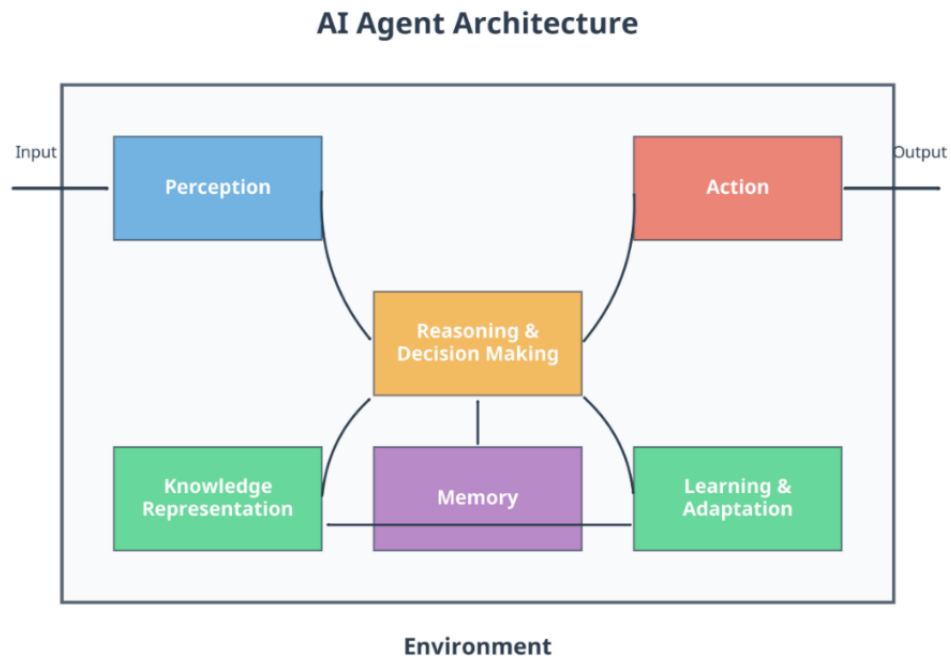


Figure 1: Core components and information flow in modern AI agent architecture

Hình 1.5. Sơ đồ kiến trúc các thành phần của AI Agent

1.4.2.1. Cơ chế nhận thức – Perception

Cơ chế nhận thức là giao diện cho phép thu thập và xử lý thông tin bên ngoài từ môi trường. Trong các Agent dựa trên ngôn ngữ, nhận thức chủ yếu bao gồm các mô-đun hiểu ngôn ngữ tự nhiên (NLU), trích xuất các thực thể và ý định liên quan, chuyển đổi các văn bản phi cấu trúc thành các đối tượng có cấu trúc để xử lý tiếp. Trong các tác nhân hiện thực hóa, nhận thức có thể bao gồm thị giác máy tính, nhận dạng giọng nói, xử lý dữ liệu cảm biến, v.v. Các AI Agent dựa trên thông tin chúng nhận thức được để hành động, làm cho khả năng nhận thức mạnh mẽ trở thành nền tảng cho hoạt động hiệu quả của Agent.

1.4.2.2. Hệ thống biểu diễn tri thức – Knowledge representation systems

Các hệ thống này phải cân bằng nhiều yêu cầu, bao gồm **tính biểu đạt** (khả năng biểu diễn các loại tri thức đa dạng), **hiệu quả tính toán** (cho phép truy cập và thao tác nhanh chóng), và **khả năng học hỏi** (hỗ trợ việc tích hợp thông tin mới). Các kiến trúc Agent hiện đại thường sử dụng các phương pháp biểu diễn tri thức lai kết hợp các cấu trúc ký hiệu (bản thể luận, đồ thị tri thức, các khẳng định logic) với các biểu diễn phân tán (nhúng vector, các kích hoạt mạng nơ-ron). Sự tích hợp này cho phép các tác nhân tận dụng cả tính chính xác của suy luận ký hiệu và khả năng nhận dạng mẫu của các phương pháp vector, nơ-ron.

Một Agent hiệu quả cần phải đảm bảo phân biệt giữa nhiều dạng tri thức khác nhau bao gồm:

- **Context:** Sự thật về thế giới xung quanh.
- **Task:** Các nhiệm vụ cụ thể cần thực hiện.
- **Steps:** Các tri thức thủ tục.
- **Experiments:** Những trải nghiệm thực tế.
- **Meta data:** Siêu tri thức - tự nhận thức về vai trò, khả năng và hạn chế của chính Agent.

1.4.2.3. Mô-đun suy luận và ra quyết định – Reasoning & Decision Making

Các mô-đun suy luận và ra quyết định cho phép Agent xử lý thông tin có sẵn, đánh giá các phương án thay thế và lựa chọn hành động phù hợp. Các mô-đun này có nhiệm vụ lập kế hoạch để Agent thực hiện. Một ví dụ tiêu biểu cho khả năng Reasoning trong AI Agent là các mô hình ngôn ngữ lớn hiện đại có khả năng phân tích ngữ cảnh, lập luận nhiều bước và đưa ra quyết định nhằm hỗ trợ giải quyết các bài toán phức tạp trong nhiều lĩnh vực khác nhau.

1.4.2.4. Thành phần thực thi – Action

Action là các thành phần chuyển đổi các quyết định thành các hành vi cụ thể tác động đến môi trường. Action có thể bao gồm:

- Tạo phản hồi.
- Gọi API.
- Gọi các công cụ.
- Đặt câu hỏi làm rõ cho người dùng.

Như Microsoft giải thích, các tác nhân cần quyền truy cập vào các chương trình máy tính mà chúng cần để thực hiện hành động thay mặt bạn, như Teams và PowerPoint. Sự tích hợp này với các công cụ và hệ thống bên ngoài giúp mở rộng đáng kể phạm vi hành động có sẵn cho các tác nhân, cho phép chúng hoàn thành các tác vụ phức tạp mà không thể thực hiện chỉ bằng việc tạo ngôn ngữ.

1.4.2.5. Các thành phần khác

Ngoài các thành phần chính, AI Agent hiện đại thường bao gồm một số mô-đun và công cụ khác như:

- **Mô-đun quản lý bộ nhớ:** Duy trì thông tin qua nhiều tương tác và thang thời gian, cho phép tác nhân học hỏi từ kinh nghiệm và duy trì ngữ cảnh.
- **Thành phần tự phản chiếu (Reflection) và sửa lỗi:** Cho phép tác nhân đánh giá hiệu suất của chính mình, nhận ra các hạn chế và điều chỉnh phương pháp tiếp cận cho phù hợp. Những khả năng này rất cần thiết cho hoạt động mạnh mẽ trong các môi trường phức tạp, cho phép tác nhân phát hiện khi chúng thiếu thông tin cần thiết, khi độ tin cậy của chúng vào các kết luận cụ thể thấp, hoặc khi chúng cần xem xét lại các quyết định trước đó dựa trên thông tin mới.
- **Cơ chế an toàn và điều chỉnh:** Đảm bảo rằng hành vi của tác nhân vẫn nằm trong giới hạn phù hợp với các giá trị và ý định của con người. Các cơ chế này có thể bao gồm các ràng buộc rõ ràng về lựa chọn hành động, quy trình xác minh cho các quyết định quan trọng, các tính năng diễn giải giúp con người xem xét lý luận của tác nhân và các kỹ thuật điều chỉnh giá trị để hiệu chỉnh mục tiêu của tác nhân với sở thích của con người.

1.4.3. Tool trong Agent

Một Agent không nhất thiết truy cập vào các công cụ bên ngoài. Tuy nhiên, nếu không có các công cụ bên ngoài, khả năng của Agent sẽ bị hạn chế. Bản thân

một mô hình LLM thường chỉ có thể thực hiện một hành động - một LLM có thể tạo văn bản. Các công cụ bên ngoài làm cho Agent có khả năng hơn rất nhiều, cho phép vừa tạo văn bản vừa truy cập vào các dữ liệu pháp luật nội bộ và web.

Các công cụ giúp Agent vừa nhận biết môi trường vừa hành động dựa trên nó. Các hành động cho phép Agent nhận biết môi trường là hành động đọc, trong khi các hành động cho phép Agent tác động lên môi trường là hành động ghi.

Ví dụ: Một Agent tư vấn pháp luật có thể có tool để truy cập vào ChromaDB nhằm lấy các điều luật liên quan (đọc), vừa truy cập vào tool tìm kiếm web Tavily để lấy văn bản pháp luật mới ban hành (đọc), vừa tổng hợp và streaming kết quả về giao diện người dùng (ghi).

Các tác nhân thường lựa chọn công cụ để sử dụng trong quá trình lên kế hoạch thực hiện. Việc lựa chọn công cụ nào sẽ phụ thuộc vào mô tả về khả năng, phạm vi, ... của công cụ đó.

1.4.4. Planning trong Agent

Planning là cốt lõi của một AI Agent tốt. Các AI Agent có thể phải giải quyết những nhiệm vụ rất phức tạp, ví dụ: *"Mức phạt vượt đèn đỏ bằng xe máy theo quy định hiện hành là bao nhiêu và có thay đổi so với Nghị định 100/2019 không?"*. Một nhiệm vụ được xác định bởi mục tiêu và các ràng buộc của nó. Trong ví dụ trên, mục tiêu là thông tin về mức phạt và sự thay đổi theo thời gian, ràng buộc là loại phương tiện và văn bản pháp luật cụ thể.

Các nhiệm vụ phức tạp đòi hỏi phải lập kế hoạch. Đầu ra của quá trình lập kế hoạch là một kế hoạch, tức là một lộ trình vạch ra các bước cần thiết để hoàn thành nhiệm vụ. Đối với một nhiệm vụ bất kỳ, để lập kế hoạch tốt, cần thực hiện:

- Hiểu nhiệm vụ cần thực hiện.
- Xem xét tài nguyên và công cụ có sẵn.
- Lập ra các hướng giải quyết khác nhau và lựa chọn hướng đi tốt nhất.

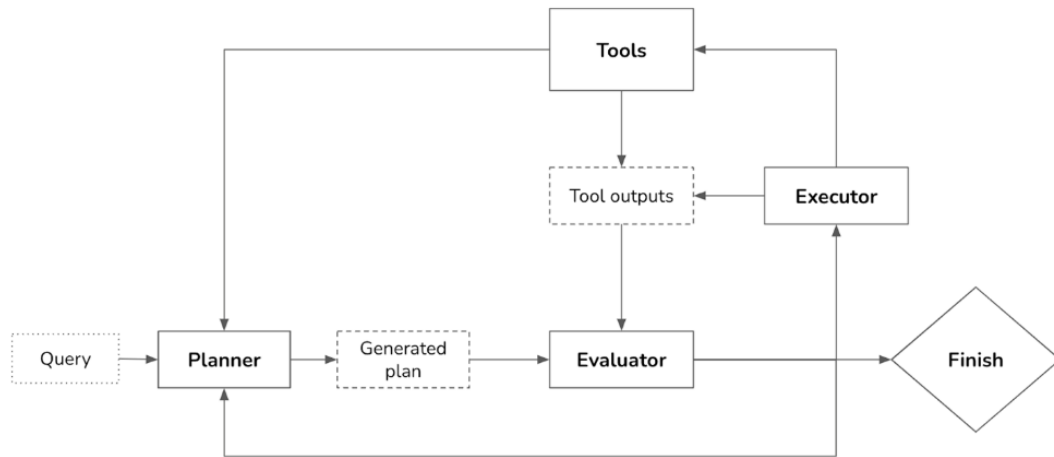
Đối với AI Agent cũng tương tự, Agent sẽ thực hiện:

- Hiểu các nhiệm vụ cần thực hiện.
- Xem xét các Tool và Knowledge có sẵn.
- Lựa chọn plan tốt nhất.

Tuy nhiên, trong quá trình Planning, Agent hoàn toàn có thể mắc sai lầm và đưa ra một kế hoạch quá dài dòng mà thậm chí không đạt được mục đích. Do đó, việc lập kế hoạch thường được tách riêng ra khỏi việc thực thi (thành các mô-đun

riêng chẳng hạn) và sau khi thực thi một số bước, AI-Agent có thể sẽ cần phải có những bước đánh giá tình hình và tiến hành re-plan nếu cần thiết:

- Nếu kế hoạch là hợp lý và có thể thực hiện tiếp: tiếp tục thực thi.
- Nếu kế hoạch không còn hợp lý: tiến hành lập kế hoạch lại dựa trên dữ liệu thu hoạch được từ các bước thực thi trước đó.



Hình 1.6. Mô hình quá trình Planning và Re-planning của Agent

Trong hình minh hoạ bên trên, ta thấy Agent có quá trình Planner dựa trên đầu vào Query. Plan đã tạo sẽ được đánh giá, sau đó tiến hành thực hiện với các Tools và sau mỗi bước sử dụng Tool, ta sẽ lập kế hoạch lại và tiếp tục chu trình cho đến khi thu được kết quả cuối cùng.

1.4.5. Tự đánh giá và sửa lỗi

Như đã trình bày ở mục trước, ngay cả những kế hoạch tốt nhất cũng cần được đánh giá và điều chỉnh liên tục để tối đa hoá độ chính xác của kết quả. Quá trình tự đánh giá hay tự nhìn nhận này được gọi là **Reflection**. Trong lĩnh vực pháp luật, bước kiểm chứng này là thiết yếu vì thông tin sai có thể dẫn đến hậu quả pháp lý nghiêm trọng.

Mặc dù Reflection không hoàn toàn cần thiết để một Agent hoạt động, nhưng nó lại cần thiết để Agent đạt được độ chính xác cao nhất. Có nhiều thời điểm trong quá trình thực hiện tác vụ mà việc tự đánh giá có thể hữu ích:

- Sau khi nhận được truy vấn của người dùng: đánh giá xem yêu cầu có khả thi và trong phạm vi không.
- Sau khi tạo kế hoạch ban đầu để đánh giá xem kế hoạch có hợp lý hay không.
- Sau mỗi bước thực thi để đánh giá xem nó có đi đúng hướng hay không.

- Sau khi toàn bộ kế hoạch đã được thực thi để xác định xem tác vụ đã hoàn thành hay chưa.

Tự nhìn nhận và sửa lỗi là hai cơ chế khác nhau nhưng đi đôi với nhau. Tự nhìn nhận tạo ra những hiểu biết sâu sắc giúp phát hiện ra các lỗi cần sửa. Việc tự nhìn nhận có thể được thực hiện bởi cùng một Agent với các lời nhắc tự phê bình. Nó cũng có thể được thực hiện bởi một thành phần riêng biệt.

Lần đầu tiên được đề xuất trong bài báo "*ReAct: Synergizing Reasoning and Acting in Language Models*" (Yao et al., ICLR 2023), việc xen kẽ giữa suy luận (reasoning) và hành động (action) đã trở thành một khuôn mẫu phổ biến cho các Agent. Ở mỗi bước, Agent được yêu cầu giải thích suy nghĩ của mình (lập kế hoạch), thực hiện hành động, sau đó phân tích các kết quả đạt được (tự nhìn nhận). Các AI Agent thường được viết Prompt để tạo ra các output có dạng như sau:

```
Thought 1: [Suy luận về tình huống hiện tại]
Act 1:      [Thực thi hành động cụ thể]
Observation 1: [Quan sát kết quả trả về từ CSDL]
...
[Lặp lại cho đến khi đủ thông tin]
...
Thought N: [Suy luận cuối - tổng hợp câu trả lời]
Act N:
Finish [Câu trả lời hoàn chỉnh]
```

Ta thấy, trong output này đã bao gồm kế hoạch thực hiện (Thought 1, Thought 2,... Thought N), hành động tương ứng (Act 1, Act 2,... Act N) và quan sát (có thể bao gồm cả kết quả và đánh giá kết quả) của hành động (Observation 1, Observation 2,... Observation N).

Dưới đây là một ví dụ thực tế cho quá trình Planning-Reflection-Recorrect (thường gọi là ReAct Framework) để trả lời một câu hỏi có thể có nhiều hướng.

(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.

Act 1: `Search[Apple Remote]`

Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the Front Row media center program ...

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.

Act 2: `Search[Front Row]`

Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .

Act 3: `Search[Front Row (software)]`

Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.

Act 4: `Finish[keyboard function keys]`



Hình 1.7. Ví dụ áp dụng ReAct Agent để trả lời câu hỏi

Đây cũng là cơ sở cho tính năng **ThinkingPanel** trên ứng dụng di động của hệ thống - hiển thị 5 bước xử lý của pipeline theo thời gian thực, giúp người dùng hiểu được quá trình Agent đang suy luận để đưa ra câu trả lời.

1.4.6. Tích hợp RAG vào AI Agent - Agentic RAG

Sự kết hợp giữa kiến trúc RAG và AI Agent tạo ra **Agentic RAG** - bước tiến quan trọng so với Naive RAG. Thay vì một pipeline cố định (Encode → Retrieve → Generate), Agentic RAG giao quyền kiểm soát luồng xử lý cho Agent: Agent tự quyết định khi nào cần tra cứu, tra cứu nguồn nào (CSDL nội bộ hay web), có cần tra cứu thêm không, và khi nào đủ điều kiện để sinh câu trả lời cuối.

Trong đề tài, Agentic RAG được triển khai thông qua framework LangGraph với đồ thị gồm các bước chính: Guardrail → Query Analysis → Agent ↔ Tools → Verifier. Kiến trúc cụ thể được trình bày chi tiết ở Chương 2.

1.5. Các công nghệ hỗ trợ phát triển hệ thống

1.5.1. Ngôn ngữ lập trình Python

Python là một ngôn ngữ lập trình bậc cao, được Guido van Rossum phát triển và phát hành lần đầu vào năm 1991. Python nổi tiếng với cú pháp rõ ràng và dễ đọc, điều này giúp lập trình viên dễ dàng viết và duy trì mã nguồn. Ngôn ngữ này được thiết kế để khuyến khích việc viết mã có cấu trúc rõ ràng và logic, giúp giảm thiểu lỗi và tăng hiệu suất làm việc.

- **Đặc điểm nổi bật của Python**

- **Cú pháp đơn giản và dễ hiểu:** Python có cú pháp ngắn gọn, dễ học, dễ đọc và dễ viết. Cú pháp của Python gần gũi với ngôn ngữ tự nhiên, giúp người học lập trình nhanh chóng nắm bắt được cách viết mã.
- **Đa nền tảng:** Python có thể chạy trên nhiều hệ điều hành khác nhau như Windows, macOS, Linux, và các hệ điều hành dựa trên Unix. Điều này làm cho Python trở thành một lựa chọn phổ biến cho phát triển phần mềm đa nền tảng.
- **Thư viện phong phú và đa dạng:** Python có một hệ thống thư viện phong phú, phục vụ nhiều mục đích khác nhau từ phát triển web, khoa học dữ liệu, trí tuệ nhân tạo, học máy, đến xử lý hình ảnh và video. Các thư viện nổi bật bao gồm NumPy, Pandas, Matplotlib, Scikit-learn, TensorFlow, và Flask.
- **Cộng đồng lớn mạnh:** Python có một cộng đồng lập trình viên lớn mạnh và năng động. Cộng đồng này đóng góp vào việc phát triển các thư viện mới, giải đáp thắc mắc và chia sẻ kiến thức thông qua các diễn đàn, blog, và hội thảo. Điều này giúp người học và lập trình viên dễ dàng tìm kiếm hỗ trợ khi gặp khó khăn.

- **Ứng dụng của Python**

- **Khoa học dữ liệu và Học máy:** Python là ngôn ngữ hàng đầu trong lĩnh vực khoa học dữ liệu và học máy nhờ vào các thư viện như NumPy, Pandas, Matplotlib, Scikit-learn, và TensorFlow. Các công cụ này giúp dễ dàng xử lý, phân tích và trực quan hóa dữ liệu, cũng như phát triển các mô hình học máy phức tạp.
- **Phát triển web:** Python có nhiều framework mạnh mẽ như Django và FastAPI hỗ trợ phát triển các ứng dụng web. Các framework này cung

cấp các công cụ cần thiết để xây dựng từ các ứng dụng web nhỏ đến các hệ thống web lớn và phức tạp.

Python được sử dụng để xây dựng hai FastAPI microservices với các thư viện chuyên dụng cho AI gồm LangGraph (điều phối Agent), sentence-transformers (tạo vector embedding tiếng Việt) và thư viện giao tiếp với LLMs. Sự phong phú của hệ sinh thái Python trong lĩnh vực AI là lý do chính để lựa chọn ngôn ngữ này cho phần lõi xử lý của hệ thống.

1.5.2. LangGraph Framework

1.5.2.1. Khái niệm LangGraph

LangChain là framework Python mã nguồn mở được phát triển từ tháng 10 năm 2022, cung cấp các abstraction chuẩn hóa để kết nối LLM với nguồn dữ liệu và công cụ bên ngoài. Tuy nhiên, LangChain Chains chỉ hỗ trợ luồng xử lý tuyến tính (Directed Acyclic Graph - DAG): dữ liệu đi một chiều từ đầu đến cuối, không thể quay lại. Điều này phù hợp với Naive RAG, nhưng chưa đủ cho AI Agent cần thực hiện vòng lặp ReAct.

LangGraph là một framework được phát hành năm 2024, xây dựng trên nền tảng LangChain, chuyên dụng cho bài toán Agent có vòng lặp. LangGraph cung cấp khả năng xây dựng, quản lý các quy trình làm việc AI Agent phức tạp dưới dạng đồ thị có chu trình.

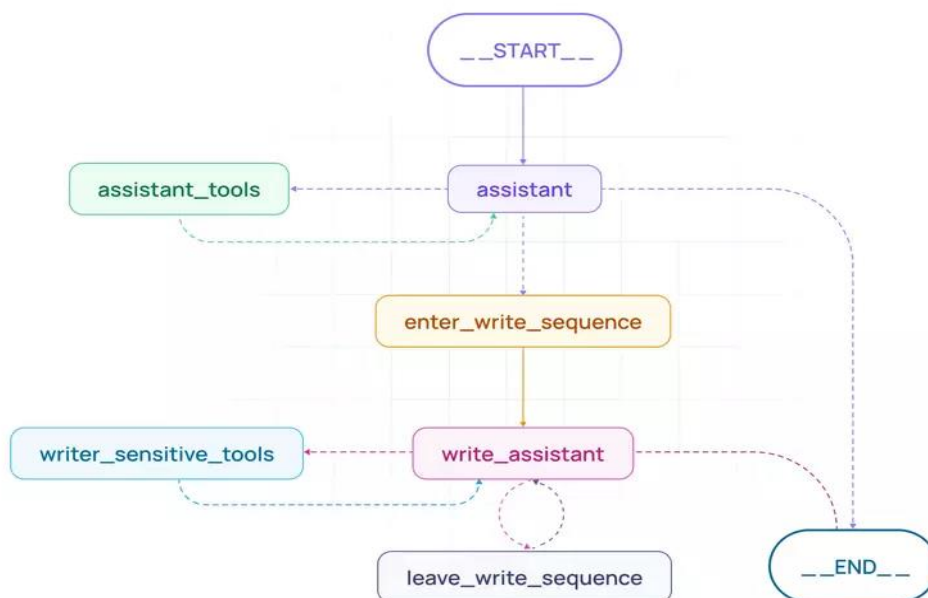
- **Ưu điểm:**
 - Hỗ trợ chu trình (cycles) - cho phép Agent quay lại node trước tùy theo kết quả quan sát.
 - Quản lý trạng thái tập trung - toàn bộ đồ thị chia sẻ một State duy nhất.
 - Hỗ trợ streaming output (SSE) từng sự kiện realtime về client.
 - Dễ gỡ lỗi - trạng thái tập trung cho phép xem đầy đủ diễn biến Agent tại mỗi node.
- **Nhược điểm:**
 - Đường cong học tập dốc hơn so với LangChain thuần.
 - Framework còn mới, một số tính năng nâng cao còn thiếu tài liệu chi tiết.

1.5.2.2. Các node và các cạnh trong LangGraph

Về bản chất, LangGraph sử dụng sức mạnh của các kiến trúc dựa trên đồ thị để mô hình hoá và quản lý các mối quan hệ phức tạp giữa các thành phần trong quy trình làm việc của AI Agent.

Thay vì sử dụng cấu trúc chain tuyến tính truyền thống, LangGraph cho phép thiết kế luồng hoạt động phức tạp hơn, tương tự như một mạng lưới với các node (nút) và edge (cạnh):

- **Nodes:** Đại diện cho các bước hoặc giai đoạn trong quá trình hoạt động của Agent. Mỗi node có thể là một task (công việc), state (trạng thái) hoặc action (hành động) cụ thể mà agent cần thực hiện. Ví dụ: "Chào người dùng", "Tìm kiếm thông tin", "Trả lời câu hỏi", v.v.
- **Edges:** Đại diện cho các kết nối giữa các node, xác định luồng di chuyển của agent. Edges có thể là direct (trực tiếp) hoặc conditional (có điều kiện), ví dụ "nếu điều kiện A đúng thì đi đến node X, ngược lại đi đến node Y".
- **Conditional edges:** Đại diện cho những cạnh rẽ nhánh đến các Nodes khác nhau dựa trên State của Agent hoặc một yếu tố nào đó khác. Conditional edges là thành phần quan trọng giúp quyết định điều hướng Agent và giúp Agent tránh khỏi vòng lặp vĩnh cửu.



Hình 1.8. Mô phỏng các Nút và các Cạnh trong LangGraph

Tính năng quan trọng nhất của LangGraph là khả năng tạo ra cycles (vòng lặp). Khác với chain truyền thống chỉ đi theo một chiều, LangGraph cho phép Agent quay lại các node đã truy cập, thực hiện lại các bước, tự điều chỉnh dựa trên kết quả và lặp lại cho đến khi đạt được mục tiêu.

Thực tế, nhiều tác vụ phức tạp đòi hỏi sự iterative (lặp đi lặp lại), dynamic (linh hoạt) và context-aware (nhận biết ngữ cảnh). LangGraph với cycles cung cấp khả năng xây dựng agent có thể:

- Xử lý các tình huống phức tạp, đa bước.
- Tương tác qua lại với người dùng (hoặc các agent khác) để thu thập thêm thông tin.
- Tự động điều chỉnh hành vi dựa trên kết quả và phản hồi.
- Thực hiện các quy trình lặp đi lặp lại để tối ưu hóa kết quả.

Ví dụ: Hãy tưởng tượng bạn xây dựng một chatbot dịch vụ khách hàng. Với chain, chatbot có thể chỉ trả lời được các câu hỏi đơn giản. Nhưng với LangGraph, chatbot có thể:

- Tiếp nhận câu hỏi của khách hàng. (Node 1)
- Phân tích câu hỏi và xác định thông tin còn thiếu. (Node 2)
- Đặt câu hỏi làm rõ yêu cầu với khách hàng. (Node 3) <- Cycle! Nếu thông tin chưa đủ, chatbot có thể quay lại Node 3 để hỏi tiếp.
- Tìm kiếm thông tin liên quan trong cơ sở dữ liệu. (Node 4)
- Tổng hợp thông tin và trả lời khách hàng. (Node 5)

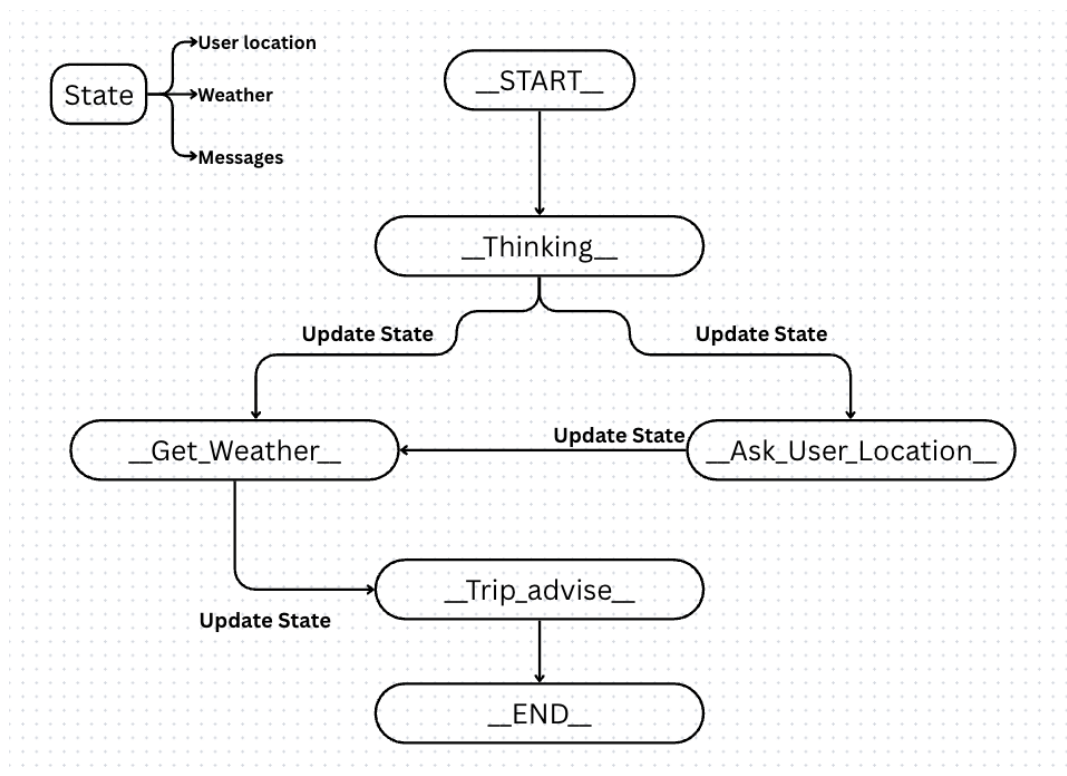
Như vậy, quy trình này có cycles (vòng lặp), cho phép chatbot tương tác và làm rõ yêu cầu của khách hàng cho đến khi hiểu đúng vấn đề và cung cấp câu trả lời chính xác nhất. Đây chính là ưu điểm nổi bật của LangGraph.

1.5.2.3. Quản lý trạng thái trong LangGraph

LangGraph làm sáng tỏ các quy trình trong luồng công việc AI, cho phép minh bạch hoàn toàn trạng thái (State) của tác nhân. Trong LangGraph, tính năng State đóng vai trò như một ngân hàng bộ nhớ ghi lại và theo dõi tất cả thông tin có giá trị được hệ thống AI xử lý. Nó tương tự như một sổ ghi chép kỹ thuật số, trong đó hệ thống ghi lại và cập nhật dữ liệu khi dữ liệu di chuyển qua các giai đoạn khác nhau của luồng công việc hoặc phân tích đồ thị.

State có thể là một chuỗi các tin nhắn, một mảng các số, một cấu trúc dữ liệu phức tạp,... State đóng vai trò như công cụ truyền đạt thông tin giữa các Node của

Agent. State này được cập nhật khi Agent chạy qua các Node, mỗi Node sẽ xử lý với State là đầu vào và cập nhật State ở đầu ra.



Hình 1.9. Ví dụ State trong Graph của một Agent về thông tin thời tiết

Trong hình trên là ví dụ nếu bạn đang chạy các Agent để theo dõi thời tiết, State có thể chứa các thông tin về thời tiết như là số lần tuyết rơi, cùng với thông tin về vị trí của người dùng. State này có thể được cập nhật khi di chuyển qua các Node như sau:

- Node `__Ask_User_Location__`: Node này lấy thông tin về vị trí của người dùng, do đó sẽ cập nhật vị trí này vào State
- Node `__Get_Weather__`: Node này lấy thông tin thời tiết dựa vào vị trí nên sẽ cập nhật thời tiết vào State
- Node `__Trip_advise__` sẽ dựa vào thông tin thời tiết đã cập nhật trong state để trả lời user

State là công cụ mạnh mẽ, truyền đạt ý chí, kế hoạch, yêu cầu, dữ liệu xuyên suốt quá trình hoạt động của Agent. Quản lý State là công việc quan trọng trong quá trình phát triển AI Agent với LangGraph. Quản lý trạng thái cũng rất hữu ích khi gỡ lỗi vì nó cho phép tập trung trạng thái của ứng dụng, do đó thường rút ngắn toàn bộ quy trình.

1.5.2.4. Các bước xây dựng AI Agent với LangGraph

Để xây dựng một AI Agent, ta sẽ đi qua các bước sau.

- Bước 1: Trước hết, để xây dựng AI Agent, ta cần xác định được các tính chất của Agent bao gồm:
 - Môi trường của Agent: Bối cảnh của Agent là gì, vai trò của Agent là gì, đối tượng của Agent là ai, dữ liệu mà Agent có thể tiếp cận là gì,...
 - Xác định miền tri thức của Agent: Agent có thể truy cập vào miền tri thức nào, giới hạn tri thức nằm ở đâu,...
 - Xác định tập hành động của Agent đối với môi trường: Agent có thể có hành động gì đối với môi trường xung quanh
- Bước 2: Sau khi đã định hình được tính chất của Agent, ta cần lên kế hoạch luồng hoạt động cho Agent:
 - Từ input của Agent tới output mong muốn, cần trả qua những bước gì
 - Những trường hợp nào có thể xảy ra trong quá trình thực hiện các task
- Bước 3: Từ workflow trên, ta đã có thể nắm được các Node trong đồ thị của agent.
- Bước 4: Dựa vào workflow, xác định các cạnh, các cạnh điều kiện của Agent
- Bước 5: Xây dựng State cho agent
- Bước 6: Xây dựng các Tool cho agent
- Bước 7: Triển khai

1.5.3. FastAPI Framework

FastAPI là framework phát triển API dựa trên ngôn ngữ Python, được xây dựng trên Starlette và Pydantic. Framework này hỗ trợ lập trình bất đồng bộ, khai báo kiểu dữ liệu rõ ràng và tự động kiểm tra dữ liệu đầu vào, do đó phù hợp với các hệ thống backend có nhiều luồng xử lý đồng thời.

Trong đề tài, FastAPI được sử dụng để xây dựng hai dịch vụ backend chính là main-service và rag-service. main-service đảm nhận các chức năng nghiệp vụ như xác thực người dùng, quản lý hội thoại, tra cứu thư viện pháp luật và tải lên văn bản. Trong khi đó, rag-service phụ trách các tác vụ liên quan đến Agentic RAG, truy xuất vector, gọi mô hình ngôn ngữ lớn và kiểm chứng câu trả lời.

Việc lựa chọn FastAPI phù hợp với kiến trúc của hệ thống vì các thao tác trong backend chủ yếu là tác vụ I/O-bound như truy vấn cơ sở dữ liệu, gọi dịch vụ RAG, kết nối ChromaDB và truyền phản hồi theo thời gian thực qua SSE. Ngoài ra,

khả năng tự sinh tài liệu OpenAPI từ khai báo schema giúp quá trình tích hợp giữa backend, ứng dụng di động và trang quản trị được nhất quán hơn.

1.5.4. Next.js 16

Next.js là framework phát triển ứng dụng web dựa trên React, hỗ trợ tổ chức giao diện theo cấu trúc route rõ ràng và tích hợp tốt với TypeScript. Trong đề tài, Next.js 16 được sử dụng để xây dựng trang quản trị hệ thống, phục vụ các chức năng như đăng nhập quản trị, tải lên văn bản pháp luật, theo dõi tiến trình xử lý dữ liệu và quan sát các chỉ số thống kê.

Đặc điểm phù hợp của Next.js trong hệ thống là khả năng tổ chức các màn hình quản trị thành những module độc lập như dashboard, danh sách tài liệu, chi tiết tài liệu và trang upload. Cách tổ chức này giúp mã nguồn giao diện dễ mở rộng, đồng thời phù hợp với đặc thù của một hệ thống quản trị dữ liệu pháp luật có nhiều nhóm chức năng khác nhau.

Bên cạnh đó, trang quản trị cần hiển thị trạng thái xử lý tài liệu theo thời gian thực thông qua WebSocket. Next.js kết hợp với React, TailwindCSS và shadcn/ui cho phép xây dựng giao diện có khả năng hiển thị bảng dữ liệu, biểu đồ thống kê, thanh tiến trình và thông báo trạng thái một cách thống nhất. Điều này phù hợp với yêu cầu vận hành và giám sát quy trình cập nhật tri thức pháp luật trong hệ thống.

1.5.5. Kotlin Multiplatform và Compose Multiplatform - Ứng dụng di động

Kotlin Multiplatform (KMP) là công nghệ cho phép chia sẻ mã nguồn Kotlin giữa nhiều nền tảng, đặc biệt là Android và iOS. Compose Multiplatform là framework giao diện khai báo, kế thừa tư tưởng thiết kế của Jetpack Compose, cho phép xây dựng giao diện người dùng bằng cùng một ngôn ngữ và cùng một mô hình lập trình.

Trong đề tài, KMP và Compose Multiplatform được sử dụng để phát triển ứng dụng Vietnam Law Chatbot trên thiết bị di động. Việc lựa chọn công nghệ này phù hợp với mục tiêu xây dựng ứng dụng đa nền tảng, trong khi vẫn giữ được sự thống nhất về tầng nghiệp vụ, tầng giao tiếp API, quản lý trạng thái và cách tổ chức giao diện.

Các thành phần như gọi API bằng Ktor, lưu trữ token, xử lý luồng SSE, render câu trả lời dạng Markdown, quản lý trạng thái theo mô hình MVI và điều hướng giữa các màn hình có thể được chia sẻ trong cùng một codebase. Nhờ đó, các chức năng

trọng tâm như hỏi đáp pháp luật, tư vấn có hướng dẫn, tra cứu thư viện văn bản và tìm kiếm ngữ nghĩa được triển khai đồng nhất trên cả Android và iOS.

Compose Multiplatform cũng phù hợp với đặc thù giao diện của ứng dụng vì nhiều màn hình có trạng thái thay đổi liên tục, chẳng hạn như quá trình nhận câu trả lời theo dòng sự kiện, bảng hiển thị tiến trình suy luận, danh sách hội thoại và bộ lọc văn bản pháp luật. Mô hình giao diện khai báo giúp trạng thái dữ liệu và giao diện được liên kết chặt chẽ hơn, phù hợp với kiến trúc MVI được áp dụng trong ứng dụng.

Trong đề tài, hệ thống sử dụng Kotlin 2.3.0 và Compose Multiplatform 1.10.0. Lựa chọn này giúp đồ án triển khai ứng dụng di động đa nền tảng nhưng vẫn giữ được sự nhất quán về kiến trúc, ngôn ngữ lập trình và cách tổ chức mã nguồn.

1.5.6. Hệ thống cơ sở dữ liệu đa mô hình (Polyglot Persistence)

Hệ thống Vietnam Law Chatbot áp dụng chiến lược Polyglot Persistence, tức là sử dụng nhiều loại cơ sở dữ liệu trong cùng một hệ thống. Cách tiếp cận này phù hợp với đề tài vì dữ liệu của hệ thống không đồng nhất: dữ liệu người dùng và hội thoại có tính giao dịch cao, dữ liệu văn bản pháp luật có cấu trúc tài liệu linh hoạt, còn dữ liệu phục vụ RAG lại cần lưu trữ và truy vấn vector embedding.

Thay vì cố gắng lưu toàn bộ dữ liệu vào một hệ quản trị duy nhất, hệ thống lựa chọn PostgreSQL, MongoDB và ChromaDB cho ba nhóm dữ liệu khác nhau. Mỗi cơ sở dữ liệu đảm nhận một vai trò rõ ràng, từ đó giúp kiến trúc lưu trữ vừa đảm bảo tính nhất quán cho nghiệp vụ người dùng, vừa đáp ứng được yêu cầu truy xuất văn bản pháp luật và tìm kiếm ngữ nghĩa.

1.5.6.1. PostgreSQL - Dữ liệu giao dịch

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ, phù hợp với các dữ liệu có cấu trúc rõ ràng, có quan hệ khóa ngoại và yêu cầu đảm bảo tính toàn vẹn. Trong hệ thống, PostgreSQL được sử dụng để lưu các dữ liệu giao dịch gồm users, conversations, messages, document_tasks và refresh_tokens.

Nhóm dữ liệu này cần đảm bảo các thao tác như đăng ký, đăng nhập, làm mới token, lưu hội thoại và ghi nhận tiến trình xử lý tài liệu được thực hiện nhất quán. Ví dụ, một tin nhắn phải gắn với đúng hội thoại, một refresh token phải gắn với đúng người dùng, và một tác vụ xử lý tài liệu phải có trạng thái rõ ràng trong suốt quá trình upload, parse và ingest. Do đó, mô hình quan hệ của PostgreSQL phù hợp để biểu diễn các ràng buộc dữ liệu này.

Bên cạnh đó, PostgreSQL hỗ trợ giao dịch ACID và truy vấn SQL mạnh, giúp hệ thống dễ dàng lọc, sắp xếp và thống kê dữ liệu nghiệp vụ. Trong đề tài, backend

sử dụng SQLAlchemy 2.0 kết hợp với asyncpg để làm việc với PostgreSQL theo hướng bất đồng bộ, phù hợp với kiến trúc FastAPI của Main Service.

1.5.6.2. MongoDB - Kho văn bản pháp luật

MongoDB là cơ sở dữ liệu NoSQL theo mô hình tài liệu, phù hợp với dữ liệu có cấu trúc linh hoạt và kích thước nội dung lớn. Trong hệ thống, MongoDB được sử dụng để lưu toàn văn các điều luật trong collection VietnamLawDB.articles.

Dữ liệu pháp luật không phải lúc nào cũng có cấu trúc đồng nhất. Mỗi điều luật có thể có tiêu đề, nội dung, số văn bản, năm ban hành, chủ đề, từ khóa, tóm tắt và các metadata khác nhau tùy theo loại văn bản. Mô hình document của MongoDB cho phép lưu mỗi điều luật như một tài liệu độc lập, trong đó phần nội dung và metadata có thể mở rộng linh hoạt mà không cần thay đổi schema quan hệ phức tạp.

Theo dữ liệu thực tế của đề tài, MongoDB đang lưu 528.620 điều luật trải dài từ năm 1945 đến năm 2026. Kho dữ liệu này phục vụ các chức năng duyệt thư viện pháp luật, xem chi tiết điều luật, tìm kiếm theo từ khóa, lọc theo năm ban hành và chủ đề. Driver motor được sử dụng để truy vấn MongoDB theo hướng bất đồng bộ, giúp Main Service xử lý tốt các thao tác đọc dữ liệu văn bản lớn.

1.5.6.3. ChromaDB - Kho vector embedding

ChromaDB là cơ sở dữ liệu vector, được sử dụng để lưu trữ embedding của các đoạn văn bản pháp luật sau khi đã được chia nhỏ. Đây là thành phần cần thiết cho các chức năng dựa trên tìm kiếm ngữ nghĩa, đặc biệt là Agentic RAG và AI-Powered Search.

Trong hệ thống, toàn văn điều luật từ MongoDB không được đưa trực tiếp vào mô hình ngôn ngữ lớn. Thay vào đó, văn bản được chia thành các đoạn nhỏ, mã hóa thành vector bằng mô hình embedding tiếng Việt và lưu trong collection vietnamese_law của ChromaDB. Khi người dùng đặt câu hỏi, hệ thống mã hóa câu hỏi thành vector và tìm các đoạn luật có ngữ nghĩa gần nhất thông qua cosine similarity.

Collection vietnamese_law hiện chứa 690.360 chunks, mỗi vector có 768 chiều và đi kèm metadata như law_id, article_id, year, topics, keywords. Các metadata này giúp hệ thống vừa truy xuất theo ngữ nghĩa, vừa giữ được liên kết ngược về văn bản pháp luật gốc để phục vụ trích dẫn nguồn. ChromaDB phù hợp với đồ án vì có thể triển khai cục bộ bằng Docker, tích hợp thuận tiện với Python/FastAPI và hỗ trợ chỉ mục HNSW cho truy vấn vector tốc độ cao.

1.6. Tổng kết chương 1

Chương 1 đã trình bày một cách tổng quan về hệ thống chatbot tư vấn pháp luật và các nền tảng lý thuyết cần thiết để hiểu kiến trúc của hệ thống Vietnam Law Chatbot. Nội dung chính bao gồm phần giới thiệu tổng quan về chatbot và tiềm năng ứng dụng trong lĩnh vực pháp luật Việt Nam, các phương pháp xây dựng chatbot từ tập luật đến LLMs+RAG và AI Agent. Bên cạnh đó, chương 1 đã trình bày chi tiết về Mô hình ngôn ngữ lớn (LLM) và cách lựa chọn mô hình phù hợp, về AI Agent với kiến trúc đầy đủ gồm Perception, Knowledge, Reasoning, Action và các cơ chế Planning, ReAct, Verifier - cùng khái niệm Agentic RAG kết hợp sức mạnh của cả hai.

Trong chương tiếp theo, báo cáo sẽ trình bày chi tiết phương pháp xây dựng hệ thống Vietnam Law Chatbot với AI Agent và Agentic RAG - bao gồm đồ thị LangGraph Guardrail → Query Analysis → Agent ↔ Tools → Verifier, thuật toán Two-stage Retrieval với Temporal Conflict Resolution phát hiện xung đột văn bản cũ/mới, và luồng Tư vấn có hướng dẫn (Guided Consultation) với SSE streaming.

CHƯƠNG 2: PHƯƠNG PHÁP XÂY DỰNG HỆ THỐNG TƯ VẤN PHÁP LUẬT VỚI AGENTIC RAG

Chương này trình bày phương pháp xây dựng hệ thống Vietnam Law Chatbot. Trọng tâm của chương là làm rõ cách tổ chức kiến trúc, cách hình thành kho dữ liệu vector pháp luật ban đầu, cách xác định tác nhân AI, phương pháp truy hồi tri thức, luồng tư vấn có hướng dẫn và quy trình cập nhật văn bản mới từ phía quản trị. Nội dung trong chương được trình bày theo hướng phương pháp luận: trước hết xác định mô hình tổng quát và nền tảng dữ liệu, sau đó phân tích các luồng xử lý chính và cuối cùng giải thích cách các lớp ứng dụng khai thác các luồng đó.

2.1. Kiến trúc tổng quan hệ thống Vietnam Law Chatbot

2.1.1. Lý do chọn kiến trúc *Microservices*

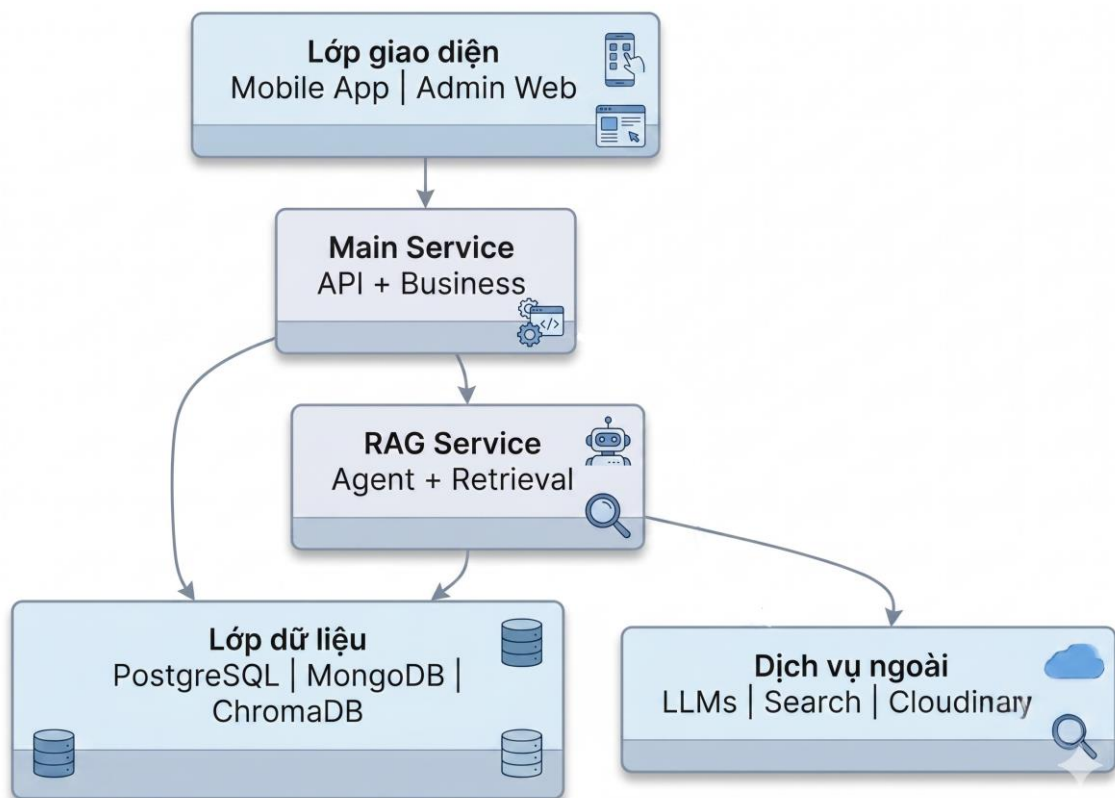
Hệ thống Vietnam Law Chatbot được xây dựng theo kiến trúc *Microservices*. Thay vì gom toàn bộ chức năng vào một ứng dụng duy nhất, hệ thống tách các nhóm nghiệp vụ thành những dịch vụ có vai trò riêng. Cách tổ chức này phù hợp với bài toán chatbot pháp luật vì hệ thống vừa cần xử lý các nghiệp vụ thông thường như xác thực, quản lý hội thoại, quản trị văn bản, vừa cần thực hiện các tác vụ AI có độ trễ và chi phí tính toán lớn hơn.

Lý do đầu tiên là yêu cầu tách riêng phần xử lý nặng. Các bước tạo embedding, truy hồi vector, xếp hạng lại kết quả và suy luận bằng LLMs thường mất nhiều thời gian hơn các thao tác nghiệp vụ thông thường. Khi đặt những bước này trong một dịch vụ riêng, các chức năng như đăng nhập, xem lịch sử hội thoại hoặc mở dashboard quản trị không bị phụ thuộc trực tiếp vào tải xử lý của pipeline AI.

Lý do thứ hai là bảo vệ phần lõi tri thức và suy luận. RAG Service chứa quy trình truy hồi, tổng hợp và kiểm chứng câu trả lời nên không nên được mở trực tiếp cho client. Người dùng cuối chỉ gửi yêu cầu đến Main Service, sau đó Main Service mới điều phối sang RAG Service khi cần xử lý câu hỏi pháp luật.

Lý do thứ ba là khả năng mở rộng và bảo trì. Main Service có nhịp thay đổi chủ yếu theo nghiệp vụ người dùng và quản trị, trong khi RAG Service có thể cần điều chỉnh thường xuyên hơn về chiến lược truy hồi, ngưỡng tin cậy, cách kiểm chứng hoặc cách sử dụng LLMs. Việc tách hai phần này giúp hệ thống dễ thử nghiệm và nâng cấp mà không làm ảnh hưởng toàn bộ ứng dụng.

2.1.2. Tổng quan kiến trúc bốn lớp



Hình 2.1. Kiến trúc tổng quan hệ thống Vietnam Law Chatbot

Dựa trên các yêu cầu trên, hệ thống được tổ chức thành bốn lớp. Lớp giao diện phục vụ hai nhóm người dùng khác nhau; lớp dịch vụ backend chịu trách nhiệm điều phối nghiệp vụ; lớp dữ liệu lưu trữ thông tin lâu dài; lớp dịch vụ bên ngoài cung cấp các năng lực hỗ trợ như LLMs, tìm kiếm cập nhật và lưu trữ file.

- **Lớp giao diện người dùng (Frontend):**

- Mobile App (Kotlin Multiplatform): ứng dụng dành cho người dùng cuối, chạy trên Android và iOS từ một nền tảng mã nguồn chung. Ứng dụng gửi yêu cầu tới backend qua HTTPS/JWT và nhận câu trả lời theo dòng sự kiện thời gian thực.
- Admin Web (Next.js 16): dashboard quản trị dành cho người quản trị hệ thống. Giao tiếp qua HTTPS/JWT và theo dõi tiến trình xử lý tài liệu qua WebSocket.

- **Lớp dịch vụ backend:**

- Main Service: cửa ngõ duy nhất tiếp xúc với frontend. Đảm nhận xác thực, quản lý hội thoại, tra cứu luật, upload tài liệu và điều phối gọi sang RAG Service.

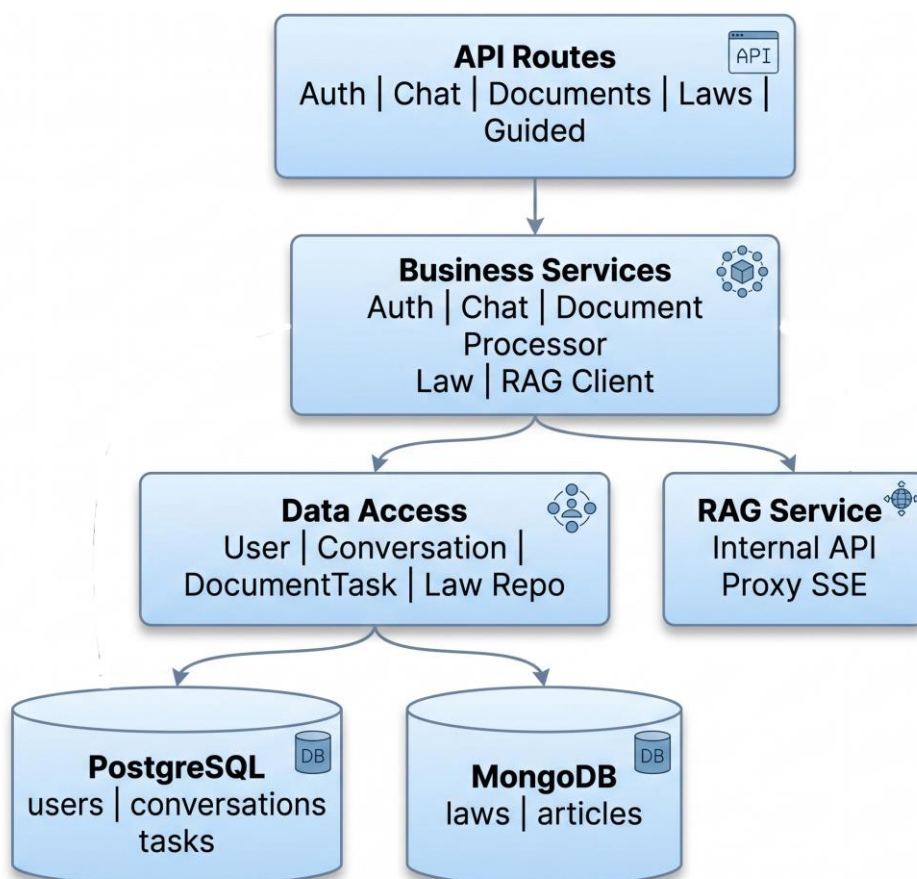
- RAG Service: dịch vụ nội bộ phụ trách truy hồi tri thức, suy luận bằng LLMs, kiểm chứng câu trả lời và xử lý cập nhật văn bản.
- **Lớp dữ liệu (3 cơ sở dữ liệu, phân tích chi tiết ở chương 3):**
 - PostgreSQL: dữ liệu quan hệ (người dùng, hội thoại, tin nhắn).
 - MongoDB: kho văn bản pháp luật (528,620 điều luật).
 - ChromaDB: kho vector phục vụ tìm kiếm ngữ nghĩa trên các đoạn văn bản đã được chia nhỏ.
- **Lớp dịch vụ bên ngoài:** nhà cung cấp LLMs, dịch vụ tìm kiếm web, dịch vụ đối chiếu thông tin cập nhật và kho lưu trữ file PDF.

Nhìn vào mô hình tổng quan, luồng tương tác của hệ thống đi từ giao diện đến Main Service, sau đó mới được chuyển tiếp sang RAG Service hoặc các kho dữ liệu phù hợp. Cách đi này giúp hệ thống có một điểm kiểm soát chung cho xác thực, phân quyền, ghi nhận lịch sử và điều phối tác vụ.

Lớp điều phối nghiệp vụ không trực tiếp thay thế các dịch vụ phía sau, mà đóng vai trò trung gian để giảm độ phức tạp cho frontend. Mobile App và Admin Web không cần biết chi tiết hệ thống đang sử dụng bao nhiêu cơ sở dữ liệu hoặc bao nhiêu bước AI, mà chỉ cần làm việc với các chức năng nghiệp vụ đã được Main Service chuẩn hóa.

Lớp dữ liệu nền bao gồm dữ liệu người dùng, hội thoại, văn bản pháp luật và vector tri thức. Các dịch vụ bên ngoài chỉ đóng vai trò bổ sung năng lực cho pipeline, ví dụ hỗ trợ suy luận ngôn ngữ, đối chiếu thông tin cập nhật hoặc lưu trữ file gốc của văn bản pháp luật.

2.1.3. Thành phần Main Service



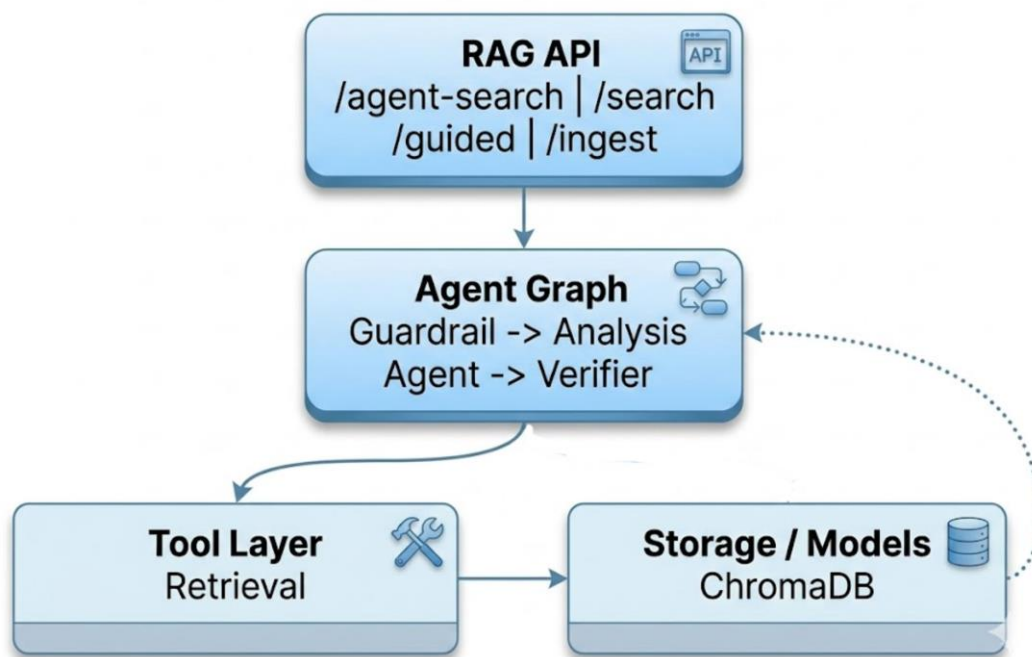
Hình 2.2. Sơ đồ thành phần Main Service

Main Service là cửa ngõ nghiệp vụ của hệ thống. Thành phần này tiếp nhận yêu cầu từ Mobile App và Admin Web, kiểm tra quyền truy cập, ghi nhận các dữ liệu cần thiết và chuyển yêu cầu đến nhóm xử lý phù hợp. Đối với người dùng cuối, Main Service quản lý hội thoại, gửi câu hỏi sang RAG Service và lưu lại câu trả lời. Đối với quản trị viên, Main Service tiếp nhận văn bản mới, tạo tác vụ xử lý và theo dõi tiến trình cập nhật dữ liệu.

Vai trò của Main Service là chuẩn hóa các thao tác từ nhiều giao diện thành các luồng nghiệp vụ thống nhất. Một yêu cầu chat và một yêu cầu cập nhật văn bản có bản chất rất khác nhau, nhưng đều cần được xác thực, ghi nhận trạng thái và điều phối qua các thành phần phía sau.

Việc đặt Main Service ở giữa cũng giúp giảm sự phụ thuộc của frontend vào cấu trúc nội bộ. Nếu sau này thay đổi cách truy hồi, thay đổi kho vector hoặc bổ sung bước kiểm chứng mới, lớp giao diện vẫn có thể giữ nguyên luồng tương tác chính với người dùng.

2.1.4. Thành phần RAG Service



Hình 2.3. Sơ đồ thành phần RAG Service

RAG Service là thành phần quyết định chất lượng trả lời pháp luật. Dịch vụ này không chỉ gọi LLMs để tạo sinh văn bản, mà tổ chức cả quá trình phân tích câu hỏi, truy hồi căn cứ, đối chiếu nguồn và kiểm chứng nội dung trước khi trả về kết quả cho Main Service.

Trong miền pháp luật, câu trả lời cần dựa trên văn bản có thể kiểm tra lại. Vì vậy, RAG Service được thiết kế để LLMs làm việc với các nguồn đã truy hồi thay vì trả lời hoàn toàn từ tri thức sẵn có. Đây là điểm khác biệt quan trọng giữa một chatbot thông thường và hệ thống tư vấn pháp luật có căn cứ.

Có thể xem RAG Service là nơi kết hợp ba nhóm kỹ thuật: truy hồi thông tin để tìm nguồn, suy luận ngôn ngữ để diễn giải nội dung, và kiểm chứng để giảm sai lệch ở đầu ra. Các phần tiếp theo của chương sẽ lần lượt phân tích cách hệ thống tổ chức ba nhóm kỹ thuật này.

2.1.5. Kênh giao tiếp giữa các thành phần

Sau khi xác định các lớp kiến trúc, cần làm rõ cách các thành phần trao đổi dữ liệu với nhau. Trong hệ thống này, kênh giao tiếp không chỉ phục vụ truyền dữ liệu

mà còn thể hiện ranh giới trách nhiệm giữa các dịch vụ. Thành phần giao diện chỉ giao tiếp với Main Service, còn RAG Service và các kho dữ liệu được đặt phía sau lớp điều phối.

Bảng 2.1 tổng hợp các kênh giao tiếp chính, bao gồm hướng kết nối, giao thức sử dụng, cơ chế xác thực và vai trò của từng kênh. Việc tách rõ các kênh này giúp hệ thống dễ kiểm soát bảo mật hơn, đồng thời giúp quá trình triển khai và mở rộng từng dịch vụ độc lập hơn.

Bảng 2.1. Đặc tả các kênh giao tiếp trong hệ thống

Kênh	Từ	Đến	Giao thức	Xác thực	Ghi chú
Mobile → Main	Mobile App	Main Service	HTTPS REST + SSE	JWT Bearer	Access token 30 phút, tự refresh
Web → Main	Admin Web	Main Service	HTTPS REST + WebSocket	JWT Bearer	Cookie admin_token
Main → RAG	Main Service	RAG Service	HTTP nội bộ	X-API-Key header	Không expose ra internet
RAG → ChromaDB	RAG Service	ChromaDB	HTTP/Persistent client	Docker network	Vector search và lưu embeddings
Main → MongoDB	Main Service	MongoDB	Motor async	Không (Docker network)	Tra cứu văn bản và điều khoản pháp luật
Main → PostgreSQL	Main Service	PostgreSQL	asyncpqg	Không (Docker network)	Users, conversations, messages
RAG → LLM Provider	RAG Service	LLM API	HTTPS	API Key	Gọi thông qua lớp LLMService, có thể thay đổi provider/model bằng cấu hình
RAG → Tavily	RAG Service	Tavily API	HTTPS	API Key	Web search

2.2. Xây dựng kho dữ liệu vector pháp luật ban đầu

Kho dữ liệu vector là nền tảng để xây dựng hệ thống truy hồi căn cứ pháp luật trong Agentic RAG. Mục tiêu của giai đoạn này là chuyển tập văn bản pháp luật ban đầu thành các đơn vị dữ liệu có thể tìm kiếm theo ngữ nghĩa, có khả năng truy ngược nguồn và đủ ổn định để phục vụ quá trình hỏi đáp thời gian thực.

Quy trình xây dựng kho vector gồm bốn bước chính: chuẩn hóa cấu trúc JSON đầu vào, tách dữ liệu theo đơn vị điều luật, chia nhỏ các điều luật dài thành chunk, sau đó tạo embedding và lưu vào ChromaDB kèm metadata.

Các bước chuẩn hóa ở mục này tạo ra một hợp đồng dữ liệu chung giữa ba phần: tập dữ liệu JSON ban đầu, kho lưu trữ điều luật dạng văn bản và kho vector dùng cho truy hồi. Nhờ thống nhất định danh từ đầu, cùng một điều luật có thể được quản lý ở dạng văn bản gốc, dạng chunk và dạng vector mà không mất quan hệ nguồn.

2.2.1. Cấu trúc dữ liệu pháp luật đầu vào

Dữ liệu pháp luật ban đầu được tổ chức theo hai cấp: cấp văn bản và cấp điều luật. Ở cấp văn bản, trường `law_id` định danh văn bản pháp luật. Ở cấp điều luật, mỗi phần tử trong `articles` chứa `article_id`, `title`, `text` và `metadata`. Cấu trúc này phù hợp với cách truy vấn pháp luật vì phần lớn câu hỏi cần căn cứ ở một điều khoản cụ thể.

Một bản ghi sau khi chuẩn hóa có thể rút gọn theo cấu trúc sau:

```
{
  "law_id": "28/2026/QĐ-UBND",
  "articles": [
    {
      "article_id": "1",
      "title": "Điều 1. Phạm vi điều chỉnh và đối tượng áp dụng",
      "text": "...",
      "metadata": {
        "topics": ["Phạm vi điều chỉnh", "Đối tượng áp dụng"],
        "keywords": ["tàu cá", "hỗ trợ", "chuyển đổi nghề"],
        "summary": "...",
        "year": "2026"
      }
    }
  ]
}
```

Trước khi tạo embedding, dữ liệu được làm phẳng từ cấp văn bản xuống cấp article. Mỗi article được gán `law_id` của văn bản gốc để duy trì quan hệ nguồn. Cách tổ chức này tránh việc embedding toàn bộ văn bản dài, vốn dễ làm vector biểu diễn quá nhiều chủ đề và giảm độ chính xác khi truy hồi.

Đoạn xử lý làm phẳng dữ liệu có thể mô tả ngắn gọn như sau:

```
all_articles = []
```



```

for law in data:
    law_id = law.get("law_id", "Unknown")
    for article in law.get("articles", []):
        article["law_id"] = law_id
        all_articles.append(article)

```

Sau bước này, pipeline không còn xử lý theo cấp văn bản lớn mà xử lý theo danh sách article độc lập. Đây là điểm quan trọng vì ChromaDB cần các document có kích thước vừa đủ để so sánh ngữ nghĩa, trong khi hệ thống quản trị vẫn cần giữ được law_id để truy ngược về văn bản gốc.

2.2.2. Chuẩn hóa metadata cho từng chunk

Metadata được lưu cùng từng chunk để bảo đảm kết quả truy hỏi luôn có thông tin nguồn. Đây là yêu cầu quan trọng đối với bài toán pháp luật, vì câu trả lời không chỉ cần đúng nội dung mà còn phải chỉ ra được văn bản, điều luật và bối cảnh áp dụng.

Các trường metadata chính được tạo trong pipeline ingest gồm:

```

metadata = {
    "law_id": article.get("law_id", ""),
    "article_id": article.get("article_id", ""),
    "title": article.get("title", ""),
    "chunk_index": chunk_index,
    "total_chunks": total_chunks,
    "year": article.get("metadata", {}).get("year", ""),
    "topics": json.dumps(article.get("metadata",
    {}).get("topics", []), ensure_ascii=False),
    "keywords": json.dumps(article.get("metadata",
    {}).get("keywords", []), ensure_ascii=False),
    "summary": article.get("metadata", {}).get("summary", "")
}

```

Trong đó chunk_index và total_chunks cho biết vị trí của đoạn trong điều luật gốc. topics và keywords được chuyển thành chuỗi JSON để lưu ổn định trong ChromaDB nhưng vẫn giữ nguyên nội dung tiếng Việt. Nhờ vậy, dù một điều luật dài bị chia thành nhiều phần, mỗi chunk vẫn có đủ dữ liệu để truy ngược về nguồn ban đầu.

Về mặt sử dụng, metadata có ba vai trò chính trong pipeline RAG. Thứ nhất, metadata giúp hiển thị trích dẫn nguồn cho câu trả lời. Thứ hai, year và title hỗ trợ các bước xử lý văn bản mới, văn bản cũ hoặc văn bản có khả năng xung đột. Thứ ba, law_id và article_id giúp xóa hoặc cập nhật lại toàn bộ các chunk liên quan khi dữ liệu pháp luật thay đổi.

2.2.3. Chiến lược chunking cho văn bản pháp luật

Chiến lược chunking được thiết kế theo hướng lai. Các article ngắn được giữ nguyên để không làm mất ngữ cảnh. Các article dài được chia thành những chunk nhỏ hơn nhằm tránh việc một vector phải biểu diễn quá nhiều nội dung pháp lý khác nhau.

Các tham số chính của quá trình chunking và embedding được cấu hình như sau:

```
CHROMA_COLLECTION = "vietnamese_law"
EMBEDDING_MODEL = "bkai-foundation-models/vietnamese-bi-encoder"
MAX_CHUNK_LENGTH = 1000
CHUNK_OVERLAP = 150
USE_HYBRID_CHUNKING = True
BATCH_SIZE = 100
```

Với cấu hình trên, mỗi chunk có độ dài tối đa khoảng 1.000 từ và có phần chồng lấn 150 từ với chunk liền kề. Phần overlap giúp hạn chế mất ngữ cảnh ở ranh giới cắt, đặc biệt trong các điều luật có nhiều khoản, điểm hoặc điều kiện áp dụng nối tiếp nhau.

Quá trình chia chunk không sử dụng LLMs mà dựa trên luật xử lý cố định: đếm số từ, cắt theo ngưỡng độ dài và ưu tiên điểm ngắt gần đầu câu. Cách tiếp cận này giúp quá trình xây dựng lại kho dữ liệu có tính lặp lại, giảm chi phí và tránh sự không ổn định của mô hình sinh văn bản.

Logic quyết định giữ nguyên hay chia nhỏ article được triển khai theo hướng sau:

```
if not use_hybrid or word_count <= max_words:
    content = create_chunk_content(article)
    metadata = create_metadata(article, 0, 1)
else:
    text_chunks = split_text_with_overlap(text, max_words,
overlap)
    for idx, chunk_text in enumerate(text_chunks):
        content = create_chunk_content(article, chunk_text,
idx)
```

Cách đặt ngưỡng 1.000 từ là điểm cân bằng giữa hai yêu cầu. Nếu chunk quá dài, vector có thể bao phủ nhiều vấn đề pháp lý khác nhau; nếu chunk quá ngắn, đoạn được truy hồi có thể thiếu điều kiện áp dụng. Phần overlap 150 từ giúp giữ lại mạch lập luận khi một điều luật dài bị tách qua nhiều chunk.

Nội dung đưa vào embedding được ghép từ title và text của chunk:

```
content = f"{title}\n\n{text}"
```

Việc ghép tiêu đề vào nội dung giúp vector giữ được chủ đề pháp lý của điều luật. Điều này đặc biệt hữu ích khi phân thân điều luật sử dụng cách diễn đạt phụ thuộc vào tiêu đề hoặc không nhắc lại đầy đủ phạm vi điều chỉnh.

2.2.4. Tạo *embedding* và lưu vào *ChromaDB*

Sau khi tạo chunk, hệ thống sử dụng SentenceTransformer với mô hình `bai-foundation-models/vietnamese-bi-encoder` để mã hóa văn bản tiếng Việt thành vector ngữ nghĩa. Đây là mô hình bi-encoder, trong đó document được mã hóa khi ingest, còn truy vấn được mã hóa ở thời điểm người dùng đặt câu hỏi.

Trong pipeline, bi-encoder và cross-encoder được dùng ở hai giai đoạn khác nhau. Bi-encoder phù hợp cho bước truy hồi rộng vì document đã được mã hóa sẵn và có thể tìm nhanh trong ChromaDB. Cross-encoder được dùng sau đó để đánh giá sâu hơn từng cặp truy vấn - đoạn luật, giúp loại bỏ các kết quả chỉ giống từ khóa nhưng không trực tiếp trả lời câu hỏi.

Mỗi chunk được gán một định danh duy nhất theo `law_id`, `article_id` và vị trí chunk:

```
chunk_id =  
f"{article['law_id']}_{article['article_id']}_chunk{metadata['chunk'  
_index']}"
```

Dữ liệu sau khi tạo embedding được ghi vào collection `vietnamese_law` của ChromaDB theo batch. Mỗi lần ghi bao gồm `ids`, `embeddings`, `documents` và `metadatas`:

```
collection.add(  
    ids=all_ids,  
    embeddings=all_embeddings,  
    documents=all_documents,  
    metadatas=all_metadatas  
)
```

Cách lưu này tách rõ ba vai trò: document dùng để trích dẫn, embedding dùng để tìm kiếm tương đồng, metadata dùng để truy ngược nguồn và hỗ trợ kiểm chứng. Khi người dùng đặt câu hỏi, hệ thống chỉ cần mã hóa truy vấn, tìm các chunk gần nhất trong ChromaDB, xếp hạng lại kết quả và đưa nguồn phù hợp vào ngữ cảnh cho Agent.

Việc ghi dữ liệu theo batch giúp quá trình ingest ổn định hơn khi số lượng chunk lớn. Thay vì gửi từng vector riêng lẻ vào ChromaDB, hệ thống gom nhiều bản

ghi rồi lưu cùng lúc, giảm chi phí giao tiếp với cơ sở dữ liệu vector và giúp kiểm soát tốt hơn trạng thái hoàn tất của quá trình xây dựng kho ban đầu.

2.2.5. Vai trò của kho vector trong Agentic RAG

Chất lượng kho vector ảnh hưởng trực tiếp đến chất lượng trả lời của Agent. Nếu chunk quá dài, kết quả truy hồi dễ thiếu trọng tâm; nếu chunk quá ngắn, câu trả lời có thể thiếu điều kiện áp dụng; nếu metadata không đầy đủ, hệ thống khó trích dẫn và khó kiểm chứng nguồn.

Vì vậy, kho vector ban đầu không chỉ là nơi lưu embedding. Đây là lớp dữ liệu quyết định khả năng tìm đúng căn cứ, kiểm soát nguồn và duy trì tính tin cậy của toàn bộ pipeline Agentic RAG.

2.3. Xác định tính chất AI Agent

Sau khi xác định kiến trúc tổng thể và nền tảng dữ liệu pháp luật, bước tiếp theo là xác định tác nhân AI hoạt động trong môi trường nào. Với bài toán tư vấn pháp luật, tác nhân không chỉ sinh câu trả lời mà còn phải biết sử dụng kho tri thức đã chuẩn hóa, đặt giới hạn phạm vi, chủ động tra cứu khi thiếu căn cứ và từ chối những yêu cầu không phù hợp.

2.3.1. Tính chất môi trường của Agent

Trước khi xây dựng luồng suy luận, cần xác định môi trường hoạt động của tác nhân AI. Đây là bước quan trọng vì cách tác nhân quan sát dữ liệu, lựa chọn hành động và phản hồi người dùng phụ thuộc trực tiếp vào đặc điểm của môi trường.

Đối với bài toán tư vấn pháp luật, môi trường không hoàn toàn quan sát được. Tác nhân không đọc toàn bộ kho văn bản trong một lần, mà chỉ nhìn thấy câu hỏi, một phần lịch sử hội thoại và những nguồn được truy hồi trong từng lượt xử lý. Do đó, thiết kế Agent phải ưu tiên khả năng tra cứu bổ sung và kiểm chứng thay vì trả lời ngay.

Bảng 2.2. Tính chất môi trường của Vietnam Law AI Agent

Tính chất	Phân loại	Giải thích
Khả năng quan sát	Quan sát được một phần (partially observable)	Tác nhân AI chỉ nhìn thấy câu hỏi, lịch sử liên quan và các kết quả truy hồi trong từng lượt xử lý.

Tính xác định	Ngẫu nhiên (stochastic)	Cùng câu hỏi có thể cho kết quả khác nhau do LLM có tính ngẫu nhiên
Tính tuần tự	Tuần tự (sequential)	Câu trả lời hiện tại phụ thuộc vào lịch sử hội thoại trước đó
Thời gian	Bán tĩnh (semi-dynamic)	Dữ liệu pháp luật thay đổi chậm; Agent không thay đổi trong lúc người dùng suy nghĩ
Không gian hành động	Rời rạc (discrete)	Các hành động của tác nhân là những lựa chọn rời rạc như tra cứu, đối chiếu, tổng hợp hoặc từ chối.
Số tác nhân	Một tác nhân chính	Một tác nhân chịu trách nhiệm điều phối quá trình trả lời cho từng yêu cầu độc lập.
Môi trường	Người dùng (mobile/web) + CSDL pháp luật + Web search + LLM	Môi trường gồm người dùng, kho dữ liệu pháp luật, nguồn đối chiếu bên ngoài và LLMs.

2.3.2. Đặc tả vai trò và phạm vi Agent

Sau khi xác định tính chất môi trường, cần giới hạn rõ vai trò của Agent. Agent không đóng vai trò thay thế luật sư hoặc cơ quan có thẩm quyền, mà là trợ lý hỗ trợ tra cứu, diễn giải và chỉ ra căn cứ pháp lý liên quan đến câu hỏi của người dùng.

Việc xác định phạm vi giúp hệ thống tránh hai rủi ro: trả lời vượt quá dữ liệu có căn cứ và đưa ra kết luận mang tính cam kết pháp lý. Vì vậy, các giới hạn trong Bảng 2.3 được dùng như nguyên tắc thiết kế cho toàn bộ pipeline trả lời.

Bảng 2.3. Định nghĩa vai trò và giới hạn hoạt động của Agent

Mục	Nội dung
Vai trò	Luật sư tư vấn pháp luật Việt Nam - tư vấn dựa trên văn bản quy phạm pháp luật, không đưa ra ý kiến pháp lý cá nhân
Đối tượng phục vụ	Người dân, doanh nghiệp, cán bộ pháp chế, sinh viên luật
Phạm vi thẩm quyền	Chỉ pháp luật Việt Nam (luật, nghị định, thông tư, quyết định) - từ chối domain khác (y tế, tài chính, kỹ thuật thuần túy)

Giới hạn đạo đức	Không tư vấn cách lách luật, trốn thuế, thực hiện hành vi bất hợp pháp
Giới hạn kỹ thuật	Dữ liệu nội bộ cập nhật đến năm 2026; với luật rất mới → dựa vào web search
Tuyên bố miễn trừ	Kết quả mang tính tham khảo; vụ việc phức tạp nên tham khảo luật sư chuyên môn

2.3.3. Nguồn dữ liệu hỗ trợ Agent và hệ thống

Nguồn dữ liệu quyết định trực tiếp khả năng trả lời của Agent. Với miền pháp luật, dữ liệu cần vừa có tính đầy đủ, vừa có khả năng truy ngược về văn bản gốc. Trên nền kho vector đã xây dựng ở mục 2.2, Agent sử dụng metadata, thông tin nguồn và vector ngữ nghĩa để tìm căn cứ phù hợp cho từng câu hỏi.

Các nguồn trong Bảng 2.4 được phân chia theo vai trò: nguồn nội bộ dùng làm căn cứ chính, nguồn bên ngoài dùng để đối chiếu tính cập nhật, còn dữ liệu hội thoại hỗ trợ hiểu ngữ cảnh người dùng. Cách phân chia này giúp tránh nhầm lẫn giữa ngữ cảnh hội thoại và căn cứ pháp lý.

Bảng 2.4. Nguồn dữ liệu phục vụ Agent và các chức năng tra cứu

Nguồn	Loại	Cơ chế truy cập	Thế mạnh	Hạn chế
ChromaDB (CSDL nội bộ)	Vector DB	Tool <code>retrieve_internal_law</code>	Nguyên văn điều khoản đầy đủ, có cấu trúc	Không cập nhật real-time, cần re-ingest
MongoDB	Document DB	Đọc thông qua lớp dịch vụ pháp luật của Main Service	Lưu toàn văn, metadata và hỗ trợ tìm kiếm, thống kê, hiển thị chi tiết nguồn	Agent không truy cập trực tiếp trong graph
Tavily Search	Web search	Tool <code>search_web_for_law</code>	Real-time, ưu tiên domain gov.vn	Chỉ trả snippet, không toàn văn
Google Grounding	Web search	Tool <code>search_web_for_law</code>	Google index đầy đủ, real-time	Phụ thuộc kết quả Google

Lịch sử hội thoại	Context	Truyền trực tiếp qua messages	Hiểu context đa lượt	Bị giới hạn bởi context window LLM
-------------------	---------	-------------------------------	----------------------	------------------------------------

2.3.4. Tập hành động của Agent

Từ các nguồn dữ liệu và phạm vi đã xác định, tác nhân AI trong hệ thống có các nhóm hành động chính sau:

1. Tra cứu tri thức nội bộ để tìm văn bản, điều luật và đoạn nội dung liên quan đến câu hỏi.
2. Đối chiếu nguồn cập nhật bên ngoài khi câu hỏi có khả năng phụ thuộc vào văn bản mới hoặc tình trạng hiệu lực.
3. Tổng hợp câu trả lời dựa trên các căn cứ đã thu thập và diễn giải theo ngôn ngữ dễ hiểu.
4. Từ chối hoặc giới hạn câu trả lời khi câu hỏi nằm ngoài phạm vi tư vấn pháp luật.
5. Tra cứu bổ sung khi bằng chứng hiện có chưa đủ để đưa ra câu trả lời đáng tin cậy.

2.4. Nguồn tri thức và cơ chế truy hồi của Agent

Tác nhân AI chỉ có thể trả lời chính xác khi có nguồn tri thức đáng tin cậy. Trong hệ thống này, nguồn tri thức được chia thành hai nhóm: kho pháp luật nội bộ đã được chuẩn hóa và nguồn cập nhật bên ngoài dùng để đối chiếu tính hiện hành của quy định.

2.4.1. Tra cứu kho pháp luật nội bộ

Kho pháp luật nội bộ là nguồn tri thức chính của pipeline RAG, vì đây là nơi lưu các điều luật đã được chuẩn hóa và có thể truy xuất lại theo ngữ nghĩa. Trong phạm vi sử dụng của Agent, kho này được nhìn nhận như tập các đoạn văn bản pháp luật đã có nội dung, vector biểu diễn ngữ nghĩa và metadata nguồn đi kèm.

Khi người dùng đặt câu hỏi, hệ thống không đưa trực tiếp toàn bộ câu hỏi tự nhiên vào truy hồi trong mọi trường hợp. Bước phân tích truy vấn tạo ra một truy vấn nội bộ ngắn gọn hơn, tập trung vào vấn đề pháp lý cần tìm. Truy vấn này được mã hóa thành embedding và gửi đến ChromaDB để lấy tập ứng viên có khoảng cách ngữ nghĩa gần nhất.

Kết quả trả về từ ChromaDB chưa được sử dụng ngay làm căn cứ cuối cùng. Pipeline tiếp tục xếp hạng lại bằng cross-encoder để phân biệt các đoạn chỉ giống về

từ khóa với các đoạn thật sự trả lời đúng vấn đề. Cách tổ chức này giúp tăng độ chính xác của nguồn trước khi Agent dùng chúng để tổng hợp câu trả lời.

Mỗi đoạn được truy hồi luôn đi kèm thông tin nguồn như số hiệu văn bản, mã điều, tiêu đề, năm ban hành, chủ đề và vị trí chunk trong điều luật. Nhờ vậy, Agent không chỉ biết nội dung pháp lý cần dùng mà còn có đủ dữ liệu để trích dẫn, hiển thị nguồn và hỗ trợ kiểm tra xung đột giữa văn bản cũ và văn bản mới.

Quá trình hình thành kho vector nói trên đã được trình bày ở mục 2.2. Vì vậy, trong mục này trọng tâm không còn là cách xây dựng dữ liệu ban đầu, mà là cách Agent khai thác kho đó trong quá trình truy hồi, xếp hạng lại và tổng hợp câu trả lời.

2.4.2. Tìm kiếm nguồn cập nhật bên ngoài

Nguồn bên ngoài không thay thế kho dữ liệu nội bộ mà đóng vai trò đối chiếu. Khi câu hỏi liên quan đến quy định mới, mức phạt mới hoặc văn bản có khả năng đã thay đổi, hệ thống có thể kiểm tra thêm nguồn cập nhật trước khi đưa ra kết luận.

Khi hai nhóm nguồn cùng được sử dụng, hệ thống ưu tiên nguyên tắc an toàn: chỉ khẳng định nội dung pháp lý khi có căn cứ rõ ràng. Nếu nguồn nội bộ và nguồn cập nhật có dấu hiệu khác nhau, hệ thống cần ưu tiên kiểm tra tính hiệu lực trước khi tổng hợp câu trả lời.

Như vậy, kho nội bộ đảm nhiệm vai trò cung cấp tri thức đã chuẩn hóa, còn nguồn bên ngoài giúp giảm rủi ro sử dụng văn bản không còn phù hợp. Hai nguồn này bổ sung cho nhau thay vì trùng lặp chức năng.

Trong phần kiểm tra nguồn cập nhật bên ngoài, hệ thống không phụ thuộc vào một kênh tìm kiếm duy nhất. Các nguồn có khả năng cung cấp nội dung đầy đủ được ưu tiên khi cần đối chiếu văn bản pháp luật, trong khi nguồn có phạm vi bao phủ rộng được dùng để phát hiện tin tức pháp lý mới, văn bản thay thế hoặc thông tin hiệu lực được cập nhật gần đây. Cách kết hợp này giúp giảm rủi ro bỏ sót quy định mới nhưng vẫn giữ được khả năng đối chiếu với nội dung cụ thể.

Về mặt xử lý, nguồn bên ngoài được sử dụng như lớp kiểm chứng bổ sung cho kho tri thức nội bộ. Khi câu hỏi liên quan đến mức phạt, thời hạn, hiệu lực hoặc văn bản có khả năng đã được thay thế, hệ thống cần ưu tiên tìm dấu hiệu cập nhật từ nguồn bên ngoài trước khi đưa ra kết luận. Nếu kết quả bên ngoài chỉ cung cấp đoạn trích ngắn, hệ thống không mở rộng nội dung ngoài phần có căn cứ; nếu tìm thấy văn bản đầy đủ hoặc nguồn chính thức, kết quả đó được dùng để đối chiếu với phần truy hồi nội bộ.

Thiết kế này phù hợp với đặc thù dữ liệu pháp luật vì kho nội bộ mạnh ở khả năng lưu trữ có cấu trúc và truy hồi nguyên văn, còn nguồn cập nhật bên ngoài mạnh ở khả năng phản ánh thay đổi mới. Hai nhóm nguồn không thay thế nhau mà bổ sung cho nhau: kho nội bộ tạo nền tảng trích dẫn ổn định, còn nguồn bên ngoài giúp tránh trả lời dựa trên văn bản đã lỗi thời.

2.5. Luồng Agentic RAG có kiểm chứng

Từ mô hình tác nhân và các nguồn tri thức đã xác định, hệ thống xây dựng luồng RAG có kiểm chứng cho một lượt hỏi đáp. Luồng này không để LLMs trả lời ngay sau khi nhận câu hỏi, mà chia quá trình xử lý thành các bước có trách nhiệm rõ ràng. Nhờ đó, hệ thống có thể kiểm soát đầu vào, xác định hướng tìm kiếm, thu thập bằng chứng và kiểm tra lại câu trả lời trước khi hiển thị cho người dùng.

Điểm cốt lõi của phương pháp này là tách phần suy luận khỏi phần kiểm chứng. LLMs có thể diễn giải và tổng hợp tốt, nhưng trong lĩnh vực pháp luật, kết luận cuối cùng cần được ràng buộc bởi nguồn. Vì vậy, mỗi bước trong pipeline đều phục vụ mục tiêu đưa câu hỏi của người dùng về gần hơn với các căn cứ pháp lý phù hợp.

2.5.1. Trạng thái xử lý trong Agent

Trong một lượt hỏi đáp, hệ thống cần duy trì các thông tin trung gian như câu hỏi hiện tại, ngữ cảnh hội thoại, kết quả phân tích truy vấn, các nguồn đã truy hồi và trạng thái hợp lệ của câu hỏi. Những thông tin này giúp pipeline biết khi nào cần tìm thêm bằng chứng và khi nào có thể chuyển sang kiểm chứng.

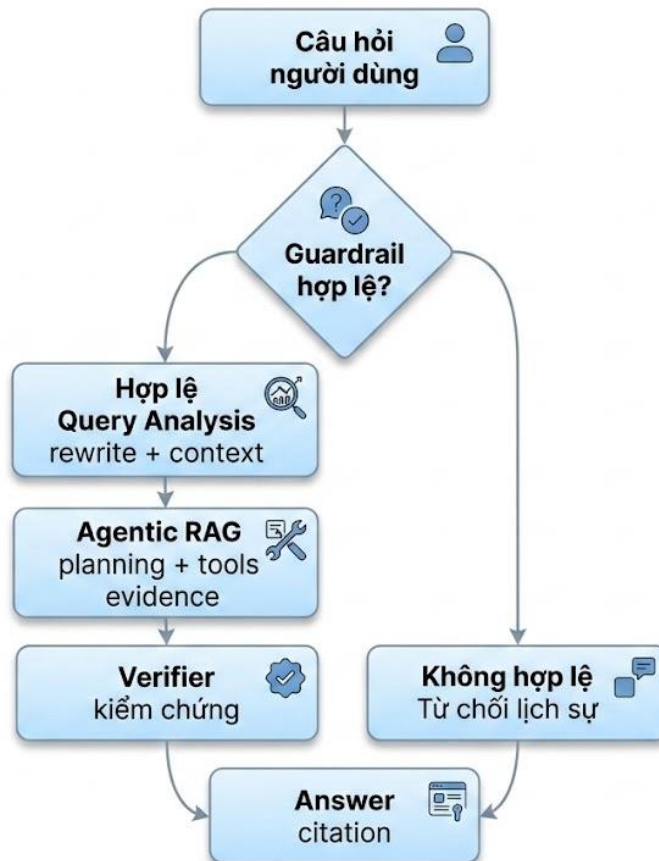
Bảng 2.5. Trạng thái trung gian trong luồng RAG có kiểm chứng

Nhóm dữ liệu	Dạng thông tin	Được cập nhật trong bước	Mục đích
Lịch sử xử lý	Danh sách thông điệp	Toàn bộ pipeline	Lưu câu hỏi, kết quả truy hồi và các phản hồi trung gian.
Kết quả phân tích truy vấn	Thông tin có cấu trúc	Phân tích truy vấn	Làm rõ ý định, phạm vi pháp lý và hướng truy hồi phù hợp.
Tài liệu truy hồi	Danh sách căn cứ	Suy luận và tra cứu	Lưu các đoạn văn bản được dùng làm bằng chứng cho câu trả lời.

Số vòng suy luận	Số nguyên	Suy luận	Giới hạn số lần tra cứu bổ sung để kiểm soát độ trễ.
Kết quả kiểm soát đầu vào	Đúng/Sai	Kiểm soát đầu vào	Cho biết câu hỏi có được tiếp tục xử lý hay phải từ chối.
Lý do từ chối	Văn bản ngắn	Kiểm soát đầu vào	Giải thích lý do hệ thống không trả lời một yêu cầu ngoài phạm vi.

Bảng 2.5 cho thấy dữ liệu không đi qua pipeline dưới dạng một chuỗi văn bản đơn lẻ. Mỗi bước đều bổ sung hoặc cập nhật một phần trạng thái, từ đó bước sau có thể biết câu hỏi đã hợp lệ chưa, nguồn nào đã được truy hồi và câu trả lời dự thảo có đủ căn cứ để kiểm chứng hay không.

2.5.2. Cấu trúc đồ thị tổng quan



Hình 2.4. Phương pháp Agentic RAG gồm bốn bước xử lý

Luồng tổng quan gồm bốn bước chính. Đầu tiên, hệ thống kiểm tra xem câu hỏi có thuộc phạm vi pháp luật hay không. Tiếp theo, câu hỏi hợp lệ được phân tích

để tạo hướng tìm kiếm rõ hơn. Sau đó, tác nhân AI thu thập bằng chứng từ các nguồn phù hợp. Cuối cùng, câu trả lời dự thảo được kiểm chứng trước khi trả về người dùng.

So với RAG tuyến tính, luồng này cho phép tác nhân linh hoạt hơn trong quá trình tìm nguồn. Nếu bằng chứng chưa đủ, hệ thống có thể truy vấn bổ sung hoặc đối chiếu thêm nguồn cập nhật. Điều này phù hợp với câu hỏi pháp luật vì một câu hỏi ngắn có thể phụ thuộc vào loại đối tượng, tình huống áp dụng và thời điểm hiệu lực.

Như vậy, Hình 2.4 không chỉ thể hiện thứ tự xử lý mà còn thể hiện hai lớp kiểm soát chất lượng: kiểm soát đầu vào để loại bỏ yêu cầu ngoài phạm vi và kiểm chứng đầu ra để hạn chế nội dung thiếu căn cứ.

2.5.3. Bước 1 - Kiểm soát đầu vào

Bước kiểm soát đầu vào xác định câu hỏi có thuộc phạm vi tư vấn pháp luật hay không. Nếu người dùng đặt câu hỏi lạc đề, hệ thống phản hồi từ chối lịch sự và không chạy các bước truy hồi phía sau. Nhờ vậy, tài nguyên xử lý được tập trung cho đúng mục tiêu của hệ thống.

Bảng 2.6. Dữ liệu đầu vào và đầu ra của bước kiểm soát đầu vào

	Thông tin	Giá trị ví dụ
Input	Câu hỏi người dùng	"Vượt đèn đỏ phạt bao nhiêu?"
Đầu ra hợp lệ	Kết quả kiểm tra	Câu hỏi thuộc phạm vi tư vấn pháp luật.
Đầu ra bị từ chối	Kết quả kiểm tra	Câu hỏi nằm ngoài phạm vi tư vấn pháp luật.
	Lý do	Yêu cầu không liên quan đến pháp luật Việt Nam.
	Phản hồi	Hệ thống giải thích ngắn gọn lý do từ chối và hướng người dùng quay lại phạm vi phù hợp.

Các trường trong bảng làm rõ hai trường hợp xử lý: câu hỏi hợp lệ được chuyển tiếp xuống pipeline, còn câu hỏi không phù hợp được phản hồi ngay với lý do từ chối. Nhờ vậy, bước kiểm soát đầu vào không chỉ là bộ lọc kỹ thuật mà còn là cơ chế bảo vệ phạm vi tư vấn của hệ thống.

Ví dụ, với một yêu cầu lập trình không liên quan đến pháp luật, hệ thống có thể dừng ngay ở bước này. Đây là một kiểm soát cần thiết vì chatbot có thể nhận nhiều loại câu hỏi khác nhau, trong khi phạm vi đề án chỉ tập trung vào tư vấn và tra cứu pháp luật Việt Nam.

2.5.4. Bước 2 - Phân tích và tái cấu trúc truy vấn

Khi câu hỏi hợp lệ, hệ thống chuyển sang bước phân tích truy vấn. Mục tiêu của bước này là tách câu hỏi tự nhiên của người dùng thành các thông tin có ích cho truy hỏi: chủ đề pháp lý, thuật ngữ chính, phạm vi tìm kiếm và các dấu hiệu cần đối chiếu.

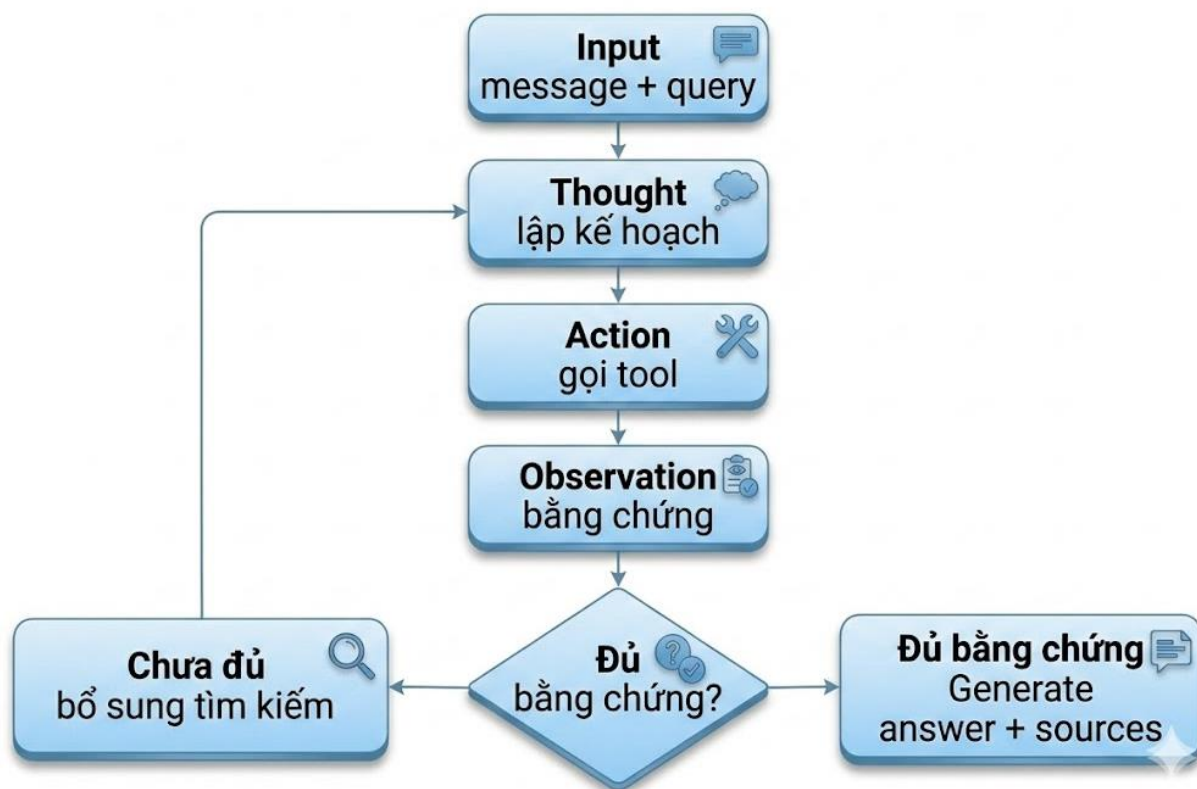
Phân tích truy vấn cũng quyết định cách sử dụng từng nguồn. Truy vấn gửi vào kho nội bộ cần cô đọng và gần với thuật ngữ pháp lý; truy vấn dành cho nguồn cập nhật có thể cần thêm dấu hiệu về văn bản mới hoặc năm hiệu lực. Nhờ vậy, các bước sau không phải xử lý trực tiếp một câu hỏi còn quá đời thường hoặc thiếu trọng tâm.

Bảng 2.7. Dữ liệu đầu vào và đầu ra của bước phân tích truy vấn

	Thông tin	Giá trị ví dụ
Input	Câu hỏi hiện tại	"Đi xe máy uống bia bị phạt bao nhiêu?"
	Ngữ cảnh hội thoại nếu có	5-10 messages trước
Output	Kết quả phân tích	Ý định, chủ đề pháp lý, phạm vi tìm kiếm và truy vấn đã được tối ưu.
	Thông tin chuyển tiếp	Phân tích được chuyển sang bước suy luận để định hướng tra cứu.

Kết quả phân tích truy vấn đóng vai trò cầu nối giữa ngôn ngữ tự nhiên của người dùng và yêu cầu truy hỏi trong kho dữ liệu. Nếu bước này hoạt động tốt, câu hỏi phía sau sẽ đúng trọng tâm pháp lý hơn so với việc dùng nguyên văn câu hỏi ban đầu.

2.5.5. Bước 3 - Suy luận và thu thập bằng chứng



Hình 2.5. Vòng lặp Re-Act trong quá trình suy luận

Ở bước suy luận, tác nhân AI vận hành theo vòng Re-Act. Trước hết, tác nhân xác định thông tin còn thiếu; sau đó chọn hành động tra cứu phù hợp; cuối cùng quan sát kết quả nhận được để quyết định tiếp tục tìm kiếm hay chuyển sang tổng hợp câu trả lời.

Cơ chế này giúp hệ thống không tạo câu trả lời quá sớm. Khi căn cứ chưa đủ, tác nhân có thể quay lại tìm thêm nguồn. Khi căn cứ đã đủ, pipeline mới chuyển sang tổng hợp. Điều này làm cho câu trả lời phụ thuộc nhiều hơn vào dữ liệu truy hồi và ít phụ thuộc hơn vào khả năng suy đoán của LLMs.

Trong miền pháp luật, một câu hỏi có thể cần nhiều loại bằng chứng: điều luật gốc, điều kiện áp dụng, mức xử phạt, thời điểm hiệu lực hoặc văn bản thay thế. Vòng Re-Act giúp quá trình thu thập bằng chứng diễn ra theo từng bước thay vì gom tất cả vào một lần tìm kiếm duy nhất.

Để tránh kéo dài không cần thiết, hệ thống vẫn đặt giới hạn số vòng lặp và luôn chuyển sang bước kiểm chứng trước khi trả lời. Đây là điểm cân bằng giữa chất lượng bằng chứng và độ trễ chấp nhận được của trải nghiệm chat.

Bảng 2.8. Dữ liệu đầu vào và đầu ra của bước suy luận

	Thông tin	Mô tả
Input	Ngữ cảnh xử lý	Câu hỏi, phân tích truy vấn và kết quả tra cứu trước đó.
	Số vòng suy luận	Số vòng lặp hiện tại
Đầu ra tiếp tục	Yêu cầu tra cứu bổ sung	Hệ thống cần thêm bằng chứng trước khi trả lời.
	Số vòng suy luận	Tăng lên để kiểm soát giới hạn lặp.
Đầu ra kết thúc	Câu trả lời dự thảo	Câu trả lời được tổng hợp từ các căn cứ đã thu thập.

Bảng 2.8 thể hiện điểm khác biệt giữa suy luận một lần và suy luận có vòng lặp. Đầu ra của bước suy luận có thể là yêu cầu tra cứu bổ sung hoặc câu trả lời dự thảo, cho phép hệ thống chủ động thu thập thêm nguồn khi bằng chứng hiện tại chưa đủ.

2.5.6. Bước 4 - Kiểm chứng câu trả lời

Bước kiểm chứng đối chiếu câu trả lời dự thảo với các nguồn đã truy hồi. Các nội dung như số điều, mức phạt, điều kiện áp dụng hoặc thời hạn phải xuất hiện trong bằng chứng. Nếu phát hiện phân diễn giải vượt quá nguồn, hệ thống cần sửa hoặc loại bỏ trước khi trả về kết quả cuối cùng.

Bảng 2.9. Dữ liệu đầu vào và đầu ra của bước kiểm chứng

	Trường	Mô tả
Input	Câu trả lời dự thảo	Câu trả lời cần kiểm chứng
	Các căn cứ đã truy hồi	Nguồn thông tin được dùng để đối chiếu nội dung trả lời.
Đầu ra đạt	Câu trả lời giữ nguyên	Câu trả lời đã có đủ căn cứ để trả về người dùng.
Đầu ra cần sửa	Câu trả lời đã hiệu chỉnh	Loại bỏ hoặc sửa phần chưa có đủ căn cứ.

Bảng 2.9 cho thấy bước kiểm chứng không tạo nguồn mới mà đối chiếu câu trả lời với các nguồn đã có. Nếu nội dung phù hợp với bằng chứng, câu trả lời được giữ nguyên; nếu không, hệ thống phải hiệu chỉnh để tránh đưa ra thông tin thiếu căn cứ.

2.6. Tối ưu pipeline RAG - truy xuất hai giai đoạn

Hai công cụ của Agent chỉ là giao diện ở mức cao; phân quyết định chất lượng tìm kiếm nằm ở pipeline truy xuất bên dưới. Đối với kho dữ liệu 690,360 vector chunks, hệ thống không thể dùng LLM đọc toàn bộ dữ liệu, cũng không thể chỉ tìm kiếm từ khóa đơn giản. Vì vậy, đề án sử dụng chiến lược truy xuất hai giai đoạn để cân bằng giữa tốc độ và độ chính xác.

Giai đoạn đầu có nhiệm vụ lọc rộng bằng tìm kiếm vector để lấy ra tập ứng viên có khả năng liên quan. Giai đoạn sau đánh giá sâu hơn từng ứng viên để chọn các đoạn văn bản phù hợp nhất với câu hỏi. Cách làm này vừa giữ được tốc độ xử lý, vừa tăng chất lượng căn cứ đưa vào LLMs.

Kho vector và bước xếp hạng lại chịu trách nhiệm chọn bằng chứng; tác nhân AI chịu trách nhiệm sử dụng bằng chứng đó để suy luận và trình bày câu trả lời. Sự phân vai này giúp pipeline rõ ràng hơn và tránh để LLMs tự suy đoán nguồn.

2.6.1. Hạn chế của tìm kiếm vector đơn lẻ

Tìm kiếm vector có ưu điểm lớn về tốc độ vì các đoạn văn bản trong kho tri thức có thể được mã hóa trước và lưu trong cơ sở dữ liệu vector. Khi người dùng đặt câu hỏi, hệ thống chỉ cần mã hóa câu hỏi và so sánh độ tương đồng để lấy ra các đoạn có khả năng liên quan.

Tuy nhiên, cách tiếp cận này chưa luôn hiểu được mối quan hệ ngữ nghĩa tinh tế giữa câu hỏi và văn bản. Ví dụ, với câu hỏi "thủ tục đăng ký kết hôn với người nước ngoài", hệ thống có thể trả về nhiều đoạn chỉ chứa cụm "đăng ký kết hôn" nhưng không trực tiếp trả lời đúng trường hợp có yếu tố nước ngoài.

2.6.2. Xếp hạng lại kết quả truy hồi

Để khắc phục hạn chế trên, hệ thống bổ sung bước xếp hạng lại. Thay vì chỉ so sánh hai vector đã được mã hóa độc lập, bước này đánh giá trực tiếp từng cặp câu hỏi và đoạn văn bản, nhờ đó nhận diện tốt hơn mức liên quan thực sự giữa yêu cầu của người dùng và căn cứ pháp lý.

Chiến lược hai giai đoạn được tổ chức như sau:

- Giai đoạn 1: tìm kiếm vector nhanh để lấy một tập ứng viên rộng từ kho tri thức.
- Giai đoạn 2: xếp hạng lại tập ứng viên để loại bỏ các đoạn chỉ liên quan bề mặt và ưu tiên đoạn trả lời trực tiếp hơn.

Cách thiết kế hai giai đoạn tạo ra sự cân bằng giữa tốc độ và chất lượng. Nếu lấy quá ít ứng viên ở giai đoạn đầu, hệ thống có nguy cơ bỏ sót điều luật phù hợp; nếu lấy quá nhiều, bước xếp hạng lại sẽ làm tăng độ trễ. Vì vậy, pipeline được cấu hình để lấy đủ rộng ở bước đầu và chỉ chắt sâu trên một tập đã được thu hẹp.

2.6.3. Công thức kết hợp điểm phù hợp

Điểm số cuối cùng kết hợp cả hai nguồn:

$$\text{FinalScore} = 0.3 \times \text{BiEncoderScore} + 0.7 \times \text{CrossEncoderScore}$$

Trong công thức trên, bước xếp hạng lại được đặt trọng số cao hơn vì nó đánh giá trực tiếp mối quan hệ giữa câu hỏi và đoạn văn bản. Điểm tìm kiếm vector vẫn được giữ lại để bảo đảm các đoạn có tương đồng ngữ nghĩa cao không bị loại bỏ hoàn toàn.

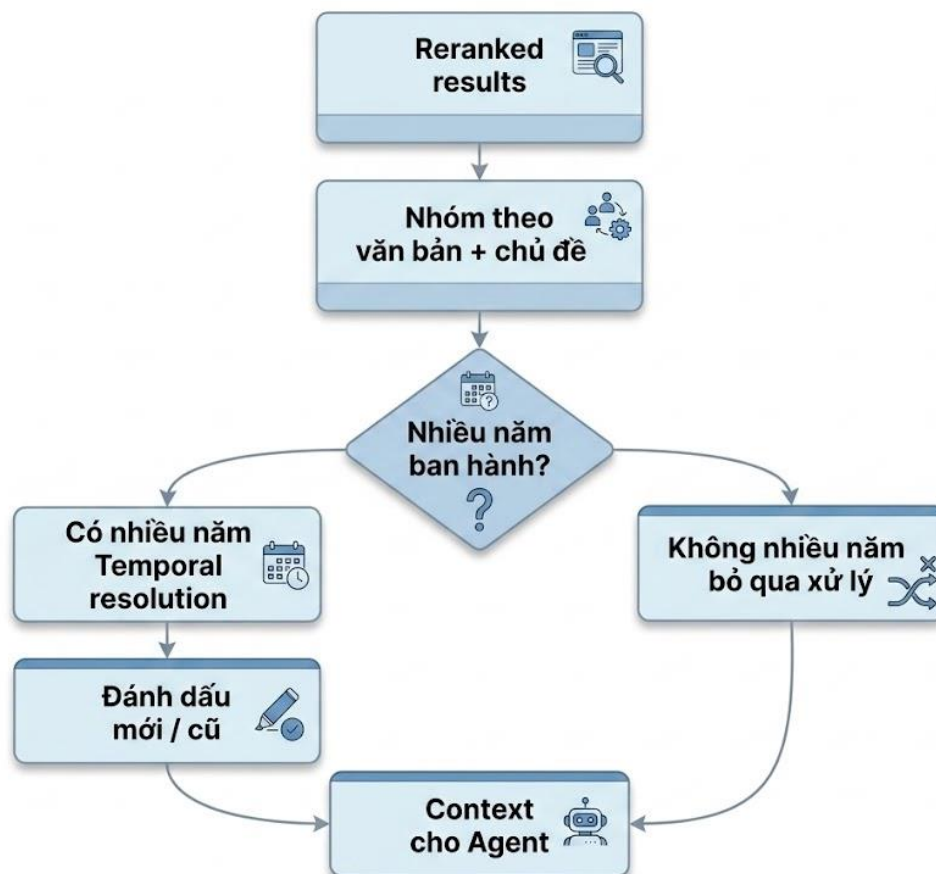
2.6.4. Ưu tiên văn bản mới

Đặc thù của dữ liệu pháp luật là nhiều nghị định, thông tư có thể được sửa đổi hoặc thay thế bởi văn bản mới. Nếu chỉ dựa vào độ tương đồng ngữ nghĩa, hai văn bản cùng chủ đề có thể có điểm số gần nhau dù một văn bản đã không còn phù hợp. Vì vậy, hệ thống bổ sung cơ chế ưu tiên văn bản mới trước khi chọn căn cứ cuối cùng.

Bảng 2.10. Tham số Year Boost

Độ mới văn bản	Điểm cộng thêm
0-2 năm gần nhất	+0.05
3-5 năm gần nhất	+0.02
6-10 năm	0.00 (không điều chỉnh)
Trên 10 năm	-0.03

2.6.5. Giải quyết mâu thuẫn pháp lý theo thời gian



Hình 2.6. Thuật toán phát hiện xung đột văn bản cũ và mới

Ngoài độ liên quan, dữ liệu pháp luật còn có yếu tố thời gian. Một văn bản cũ có thể diễn đạt gần với câu hỏi hơn, nhưng văn bản mới hơn mới là căn cứ đang có hiệu lực. Do đó, hệ thống cần xét thêm năm ban hành được rút ra từ số hiệu văn bản, loại văn bản và chủ đề chính của nguồn truy hồi để nhận diện khả năng xung đột giữa văn bản cũ và văn bản mới.

Ví dụ thực tế: Nghị định 100/2019/NĐ-CP và Nghị định 168/2024/NĐ-CP đều quy định mức xử phạt vi phạm giao thông. Nếu không có cơ chế xử lý, Agent sẽ nhận được cả hai kết quả và có thể nhầm lẫn áp dụng mức phạt của nghị định cũ đã hết hiệu lực.

Quy trình xử lý xung đột gồm ba bước: nhóm các nguồn có cùng loại văn bản và chủ đề chính, so sánh năm ban hành trong số hiệu văn bản, sau đó gắn nhãn nguồn cũ hoặc nguồn mới cho Agent. Nguồn mới hơn được ưu tiên làm căn cứ chính; nguồn cũ chỉ nên dùng để giải thích sự thay đổi khi cần.

Nhờ bổ sung tiêu chí thời gian, câu trả lời của hệ thống phù hợp hơn với bản chất của bài toán pháp luật. Người dùng không chỉ cần câu trả lời có vẻ liên quan, mà cần câu trả lời dựa trên quy định đang có giá trị áp dụng.

2.6.6. Ngưỡng tin cậy

Bên cạnh việc xếp hạng kết quả, hệ thống cần xác định ngưỡng tin cậy để quyết định khi nào có thể sử dụng một đoạn văn bản làm căn cứ. Nếu ngưỡng quá thấp, câu trả lời dễ chứa nguồn không thật sự liên quan; nếu ngưỡng quá cao, hệ thống có thể bỏ sót căn cứ hữu ích.

Các ngưỡng trong Bảng 2.11 được dùng để phân loại mức độ chắc chắn của kết quả truy hồi. Khi điểm tin cậy thấp, hệ thống có thể tiếp tục tìm kiếm, hỏi thêm ngữ cảnh hoặc trả lời thận trọng hơn thay vì khẳng định mạnh.

Bảng 2.11. Các ngưỡng tin cậy trong pipeline truy hồi

Tham số	Giá trị	Ý nghĩa
Số kết quả truy hồi ban đầu	60	Số kết quả từ kho vector được chuyển sang bước xếp hạng lại
Số ứng viên xếp hạng lại	40	Số đoạn tối đa được đánh giá sâu ở giai đoạn hai
SCORE_THRESHOLD_DISPLAY	0.60	Ngưỡng tối thiểu để kết quả xuất hiện làm "nguồn" cho người dùng
SCORE_THRESHOLD_GENERATION	0.75	Ngưỡng phân loại nguồn tin cậy cao trong chế độ RAG thường; luồng Agentic RAG dùng ngưỡng truy hồi nội bộ riêng để không bỏ sót căn cứ.
AGENT_RETRIEVAL_THRESHOLD	0.50	Ngưỡng lọc sau rerank trong công cụ retrieve_internal_law của Agentic RAG.

2.7. Luồng Tư vấn có Hướng dẫn (Guided Consultation)



Hình 2.7. Luồng tư vấn có hướng dẫn gồm hai bước

Luồng chat tự do phù hợp khi câu hỏi của người dùng đã có đủ ngữ cảnh. Tuy nhiên, trong thực tế, nhiều câu hỏi pháp luật được đặt rất ngắn, ví dụ "vượt đèn đỏ phạt bao nhiêu". Câu hỏi này chưa cho biết phương tiện, tình huống cụ thể hoặc các yếu tố có thể làm thay đổi mức xử phạt. Vì vậy, hệ thống bổ sung luồng tư vấn có hướng dẫn để chủ động hỏi lại trước khi trả lời.

Luồng tư vấn có hướng dẫn tách quá trình tư vấn thành hai bước. Bước đầu thu thập thêm dữ kiện còn thiếu bằng các câu hỏi làm rõ. Bước sau sử dụng câu hỏi ban đầu cùng các lựa chọn của người dùng để chạy truy hồi, suy luận và kiểm chứng. Cách tổ chức này giúp câu trả lời cuối cùng gắn với tình huống cụ thể hơn.

2.7.1. Vấn đề và động lực thiết kế

Trong thực tế tư vấn pháp luật, thiếu ngữ cảnh là nguyên nhân phổ biến khiến câu trả lời trở nên quá rộng hoặc không đúng trường hợp của người hỏi. Chẳng hạn, cùng là hành vi vượt đèn đỏ nhưng mức phạt có thể khác nhau giữa xe máy và ô tô. Nếu hệ thống trả lời ngay, nó buộc phải bao quát nhiều trường hợp hoặc tự giả định thông tin còn thiếu.

Nếu chỉ dùng luồng hỏi đáp một bước, hệ thống thường gặp hai khó khăn:

1. Câu trả lời dài và phân tán vì phải bao quát nhiều trường hợp có thể xảy ra.
2. Hệ thống có thể suy đoán dữ kiện còn thiếu, làm giảm độ chính xác của tư vấn.

Luồng tư vấn có hướng dẫn giải quyết vấn đề này bằng cách biến câu hỏi mơ hồ thành một tình huống rõ ràng hơn trước khi chạy pipeline trả lời.

Về mặt kỹ thuật, bước làm rõ giúp truy vấn trở nên cụ thể hơn, kết quả truy hồi tập trung hơn và bước kiểm chứng có phạm vi đối chiếu rõ ràng hơn. Đây là lý do chức năng này được tách thành một luồng riêng thay vì chỉ là một câu hỏi phụ trong chat tự do.

2.7.2. Thiết kế không lưu trạng thái phiên

Luồng tư vấn có hướng dẫn được thiết kế theo hướng không lưu trạng thái phiên trên máy chủ. Sau khi hệ thống sinh câu hỏi làm rõ, các lựa chọn của người dùng sẽ được gửi kèm trong yêu cầu trả lời ở bước tiếp theo.

Quy trình gồm ba thành phần thông tin chính:

1. Câu hỏi ban đầu của người dùng.
2. Danh sách câu hỏi làm rõ do hệ thống sinh ra.
3. Các lựa chọn hoặc thông tin bổ sung mà người dùng cung cấp trước khi nhận câu trả lời cuối cùng.

2.7.3. Bước 1 - Thu thập ngữ cảnh

Ở bước đầu tiên, hệ thống phân tích câu hỏi ban đầu để xác định những dữ kiện còn thiếu. Đầu ra của bước này là danh sách câu hỏi làm rõ, thường được biểu diễn dưới dạng các lựa chọn ngắn để người dùng thao tác nhanh trên giao diện.

Điểm cần lưu ý là bước này chưa tạo câu trả lời pháp lý cuối cùng. Nó chỉ chuẩn bị ngữ cảnh cho bước sau. Nhờ tách riêng như vậy, hệ thống có thể phản hồi nhanh ở bước làm rõ nhưng vẫn giữ yêu cầu truy hồi và kiểm chứng đối với câu trả lời chính thức.

2.7.4. Bước 2 - Trả lời theo ngữ cảnh đã làm rõ

Ở bước thứ hai, hệ thống kết hợp câu hỏi ban đầu với các lựa chọn làm rõ của người dùng. Yêu cầu sau khi được bổ sung ngữ cảnh sẽ cụ thể hơn, từ đó giúp pipeline truy hồi đúng căn cứ và hạn chế việc trả lời quá rộng.

Ví dụ, nếu câu hỏi ban đầu là "Vượt đèn đỏ phạt bao nhiêu?" và người dùng chọn phương tiện là xe máy, hệ thống sẽ ưu tiên truy hồi các quy định

liên quan đến xe mô tô, xe gắn máy thay vì trộn lẫn với ô tô hoặc các phương tiện khác.

Sau khi có ngữ cảnh rõ ràng, luồng trả lời vẫn tuân theo nguyên tắc của RAG có kiểm chứng: tìm nguồn, tổng hợp trên căn cứ và kiểm tra lại trước khi hiển thị cho người dùng.

2.7.5. Cơ chế trả lời theo dòng sự kiện

Câu trả lời trong luồng tư vấn có hướng dẫn được truyền về giao diện theo từng trạng thái xử lý. Người dùng không cần chờ trong im lặng, mà có thể thấy hệ thống đang chuẩn bị truy hồi, tra cứu nguồn, tổng hợp và kiểm tra độ chính xác.

Cơ chế này giữ trải nghiệm tương tự luồng chat thời gian thực, nhưng dữ liệu đầu vào đã rõ hơn nhờ bước làm rõ trước đó. Vì vậy, chức năng tư vấn có hướng dẫn vừa cải thiện trải nghiệm sử dụng, vừa hỗ trợ chất lượng câu trả lời.

Bảng 2.12. Các loại sự kiện trong luồng tư vấn có hướng dẫn

Loại sự kiện	Payload	Ý nghĩa
progress	Danh sách bước xử lý	Cập nhật vùng hiển thị tiến trình theo các bước: chuẩn bị, tra cứu, tổng hợp và kiểm chứng
done	Câu trả lời và nguồn trích dẫn	Kết thúc pipeline và hiển thị câu trả lời cuối cùng
error	{"message": "..."} 	Thông báo lỗi khi RAG Service hoặc pipeline trả lời thất bại

Bảng 2.12 giúp chuẩn hóa cách frontend hiển thị tiến trình xử lý của luồng tư vấn. Mỗi sự kiện không chỉ mang dữ liệu hiển thị mà còn thể hiện trạng thái nghiệp vụ: đang làm rõ, đang truy hồi, đang tổng hợp hoặc đã hoàn tất.

Ứng dụng nhận dòng sự kiện và cập nhật vùng hiển thị tiến trình theo từng trạng thái xử lý. Khi nhận được kết quả cuối cùng, giao diện hiển thị câu trả lời hoàn chỉnh cùng danh sách nguồn tham chiếu. Thiết kế này minh bạch hơn trạng thái chờ tĩnh vì người dùng thấy hệ thống đang thực sự tra cứu và kiểm chứng.

Các trạng thái hiển thị cho người dùng được giữ ở mức dễ hiểu, không đi sâu vào chi tiết kỹ thuật nội bộ. Mục tiêu là cho người dùng biết hệ thống đang xử lý có kiểm soát, đồng thời chỉ hiển thị nội dung pháp lý cuối cùng khi pipeline đã hoàn tất.

2.8. Quy trình cập nhật tri thức từ văn bản pháp luật



Hình 2.8. Luồng cập nhật tri thức pháp luật từ phía quản trị

Sau khi kho vector ban đầu đã được xây dựng từ dữ liệu JSON, hệ thống vẫn cần một cơ chế bổ sung văn bản mới trong quá trình vận hành. Đây là nhiệm vụ của luồng cập nhật tri thức từ phía quản trị. Khác với giai đoạn xây dựng dữ liệu ban đầu, đầu vào của luồng này là file PDF pháp luật do Admin tải lên, vì vậy hệ thống phải xử lý thêm các vấn đề như nhận dạng chữ, cấu trúc hóa điều luật và kiểm soát lỗi trong quá trình đồng bộ.

Về tổng thể, luồng cập nhật tri thức bắt đầu từ thao tác tải lên file PDF của quản trị viên. Sau đó hệ thống thực hiện tiền kiểm, trích xuất nội dung, chuẩn hóa thành cấu trúc điều luật, kiểm tra hợp lệ, lưu dữ liệu gốc và gọi quy trình ingest vector tương tự chiến lược đã trình bày ở mục 2.2. Kết quả cuối cùng không chỉ là một file được lưu, mà là một văn bản pháp luật mới sẵn sàng tham gia vào pipeline hỏi đáp.

2.8.1. Tổng quan quy trình

Bước tiền kiểm được đặt ở đầu quy trình để giảm rủi ro đưa dữ liệu sai vào hệ thống. Ở bước này, hệ thống có thể phát hiện tài liệu không hợp lệ, văn bản trùng

lập hoặc file không phù hợp trước khi chạy các tác vụ tốn nhiều tài nguyên như nhận dạng nội dung và tạo vector.

Sau tiền kiểm, dữ liệu được xử lý theo hai hướng. Một hướng tạo tri thức gốc để quản trị viên có thể xem, tìm kiếm và đối chiếu lại văn bản. Hướng còn lại tạo các vector phục vụ truy hồi ngữ nghĩa trong quá trình hỏi đáp. Hai hướng này phải đồng bộ với nhau để tránh tình trạng giao diện quản trị đã có văn bản nhưng chatbot chưa thể tìm được.

Như vậy, cập nhật văn bản pháp luật không chỉ là chức năng tải file. Đây là một pipeline dữ liệu hoàn chỉnh, có kiểm tra đầu vào, xử lý nội dung, ghi dữ liệu và kiểm soát trạng thái thành công hoặc thất bại.

Tiền kiểm cũng giúp tiết kiệm chi phí vận hành. Nếu quản trị viên tải nhầm một văn bản đã tồn tại, hệ thống có thể dừng sớm thay vì tiếp tục trích xuất, chuẩn hóa và tạo vector cho một dữ liệu trùng lặp.

2.8.2. Trích xuất và cấu trúc hóa văn bản PDF

Pipeline xử lý PDF được chia thành hai nhiệm vụ khác nhau: trích xuất nội dung chữ từ tài liệu và chuẩn hóa nội dung đó thành cấu trúc pháp lý. Việc tách hai nhiệm vụ giúp hệ thống tận dụng đúng vai trò của từng thành phần.

Ở nhiệm vụ thứ nhất, hệ thống cố gắng lấy lớp chữ có sẵn trong PDF. Nếu văn bản là bản scan hoặc chất lượng thấp, hệ thống sử dụng OCR để nhận dạng ký tự. Kết quả của bước này là phần văn bản thô phục vụ cho bước chuẩn hóa.

Ở nhiệm vụ thứ hai, LLMs được dùng để hiểu cấu trúc của văn bản pháp luật, chẳng hạn tên văn bản, chương, mục, điều, khoản và các metadata liên quan. LLMs không thay thế bước trích xuất chữ, mà tập trung vào việc chuyển nội dung đã đọc được thành dữ liệu có cấu trúc.

Cách tách này có ba lợi ích chính:

- Giảm chi phí xử lý vì phần lớn PDF pháp luật đã có lớp chữ để trích xuất trực tiếp.
- Cho phép kiểm soát tốt hơn chất lượng nhận dạng tiếng Việt ở những tài liệu scan.
- Giúp LLMs tập trung vào nhiệm vụ hiểu cấu trúc pháp lý thay vì phải xử lý toàn bộ file từ đầu.

Kết quả sau chuẩn hóa bao gồm thông tin nhận diện văn bản, ngày hiệu lực, danh sách điều khoản và metadata phục vụ tìm kiếm, trích dẫn và quản trị.

Nếu bước trích xuất hoặc chuẩn hóa thất bại, hệ thống mới chuyển sang đường xử lý dự phòng để đọc trực tiếp tài liệu bằng LLMs. Cách thiết kế này giúp ưu tiên đường xử lý tiết kiệm hơn trong trường hợp thông thường, nhưng vẫn có phương án thay thế với những file khó đọc.

Trong cùng giai đoạn xử lý PDF, một số tác vụ không phụ thuộc trực tiếp vào nhau được tổ chức song song, chẳng hạn lưu file gốc để đối chiếu và trích xuất nội dung phục vụ chuẩn hóa. File gốc được giữ lại để quản trị viên có thể kiểm tra nguồn khi cần, trong khi phần văn bản trích xuất trở thành đầu vào cho bước nhận diện cấu trúc điều khoản. Cách tổ chức này giúp rút ngắn thời gian chờ nhưng không cần tách thành một nhánh nghiệp vụ riêng, vì mục tiêu chính của giai đoạn này vẫn là biến tài liệu PDF thành dữ liệu pháp luật có cấu trúc và có thể kiểm tra.

Kết quả chuẩn hóa không được lưu ngay mà phải đi qua bước kiểm tra hợp lệ. Bước này đảm bảo các trường quan trọng như mã văn bản, mã điều, tiêu đề, nội dung và thông tin nguồn không bị thiếu. Đây là yêu cầu cần thiết vì đầu ra do LLMs sinh ra vẫn có thể thiếu trường, sai định dạng hoặc chia tách điều khoản chưa ổn định.

2.8.3. Cơ chế hoàn tác để giữ dữ liệu nhất quán

Sau khi nội dung được chuẩn hóa, hệ thống phải ghi dữ liệu sang nhiều kho khác nhau. Kho văn bản phục vụ quản trị và hiển thị chi tiết nguồn; kho vector phục vụ truy hồi ngữ nghĩa. Do các kho này không nằm trong một giao dịch duy nhất, hệ thống cần cơ chế hoàn tác khi có lỗi ở giữa quy trình.

- MongoDB: lưu văn bản pháp luật, nội dung điều khoản và metadata phục vụ quản trị.
- ChromaDB: lưu vector của các đoạn văn bản phục vụ truy hồi ngữ nghĩa.

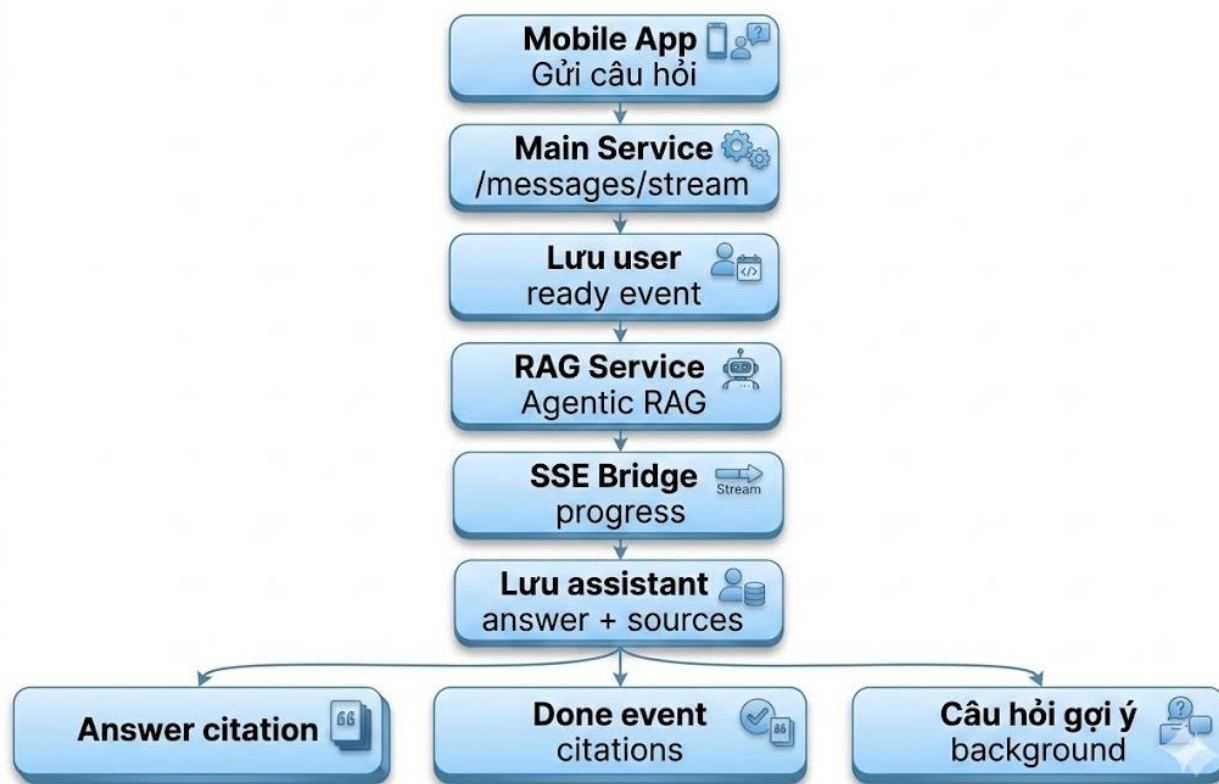
Nếu chỉ một phần dữ liệu được ghi thành công, hệ thống có thể rơi vào trạng thái không nhất quán. Ví dụ, quản trị viên thấy văn bản đã tồn tại nhưng tác nhân AI chưa tìm được văn bản đó trong kho vector. Vì vậy, một lượt cập nhật chỉ được xem là thành công khi các kho liên quan đều đã đồng bộ.

Khi một bước thất bại, tác vụ chuyển sang trạng thái lỗi và những phần dữ liệu đã ghi trước đó được thu hồi. Nguyên tắc này giúp kho tri thức luôn phản ánh trạng thái nhất quán giữa phần quản trị và phần truy hồi.

Riêng phần ghi dữ liệu vào kho vector tuân theo chiến lược đã phân tích ở mục 2.2: điều luật được chuẩn hóa thành các chunk có ngữ cảnh, mã hóa embedding và lưu kèm metadata để phục vụ truy hồi. Vì vậy, trong mục cập nhật tri thức từ

Admin, trọng tâm còn lại là xử lý đầu vào PDF và bảo đảm tính nhất quán giữa kho quản trị và kho truy hồi khi một tác vụ cập nhật thành công hoặc thất bại.

2.9. Luồng Chat SSE và Quản lý Hội thoại



Hình 2.9. Luồng trải nghiệm hỏi đáp theo thời gian thực

Bên cạnh chất lượng câu trả lời, trải nghiệm chat cũng cần cho người dùng biết hệ thống đang xử lý đến đâu. Trong luồng chat thời gian thực, câu hỏi được ghi nhận vào hội thoại, pipeline RAG được chạy ở phía sau, các trạng thái xử lý được gửi dần về giao diện và câu trả lời cuối cùng được hiển thị kèm nguồn tham chiếu.

Thiết kế này tách rõ tiến trình xử lý trung gian và nội dung pháp lý cuối cùng. Các trạng thái như đang truy hồi hoặc đang kiểm chứng chỉ giúp người dùng theo dõi tiến độ; còn câu trả lời chính thức chỉ xuất hiện sau khi pipeline hoàn tất.

2.9.1. Cơ chế trả lời theo dòng sự kiện cho chat

Luồng chat tuân theo nguyên tắc lưu dữ liệu quan trọng trước khi hiển thị kết quả cuối cùng. Câu hỏi của người dùng được ghi nhận vào lịch sử hội thoại, sau đó hệ thống mới xử lý bằng pipeline RAG. Khi có câu trả lời, kết quả tiếp tục được lưu để người dùng có thể xem lại về sau.

Nguyên tắc này phù hợp với hệ thống tư vấn pháp luật vì lịch sử hội thoại không chỉ phục vụ trải nghiệm, mà còn giúp người dùng theo dõi lại bối cảnh tư vấn và các nguồn đã được sử dụng.

2.9.2. Các trạng thái trong luồng chat

Trong luồng chat, trạng thái xử lý cần được truyền về giao diện theo cách có thể hiểu được đối với người dùng. Các trạng thái này không thay thế câu trả lời cuối cùng, nhưng giúp người dùng biết hệ thống đang kiểm tra câu hỏi, truy hồi nguồn hay kiểm chứng nội dung.

Bảng 2.13. Các sự kiện SSE của luồng chat

Event	Thời điểm phát	Nội dung chính	Vai trò trên giao diện
ready	Sau khi lưu tin nhắn người dùng	<code>conversation_id</code> , <code>is_new_conversation</code> , <code>user_message</code>	Render tin nhắn người dùng và gắn vào hội thoại đúng
progress	Khi pipeline chuyển bước	Danh sách các bước xử lý	Cập nhật vùng tiến trình: kiểm tra, phân tích, tra cứu, tổng hợp và kiểm chứng
done	Sau khi câu trả lời đã được lưu	Câu trả lời, nguồn trích dẫn và thông tin phụ trợ	Hiển thị kết quả cuối cùng cho người dùng
error	Khi lỗi xác thực hội thoại, <code>timeout</code> hoặc lỗi RAG	<code>message</code>	Hiển thị lỗi và dừng trạng thái streaming

Bảng 2.13 liệt kê các sự kiện chính được gửi trong quá trình streaming. Việc chuẩn hóa các sự kiện này giúp Mobile App cập nhật giao diện nhất quán, đồng thời tách rõ phần tiến trình xử lý và phần nội dung trả lời cuối cùng.

Các cập nhật trong quá trình streaming chỉ biểu diễn trạng thái xử lý, không phải nội dung pháp lý cuối cùng. Nhờ đó, giao diện tránh cảm giác chờ thụ động nhưng vẫn giữ nguyên tắc chỉ hiển thị kết luận khi đã qua truy hồi và kiểm chứng.

2.9.3. Quản lý lịch sử hội thoại và câu hỏi gợi ý

Lịch sử hội thoại được sử dụng để duy trì mạch trao đổi giữa người dùng và hệ thống. Trong thực tế, người dùng thường không đặt toàn bộ thông tin trong một câu hỏi duy nhất mà có xu hướng hỏi nối tiếp, ví dụ sau khi nhận câu trả lời về một

mức phạt, họ có thể hỏi tiếp về trường hợp tái phạm, giấy tờ cần chuẩn bị hoặc thủ tục khiếu nại. Nếu không truyền một phần lịch sử sang lượt xử lý mới, hệ thống sẽ khó hiểu được các đại từ như "trường hợp đó", "mức này" hoặc "văn bản trên" đang nói đến nội dung nào.

Tuy nhiên, lịch sử hội thoại chỉ được xem là ngữ cảnh giao tiếp, không phải căn cứ pháp lý. Đây là điểm rất quan trọng trong hệ thống tư vấn pháp luật. Các câu trả lời trước đó có thể giúp xác định chủ đề đang trao đổi, nhưng mỗi lượt trả lời mới vẫn phải truy hồi lại nguồn pháp luật phù hợp trong lượt hiện tại. Cách làm này tránh tình trạng hệ thống dựa vào nội dung đã trả lời trước đó rồi tiếp tục suy luận mà không kiểm chứng lại với văn bản nguồn.

Để cân bằng giữa khả năng hiểu ngữ cảnh và chi phí xử lý, hệ thống chỉ lấy một số tin nhắn gần nhất của hội thoại thay vì đưa toàn bộ lịch sử vào pipeline. Các câu trả lời của trợ lý thường dài vì chứa phân tích và trích dẫn, nên phần này được rút gọn trước khi gửi sang RAG Service. Nhờ vậy, truy vấn hiện tại vẫn giữ được bối cảnh cần thiết nhưng không làm phình to đầu vào, không tăng độ trễ quá mức và không làm giảm sự tập trung của bước phân tích câu hỏi.

Trình tự lưu dữ liệu trong luồng chat cũng được thiết kế để bảo đảm trải nghiệm ổn định. Khi người dùng gửi câu hỏi, hệ thống tạo hoặc xác định hội thoại hiện tại, sau đó lưu ngay tin nhắn người dùng và phát sự kiện `ready` để giao diện có thể hiển thị bong bóng chat tức thời. Trong lúc RAG Service xử lý, các sự kiện `progress` tiếp tục cập nhật tiến trình. Chỉ khi câu trả lời đã được tạo và lưu thành công, hệ thống mới phát sự kiện `done` kèm tin nhắn của trợ lý. Nhờ đó, dữ liệu hiển thị trên giao diện luôn khớp với dữ liệu đã lưu trong cơ sở dữ liệu.

Với hội thoại mới, tiêu đề hội thoại được suy ra từ chủ đề chính của câu hỏi nếu bước phân tích truy vấn xác định được; nếu không, hệ thống dùng một phần nội dung câu hỏi đầu tiên làm tiêu đề tạm. Cơ chế này giúp danh sách hội thoại có tên dễ nhận biết mà không cần người dùng đặt tên thủ công. Về phía người dùng, họ có thể quay lại một hội thoại cũ, tiếp tục hỏi trong cùng bối cảnh và vẫn xem được các nguồn tham chiếu đã được lưu cùng từng câu trả lời.

Sau khi câu trả lời chính hoàn tất, hệ thống sinh thêm các câu hỏi gợi ý để hỗ trợ người dùng tiếp tục khai thác vấn đề pháp lý. Các câu hỏi này không tham gia vào quá trình tạo câu trả lời chính, mà được xử lý như một tác vụ phụ sau khi tin nhắn của trợ lý đã được lưu. Cách tổ chức này giúp luồng trả lời chính không bị chậm chỉ vì phải chờ sinh câu hỏi gợi ý.

Nội dung đầu vào cho bước sinh câu hỏi gợi ý gồm câu hỏi ban đầu và phần tóm tắt ngắn của câu trả lời. Kết quả được giới hạn ở một số lượng nhỏ câu hỏi, tập trung vào những hướng hỏi tiếp có giá trị thực tế như điều kiện áp dụng, hồ sơ cần chuẩn bị, thời hạn xử lý, mức phạt liên quan hoặc quyền khiếu nại. Nếu tác vụ sinh gợi ý thất bại, hệ thống không làm lỗi toàn bộ luồng chat mà chỉ bỏ qua phần gợi ý. Điều này phù hợp với vai trò hỗ trợ của chức năng này.

Ở phía ứng dụng di động, câu hỏi gợi ý có thể xuất hiện muộn hơn câu trả lời chính. Sau khi nhận sự kiện `done`, ứng dụng tiếp tục kiểm tra trong một khoảng thời gian ngắn để lấy danh sách câu hỏi đã được tạo ở nền. Khi có kết quả, giao diện cập nhật lại tin nhắn tương ứng và hiển thị các gợi ý dưới câu trả lời. Nhờ vậy, người dùng vẫn nhận được câu trả lời nhanh, trong khi hệ thống có thêm thời gian tạo các gợi ý có chất lượng hơn.

Nhìn chung, phần quản lý lịch sử hội thoại và câu hỏi gợi ý giúp luồng chat trở thành một trải nghiệm tư vấn liên tục thay vì các lượt hỏi đáp rời rạc. Lịch sử giúp hệ thống hiểu bối cảnh, còn câu hỏi gợi ý giúp người dùng mở rộng vấn đề theo hướng tự nhiên. Dù vậy, cả hai thành phần này đều được đặt dưới nguyên tắc an toàn: chúng hỗ trợ trải nghiệm và định hướng trao đổi, nhưng căn cứ pháp lý của câu trả lời vẫn phải đến từ pipeline truy hồi và kiểm chứng trong từng lượt xử lý.

2.10. Phương pháp xây dựng lớp ứng dụng đa nền tảng

Sau khi thiết kế phần lõi backend và pipeline RAG, hệ thống cần lớp ứng dụng để người dùng cuối và quản trị viên có thể thao tác với các chức năng tương ứng. Vì mục tiêu sử dụng khác nhau, đề án tách lớp frontend thành Mobile App cho người dùng cuối và Admin Web cho quản trị viên.

2.10.1. Mobile App - Kotlin Multiplatform và Compose Multiplatform

Mobile App được xây dựng bằng Kotlin Multiplatform kết hợp Compose Multiplatform. Lựa chọn này phù hợp với định hướng phát triển phần mềm di động vì hệ thống cần chạy trên cả Android và iOS, đồng thời vẫn muốn chia sẻ các phần logic chung như mô hình dữ liệu, gọi API, xử lý dòng sự kiện và quản lý trạng thái.

Thay vì phát triển hai ứng dụng độc lập hoàn toàn, Kotlin Multiplatform cho phép chia sẻ phần logic nghiệp vụ giữa hai nền tảng:

Phần dùng chung bao gồm mô hình dữ liệu, lớp truy cập API, ánh xạ dữ liệu, xử lý stream và quản lý trạng thái. Nhờ đó, luồng chat và luồng tư vấn có hướng dẫn có thể hoạt động nhất quán trên nhiều thiết bị.

Các phần phụ thuộc đặc thù nền tảng như giao diện native, quyền hệ thống hoặc vòng đời ứng dụng vẫn được tách riêng. Cách làm này giữ được trải nghiệm phù hợp cho từng nền tảng nhưng không làm lặp lại toàn bộ logic nghiệp vụ.

Bảng 2.14. Lý do lựa chọn Kotlin Multiplatform cho Mobile App

Tiêu chí	Lý do
Chia sẻ logic nghiệp vụ	Các lớp truy cập API, ánh xạ dữ liệu, kiểm tra hợp lệ và quản lý trạng thái dùng chung giữa Android/iOS
Trải nghiệm native	Giao diện Compose chạy trực tiếp trên nền tảng, không phụ thuộc WebView
Đồng bộ tính năng	Một thay đổi ở chat streaming hoặc guided consultation được áp dụng đồng thời cho nhiều nền tảng
Phù hợp đồ án mobile	Thể hiện năng lực thiết kế ứng dụng đa nền tảng thay vì chỉ xây backend
Dễ mở rộng	Có thể bổ sung desktop JVM để demo hoặc kiểm thử nhanh

Các tiêu chí trong bảng cho thấy Kotlin Multiplatform được lựa chọn không chỉ để giảm số lượng mã nguồn phải viết, mà còn để bảo đảm các luồng nghiệp vụ quan trọng như chat, tư vấn có hướng dẫn và xử lý trạng thái hoạt động thống nhất trên Android và iOS.

2.10.2. Mô hình MVI trong Mobile App

Mobile App sử dụng mô hình MVI để tổ chức trạng thái giao diện. Mô hình này tách rõ dữ liệu đang hiển thị, hành động của người dùng và các phản hồi một lần của hệ thống.

- State mô tả toàn bộ trạng thái giao diện hiện tại.
- Intent mô tả hành động từ người dùng hoặc sự kiện vòng đời của màn hình.
- Effect mô tả các phản hồi một lần như điều hướng hoặc thông báo.
- Lớp xử lý trung gian tiếp nhận Intent, gọi nghiệp vụ cần thiết và cập nhật State.
- Màn hình chỉ hiển thị theo State hiện tại và gửi Intent khi người dùng thao tác.

Cách tổ chức này phù hợp với ứng dụng chat AI vì giao diện thay đổi liên tục: người dùng gửi tin nhắn, hệ thống nhận trạng thái xử lý, câu trả lời được cập nhật và câu hỏi gợi ý có thể xuất hiện sau đó.

Trong Mobile App, mô hình MVI được áp dụng nhất quán cho các màn hình chính nhưng không cần hiểu như một khuôn mẫu cứng. Ở màn hình đăng nhập, State chủ yếu gồm thông tin biểu mẫu, trạng thái đang gửi yêu cầu và lỗi xác thực; Intent tương ứng với các thao tác nhập liệu hoặc nhấn đăng nhập; Effect thường là điều hướng sang màn hình chính hoặc hiển thị thông báo. Với màn hình chat, State phức tạp hơn vì phải chứa danh sách tin nhắn, nội dung nhập, trạng thái streaming, các bước tiến trình và danh sách câu hỏi gợi ý.

Ở các màn hình tra cứu văn bản pháp luật, State tập trung vào danh sách văn bản, bộ lọc, từ khóa tìm kiếm và kết quả tìm kiếm bằng AI. Khi người dùng đổi chủ đề, chọn năm ban hành hoặc mở chi tiết văn bản, các thao tác này được biểu diễn thành Intent để lớp xử lý trung gian gọi API và cập nhật State. Nhờ đó, phần giao diện chỉ cần hiển thị theo State hiện tại mà không phải tự quyết định cách truy vấn dữ liệu.

Đối với luồng tư vấn có hướng dẫn, MVI đặc biệt hữu ích vì người dùng đi qua nhiều trạng thái liên tiếp: nhập câu hỏi ban đầu, nhận câu hỏi làm rõ, chọn phương án bổ sung thông tin và chờ câu trả lời cuối cùng. Việc gom các trạng thái này vào một mô hình dữ liệu thống nhất giúp giao diện xử lý được cả luồng bình thường lẫn lỗi mạng, lỗi xác thực hoặc trường hợp người dùng thay đổi lựa chọn giữa chừng.

Với MVI, các sự kiện streaming từ server được chuyển thành thay đổi trạng thái rõ ràng. Màn hình chỉ cần hiển thị theo trạng thái hiện tại, nhờ đó tránh trộn logic xử lý dữ liệu trực tiếp vào phần giao diện.

Đặc biệt với màn hình chat, tại cùng một thời điểm có thể tồn tại nhiều trạng thái như tin nhắn vừa gửi, tiến trình đang chạy, câu trả lời chưa hoàn thành hoặc danh sách câu hỏi gợi ý. MVI giúp gom các trạng thái này vào một mô hình quản lý thống nhất.

2.10.3. Thiết kế trải nghiệm chat streaming

Một vấn đề thường gặp của chatbot dùng LLMs là thời gian chờ. Với luồng RAG có kiểm chứng, hệ thống còn phải kiểm tra câu hỏi, phân tích truy vấn, truy hồi nguồn, tổng hợp và kiểm chứng. Nếu giao diện chỉ hiển thị trạng thái chờ tĩnh, người dùng khó biết hệ thống còn đang xử lý hay đã gặp lỗi.

Đề án giải quyết vấn đề này bằng cơ chế truyền tiến trình theo thời gian thực:

- Khi người dùng gửi câu hỏi, giao diện lập tức hiển thị tin nhắn và chuyển sang trạng thái đang xử lý.

- Trong quá trình xử lý, backend gửi các bước tiến trình để hiển thị như kiểm tra câu hỏi, truy hồi nguồn và kiểm chứng.
- Khi câu trả lời hoàn tất, giao diện thay thế trạng thái chờ bằng nội dung trả lời và danh sách câu hỏi gợi ý.

Việc sử dụng SSE trong trải nghiệm chat xuất phát từ đặc điểm của pipeline RAG có kiểm chứng. Người dùng không chỉ chờ một mô hình sinh câu trả lời, mà còn chờ các bước kiểm tra câu hỏi, phân tích truy vấn, truy hồi nguồn, tổng hợp và kiểm chứng. Nếu giao diện chỉ hiển thị một trạng thái chờ duy nhất, người dùng khó biết hệ thống còn hoạt động hay đã bị treo.

SSE phù hợp với luồng chat vì đây là kênh truyền một chiều từ server về client. Trong quá trình xử lý, backend chỉ cần đẩy các sự kiện tiến trình và sự kiện hoàn tất; client không cần gửi dữ liệu liên tục ngược lại như các bài toán cộng tác thời gian thực. Cơ chế này đơn giản hơn WebSocket cho trường hợp hỏi đáp, đồng thời vẫn đủ để cập nhật Thinking Panel, trạng thái đang xử lý và câu trả lời cuối cùng.

Ở góc độ trải nghiệm, streaming giúp người dùng thấy rằng hệ thống đang đi qua các bước có kiểm soát thay vì trả lời cảm tính. Các event tiến trình làm giảm cảm giác chờ, còn event hoàn tất bảo đảm câu trả lời chỉ xuất hiện khi đã được lưu và có nguồn tham chiếu. Điều này đặc biệt quan trọng với tư vấn pháp luật, nơi tốc độ cần đi kèm sự minh bạch về quá trình tra cứu.

2.10.4. Admin Web - Next.js Dashboard cho quản trị dữ liệu

Admin Web được xây dựng để phục vụ quản trị kho tri thức pháp luật. Khác với Mobile App, giao diện này không tập trung vào hội thoại mà tập trung vào các thao tác vận hành như xem thống kê, quản lý văn bản, tải văn bản mới và theo dõi tiến trình xử lý.

Các chức năng chính của Admin Web gồm:

- Đăng nhập với tài khoản quản trị.
- Xem dashboard thống kê: tổng số văn bản, tổng số điều luật, số tác vụ thành công, thất bại hoặc đang xử lý, và các chủ đề phổ biến.
- Quản lý văn bản pháp luật: phân trang, tìm kiếm, xem chi tiết, mở rộng từng điều luật và xoá văn bản.
- Tải lên file PDF văn bản mới bằng thao tác kéo thả hoặc chọn file.
- Theo dõi tiến trình xử lý tài liệu theo thời gian thực.

- Khôi phục trạng thái tải lên khi người dùng tải lại trang hoặc gián đoạn kết nối.

Do các thao tác quản trị có thể ảnh hưởng trực tiếp đến kho tri thức của hệ thống, Admin Web ưu tiên sự rõ ràng và khả năng kiểm soát. Quản trị viên cần biết văn bản nào đã được xử lý, tác vụ nào đang chạy và lỗi xảy ra ở bước nào nếu quá trình cập nhật thất bại.

Mobile App và Admin Web được tách thành hai giao diện vì phục vụ hai nhóm người dùng khác nhau. Mobile App hướng tới người dùng cuối như người dân, sinh viên, doanh nghiệp hoặc cán bộ cần tra cứu nhanh. Trọng tâm của giao diện này là hội thoại với Agent, tư vấn có hướng dẫn, tra cứu thư viện văn bản và trải nghiệm nhận phản hồi theo thời gian thực.

Admin Web lại phục vụ quản trị viên, nên trọng tâm không nằm ở hội thoại mà ở vận hành dữ liệu. Giao diện quản trị cần hỗ trợ tải lên văn bản mới, theo dõi tác vụ xử lý PDF, quan sát thống kê dữ liệu, kiểm tra trạng thái pipeline và xử lý các trường hợp cập nhật lỗi. Vì vậy, các thành phần như dashboard, bảng tác vụ, tiến trình WebSocket và thao tác quản lý văn bản được ưu tiên hơn các thành phần chat.

Sự tách biệt này giúp mỗi frontend được tối ưu theo đúng ngữ cảnh sử dụng. Mobile App cần đơn giản, nhanh và tập trung vào trải nghiệm hỏi đáp; Admin Web cần nhiều thông tin vận hành hơn, cho phép quản trị viên theo dõi trạng thái dữ liệu và phát hiện lỗi trong quá trình cập nhật tri thức. Nhờ đó, hệ thống tránh việc dồn quá nhiều chức năng vào một giao diện duy nhất.

2.10.5. WebSocket cho tiến trình xử lý tài liệu

Pipeline tải lên PDF là tác vụ nền kéo dài, gồm nhiều bước và không sinh văn bản liên tục như chat. Vì vậy, Admin Web sử dụng WebSocket để nhận trạng thái xử lý từ backend trong suốt quá trình cập nhật tài liệu.

Quy trình tổng quát:

1. Quản trị viên chọn file PDF và xác nhận tải lên.
2. Backend tạo tác vụ xử lý để theo dõi tiến trình.
3. Giao diện mở kênh nhận cập nhật trạng thái của tác vụ.
4. Backend gửi các mốc tiến trình như tiền kiểm, trích xuất, chuẩn hóa, lưu dữ liệu và tạo vector.
5. Giao diện cập nhật thanh tiến trình, trạng thái và thông báo lỗi nếu có.
6. Khi hoàn tất, văn bản mới xuất hiện trong danh sách quản trị và sẵn sàng phục vụ tra cứu.

Bảng 2.15. Các trạng thái xử lý tài liệu

Trạng thái	Ý nghĩa
pending	Task vừa được tạo, chờ xử lý
processing	Đang OCR/parse/upload/ingest
completed	Đã lưu thành công vào MongoDB và ChromaDB
failed	Có lỗi trong quá trình xử lý
cancelled	Admin hủy tác vụ

Các trạng thái trong Bảng 2.15 giúp quản trị viên biết tác vụ đang ở bước nào và lỗi xảy ra tại đâu nếu quá trình cập nhật thất bại. Đây là yếu tố quan trọng vì cập nhật văn bản pháp luật là tác vụ dài, có nhiều bước phụ thuộc nhau.

Kênh cập nhật được giới hạn theo tài khoản quản trị để mỗi người chỉ nhìn thấy tiến trình của tác vụ do mình khởi tạo. Điều này cần thiết khi hệ thống có nhiều quản trị viên cùng vận hành.

So với việc hỏi trạng thái định kỳ, WebSocket phù hợp hơn vì tiến trình xử lý không thay đổi theo chu kỳ cố định. Backend chỉ gửi cập nhật khi có trạng thái mới, nhờ đó giao diện phản hồi kịp thời hơn và giảm số lượng request không cần thiết.

2.10.6. Kết nối giữa frontend và backend

Cả Mobile App và Admin Web đều giao tiếp với hệ thống thông qua Main Service. Cách thiết kế này giữ RAG Service ở phía nội bộ, đồng thời tập trung xác thực, phân quyền và điều phối nghiệp vụ tại một điểm thống nhất.

- Bảo vệ RAG Service khỏi truy cập trực tiếp từ phía client.
- Tập trung xác thực, phân quyền và ghi nhận nhật ký nghiệp vụ ở Main Service.
- Giữ cho lớp frontend chỉ cần làm việc với một cổng API thống nhất.

Mobile App tập trung vào các nhóm chức năng phục vụ người dùng cuối, bao gồm xác thực, quản lý hội thoại, tra cứu thư viện pháp luật và luồng tư vấn có hướng dẫn.

Admin Web tập trung vào các nhóm chức năng phục vụ quản trị, bao gồm theo dõi thống kê hệ thống, quản lý văn bản pháp luật, tải văn bản mới và giám sát tiến trình xử lý tài liệu.

Như vậy, lớp ứng dụng không chỉ là phần hiển thị giao diện. Nó phản ánh các quyết định kiến trúc của hệ thống: tách trải nghiệm người dùng cuối khỏi giao diện

quản trị, dùng dòng sự kiện cho chat, dùng WebSocket cho tác vụ nền và chia sẻ logic mobile đa nền tảng.

2.11. Tổng kết chương 2

Chương 2 đã trình bày phương pháp xây dựng hệ thống Vietnam Law Chatbot từ kiến trúc tổng thể đến các luồng xử lý trọng tâm. Hệ thống được tổ chức theo kiến trúc microservices, trong đó Main Service đảm nhiệm điều phối nghiệp vụ, RAG Service đảm nhiệm truy hồi và suy luận, còn các kho dữ liệu lưu trữ tri thức pháp luật, vector tìm kiếm và lịch sử sử dụng.

Đối với chức năng hỏi đáp, báo cáo đã phân tích cách hệ thống kết hợp tác nhân AI, truy hồi hai giai đoạn, đối chiếu nguồn cập nhật và kiểm chứng đầu ra để tạo câu trả lời có căn cứ. Đối với chức năng tư vấn có hướng dẫn, hệ thống bổ sung bước làm rõ ngữ cảnh trước khi trả lời nhằm hạn chế suy đoán trong các câu hỏi thiếu dữ kiện.

Đối với dữ liệu pháp luật, báo cáo đã tách rõ hai giai đoạn: xây dựng kho vector ban đầu từ JSON và cập nhật văn bản mới từ PDF qua Admin. Giai đoạn đầu chuẩn hóa article, chunking, tạo embedding và lưu vào ChromaDB; giai đoạn sau xử lý PDF, kiểm tra hợp lệ, đồng bộ MongoDB/ChromaDB và hoàn tác khi có lỗi. Cách phân tách này phản ánh đúng quá trình phát triển của hệ thống: có kho tri thức nền trước, sau đó mới bổ sung cơ chế cập nhật trong quá trình vận hành.

Các nội dung trên tạo cơ sở cho chương 3, nơi báo cáo chuyển sang phân tích và thiết kế hệ thống ở mức chi tiết hơn, bao gồm yêu cầu chức năng, mô hình dữ liệu, API và các biểu đồ xử lý.

CHƯƠNG 3: PHÂN TÍCH, THIẾT KẾ HỆ THỐNG

Trên cơ sở phương pháp xây dựng hệ thống đã trình bày ở chương 2, chương này đi vào phân tích yêu cầu và thiết kế hệ thống Vietnam Law Chatbot ở mức chức năng. Mục tiêu của chương là làm rõ hệ thống cần đáp ứng những yêu cầu nào, các nhóm tác nhân tương tác với hệ thống ra sao, dữ liệu được tổ chức như thế nào và các thành phần trao đổi thông tin qua những API chính nào.

Nội dung chương được triển khai theo tiến trình phân tích và thiết kế hệ thống phần mềm. Trước hết, chương xác định yêu cầu về hình thức sản phẩm, yêu cầu chức năng và yêu cầu phi chức năng. Tiếp theo, các ca sử dụng trọng tâm được đặc tả và minh họa bằng biểu đồ. Sau đó, chương trình bày các luồng tuần tự đại diện cho quá trình xử lý nghiệp vụ, cuối cùng là thiết kế cơ sở dữ liệu và API làm nền tảng cho phần xây dựng, thực nghiệm ở chương sau.

3.1. Yêu cầu của hệ thống

3.1.1. Yêu cầu về hình thức sản phẩm

Hệ thống được thiết kế theo mô hình nhiều thành phần, trong đó mỗi thành phần đảm nhận một vai trò riêng trong quá trình phục vụ người dùng và quản trị dữ liệu. Việc tách ứng dụng di động, trang quản trị và các dịch vụ backend giúp hệ thống dễ bảo trì hơn, đồng thời cho phép phân xử lý nghiệp vụ và phân xử lý tri thức pháp luật được mở rộng độc lập khi khối lượng dữ liệu hoặc số lượng người dùng tăng lên.

Bảng 3.1. Yêu cầu về hình thức sản phẩm

Thành phần	Nền tảng	Vai trò
Mobile App	Android và iOS	Cung cấp chức năng hỏi đáp, tư vấn có hướng dẫn, tra cứu thư viện pháp luật và quản lý hội thoại cho người dùng cuối.
Admin Web	Trình duyệt web	Cung cấp giao diện quản trị dữ liệu pháp luật, tải lên văn bản, theo dõi tiến trình xử lý và xem thống kê hệ thống.
Main Service	Backend service	Xác thực người dùng, quản lý hội thoại, điều phối yêu cầu hỏi đáp, quản lý văn bản và kết nối với các kho dữ liệu.
RAG Service	Backend service nội bộ	Thực hiện truy hồi ngữ nghĩa, xử lý Agentic RAG, tư vấn có hướng dẫn, tạo embedding và lưu vector vào ChromaDB.

3.1.2. Yêu cầu chức năng

3.1.2.1. Yêu cầu chức năng của hệ thống đối với người dùng

Đối với người dùng cuối, các yêu cầu chức năng được xây dựng xoay quanh quá trình đặt câu hỏi pháp luật, theo dõi lịch sử hội thoại và tra cứu lại căn cứ pháp lý khi cần. Ngoài chức năng hỏi đáp trực tiếp, hệ thống còn cần hỗ trợ hình thức tư vấn có hướng dẫn để xử lý các tình huống người dùng chưa cung cấp đủ ngữ cảnh.

Bảng 3.2. Yêu cầu chức năng đối với người dùng

Mã	Chức năng	Mô tả
F-01	Đăng ký, đăng nhập và quản lý tài khoản	Người dùng tạo tài khoản, đăng nhập, làm mới phiên đăng nhập, cập nhật thông tin cá nhân và đổi mật khẩu.
F-02	Quản lý hội thoại	Người dùng xem danh sách hội thoại, mở chi tiết hội thoại, đổi tên, ghim, lưu trữ hoặc xóa hội thoại.
F-03	Gửi câu hỏi pháp luật	Người dùng gửi câu hỏi tự nhiên và nhận câu trả lời từ hệ thống theo cơ chế trả tiền trình xử lý.
F-04	Xem câu hỏi gợi ý	Sau khi câu trả lời hoàn tất, người dùng có thể xem các câu hỏi tiếp theo được gợi ý dựa trên nội dung trả lời.
F-05	Tư vấn có hướng dẫn	Hệ thống hỏi thêm các thông tin làm rõ trước khi trả lời trong các trường hợp câu hỏi ban đầu còn thiếu ngữ cảnh.
F-06	Tra cứu thư viện pháp luật	Người dùng xem danh sách văn bản, lọc theo năm, chủ đề, từ khóa và xem chi tiết điều luật.
F-07	Tìm kiếm ngữ nghĩa	Người dùng nhập truy vấn tự nhiên, xem danh sách điều luật liên quan, mở nguồn dẫn hoặc điều chỉnh từ khóa để tìm lại.

3.1.2.2. Yêu cầu chức năng của hệ thống đối với quản trị viên

Đối với quản trị viên, trọng tâm của hệ thống là duy trì kho dữ liệu pháp luật luôn sẵn sàng cho các chức năng hỏi đáp và tra cứu. Các thao tác quản trị không chỉ dừng ở việc tải lên văn bản, mà còn bao gồm theo dõi tiến trình xử lý, kiểm tra kết quả cập nhật, quản lý các tác vụ phát sinh và quan sát tình trạng hoạt động chung của hệ thống.

Bảng 3.3. Yêu cầu chức năng đối với quản trị viên

Mã	Chức năng	Mô tả
F-08	Đăng nhập quản trị	Quản trị viên đăng nhập bằng tài khoản có quyền quản trị viên để truy cập trang tổng quan.

F-09	Tải lên văn bản pháp luật	Quản trị viên chọn tệp PDF, tải lên hệ thống, theo dõi kết quả xử lý và có thể huỷ yêu cầu khi quá trình chưa hoàn tất.
F-10	Theo dõi tiến trình xử lý	Quản trị viên xem danh sách tác vụ xử lý tài liệu, theo dõi tiến trình, xem trạng thái thành công/thất bại, huỷ tác vụ đang chạy và xem thống kê tổng quan.
F-11	Quản lý văn bản pháp luật	Quản trị viên xem danh sách văn bản, tìm kiếm, xem chi tiết, làm mới dữ liệu hiển thị hoặc xóa văn bản khỏi hệ thống.
F-12	Quản lý tác vụ xử lý tài liệu	Quản trị viên xem lịch sử tác vụ, huỷ tác vụ đang chạy hoặc xóa nhật ký tác vụ không còn cần thiết.
F-13	Xem thống kê hệ thống	Quản trị viên theo dõi tổng số văn bản, tổng số điều luật, trạng thái các tác vụ xử lý, số tác vụ thành công/thất bại/đang xử lý và nhóm chủ đề phổ biến trong kho dữ liệu.

3.1.3. Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, hệ thống cần đáp ứng một số yêu cầu phi chức năng để đảm bảo có thể sử dụng ổn định trong bối cảnh tư vấn pháp luật. Các yêu cầu này tập trung vào bảo mật, hiệu năng, tính nhất quán dữ liệu và khả năng mở rộng.

Bảng 3.4. Yêu cầu phi chức năng

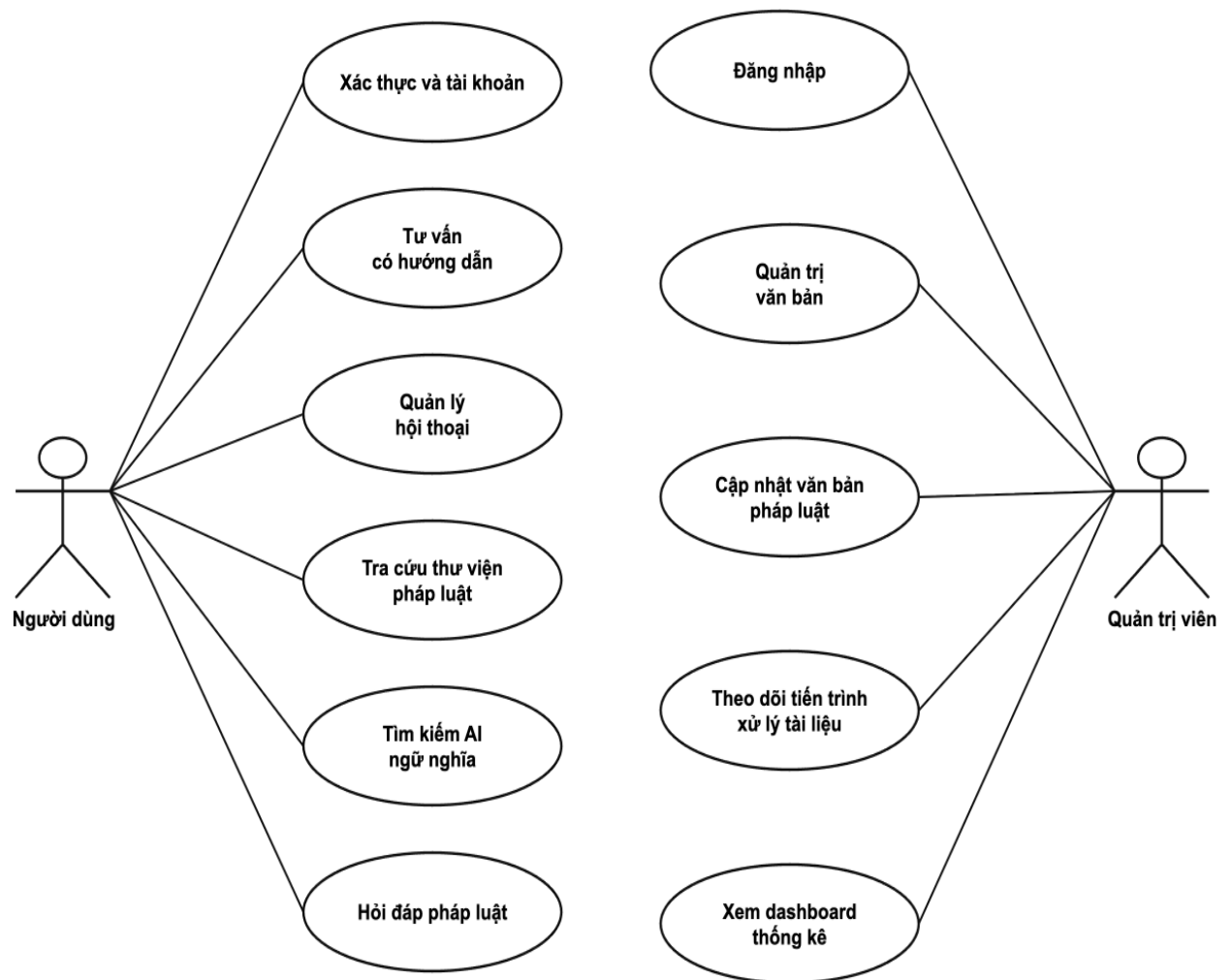
Nhóm	Yêu cầu	Mức độ
Bảo mật	Các API nghiệp vụ yêu cầu xác thực bằng access token; refresh token được lưu và có thể thu hồi.	Bắt buộc
Phân quyền	Các chức năng tải lên văn bản, xóa văn bản, làm mới dữ liệu hiển thị và xem tác vụ tài liệu chỉ dành cho quản trị viên.	Bắt buộc
Hiệu năng	Luồng hỏi đáp và tư vấn có hướng dẫn phải trả trạng thái xử lý sớm để người dùng nhận biết hệ thống đang thực hiện yêu cầu.	Bắt buộc
Tính nhất quán	Khi cập nhật văn bản mới, MongoDB, ChromaDB và dữ liệu hiển thị phải được cập nhật đồng bộ; nếu lỗi cần hoàn tác dữ liệu đã ghi.	Bắt buộc
Khả năng mở rộng	Main Service và RAG Service tách riêng để có thể mở rộng phần xử lý AI độc lập với phần nghiệp vụ.	Khuyến nghị

Trải nghiệm	Giao diện hỏi đáp cần hiển thị tiến trình xử lý, nguồn tham chiếu và câu hỏi gợi ý sau câu trả lời.	Khuyến nghị
-------------	---	-------------

3.2. Phân tích hệ thống

3.2.1. Biểu đồ ca sử dụng tổng quát

Biểu đồ ca sử dụng tổng quát thể hiện phạm vi chức năng của hệ thống dưới góc nhìn người sử dụng. Hệ thống có hai nhóm tác nhân chính: người dùng cuối và quản trị viên. Người dùng cuối sử dụng các chức năng phục vụ nhu cầu hỏi đáp, tư vấn và tra cứu pháp luật; quản trị viên chịu trách nhiệm vận hành dữ liệu, cập nhật văn bản và theo dõi trạng thái xử lý tài liệu.



Hình 3.1. Biểu đồ ca sử dụng tổng quát của hệ thống Vietnam Law Chatbot

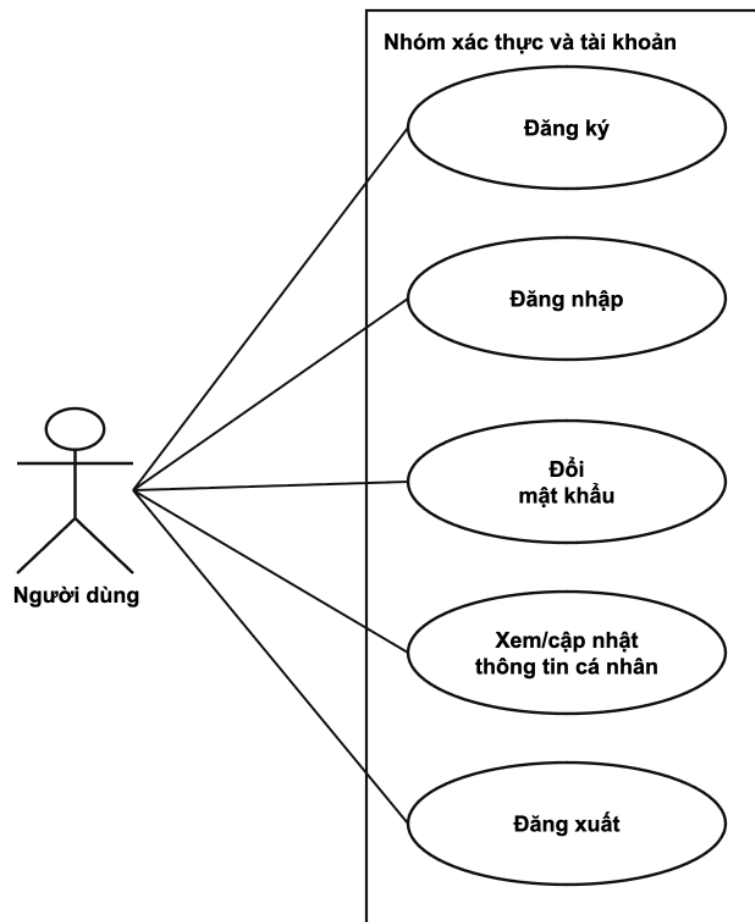
3.2.2. Đặc tả các ca sử dụng

Trong phạm vi đề án, các ca sử dụng được đặc tả theo nhóm chức năng chính thay vì tách riêng toàn bộ thao tác con thành nhiều bảng nhỏ. Cách trình bày này phù

hợp với hệ thống có nhiều thao tác liên quan chặt chẽ với nhau, giúp phần báo cáo giữ được mạch đọc liền mạch nhưng vẫn thể hiện đầy đủ mục tiêu sử dụng, tác nhân, luồng chính, luồng thay thế và kết quả của từng nhóm chức năng. Những thao tác chi tiết bên trong từng nhóm tiếp tục được làm rõ ở phần biểu đồ tuần tự.

3.2.2.1. Ca sử dụng đăng ký, đăng nhập và quản lý tài khoản

Nhóm ca sử dụng này tập trung vào các thao tác mở đầu phiên làm việc. Việc gom đăng ký, đăng nhập, làm mới phiên và cập nhật tài khoản vào cùng một đặc tả giúp thể hiện rõ vòng đời xác thực của người dùng mà không tách rời các bước có quan hệ phụ thuộc.



Hình 3.2. Ca sử dụng đăng ký, đăng nhập và quản lý tài khoản

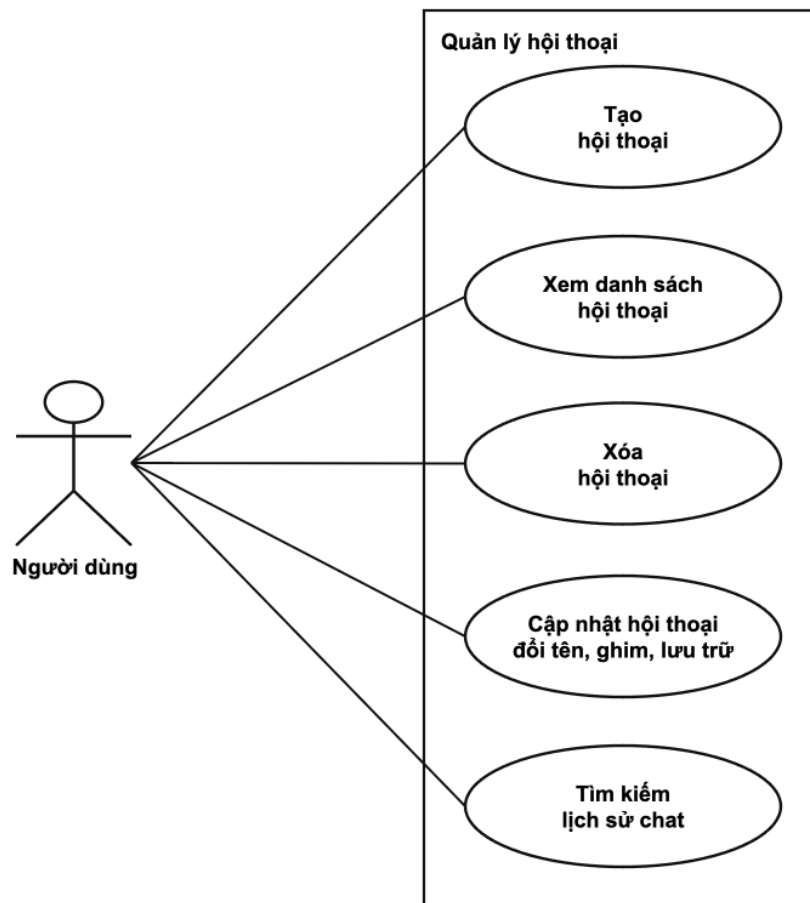
Bảng 3.5. Đặc tả ca sử dụng đăng ký, đăng nhập và quản lý tài khoản

Tên ca sử dụng	UC – Đăng ký, đăng nhập và quản lý tài khoản
Tác nhân	Người dùng
Mô tả	Ca sử dụng mô tả quá trình người dùng tạo tài khoản, đăng nhập vào ứng dụng, duy trì phiên sử dụng và cập nhật các thông tin cá nhân cơ bản trong hệ thống.

Luồng chính	1. Người dùng mở ứng dụng và chọn chức năng đăng ký hoặc đăng nhập. 2. Người dùng nhập email, mật khẩu và các thông tin cần thiết. 3. Giao diện kiểm tra định dạng dữ liệu trước khi gửi yêu cầu đến hệ thống. 4. Hệ thống kiểm tra tài khoản trong CSDL, xác thực mật khẩu hoặc tạo tài khoản mới. 5. Nếu thông tin hợp lệ, hệ thống trả về phiên đăng nhập và thông tin người dùng. 6. Ứng dụng lưu phiên đăng nhập, chuyển người dùng vào màn hình chính. 7. Khi cần, người dùng có thể xem hồ sơ, cập nhật thông tin cá nhân hoặc đổi mật khẩu.
Luồng thay thế	Nếu email đã tồn tại, mật khẩu không đúng, token hết hạn hoặc dữ liệu cập nhật không hợp lệ, hệ thống trả thông báo lỗi để người dùng nhập lại.
Kết quả	Người dùng đăng nhập thành công hoặc thông tin tài khoản được cập nhật đúng trong hệ thống.
Yêu cầu đặc biệt	Mật khẩu phải được kiểm tra và lưu trữ an toàn; các chức năng cập nhật tài khoản yêu cầu người dùng đã xác thực.

3.2.2.2. *Cả sử dụng quản lý danh sách cuộc trò chuyện*

Quản lý hội thoại được đặt thành một nhóm riêng vì đây là dữ liệu gắn trực tiếp với trải nghiệm hỏi đáp của người dùng. Các thao tác xem danh sách, mở chi tiết, ghim, lưu trữ và xoá đều nhằm giúp người dùng kiểm soát lịch sử trao đổi của mình trong hệ thống.



Hình 3.3. Ca sử dụng quản lý danh sách cuộc trò chuyện

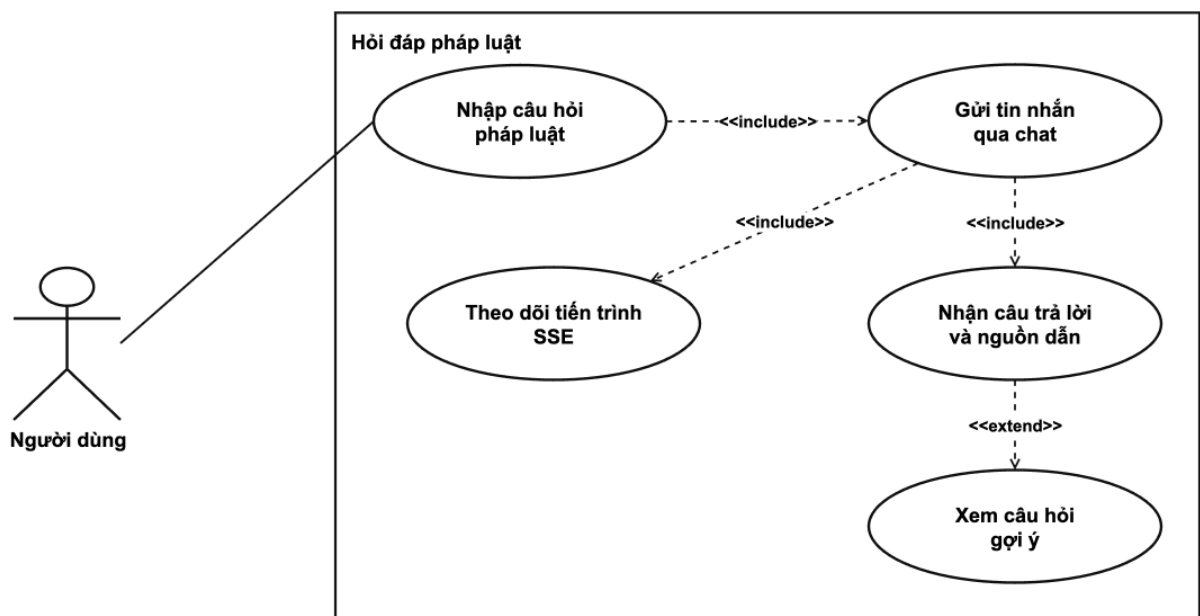
Bảng 3.6. Đặc tả ca sử dụng quản lý danh sách cuộc trò chuyện

Tên ca sử dụng	UC – Quản lý danh sách cuộc trò chuyện
Tác nhân	Người dùng
Mô tả	Ca sử dụng mô tả quá trình người dùng xem, mở và quản lý các cuộc trò chuyện đã tạo trong quá trình sử dụng chức năng hỏi đáp pháp luật.
Luồng chính	1. Người dùng truy cập màn hình danh sách hội thoại. 2. Hệ thống lấy danh sách hội thoại của người dùng, kèm tin nhắn gần nhất và trạng thái ghim/lưu trữ. 3. Giao diện hiển thị danh sách hội thoại theo thứ tự thời gian hoặc theo trạng thái ưu tiên. 4. Người dùng chọn một hội thoại để xem lại nội dung. 5. Hệ thống trả về chi tiết hội thoại và danh sách tin nhắn tương ứng. 6. Người dùng có thể đổi tên, ghim, lưu trữ hoặc xóa hội thoại. 7. Hệ thống cập nhật trạng thái hội thoại và làm mới danh sách hiển thị.

Luồng thay thế	Nếu hội thoại không tồn tại, đã bị xoá hoặc không thuộc người dùng hiện tại, hệ thống thông báo lỗi và không cho phép thao tác.
Kết quả	Danh sách hội thoại và nội dung hội thoại được hiển thị đúng theo dữ liệu của người dùng.
Yêu cầu đặc biệt	Mỗi người dùng chỉ được xem và thao tác với các hội thoại thuộc tài khoản của mình.

3.2.2.3. Ca sử dụng gửi câu hỏi pháp luật

Ca sử dụng gửi câu hỏi pháp luật là nghiệp vụ trung tâm của ứng dụng di động. Phần đặc tả nhân mạnh thao tác của người dùng và kết quả nhận được, còn các bước truy hồi, sinh câu trả lời và lưu nguồn tham chiếu được trình bày chi tiết hơn ở nhóm biểu đồ tuần tự.



Hình 3.4. Ca sử dụng gửi câu hỏi pháp luật

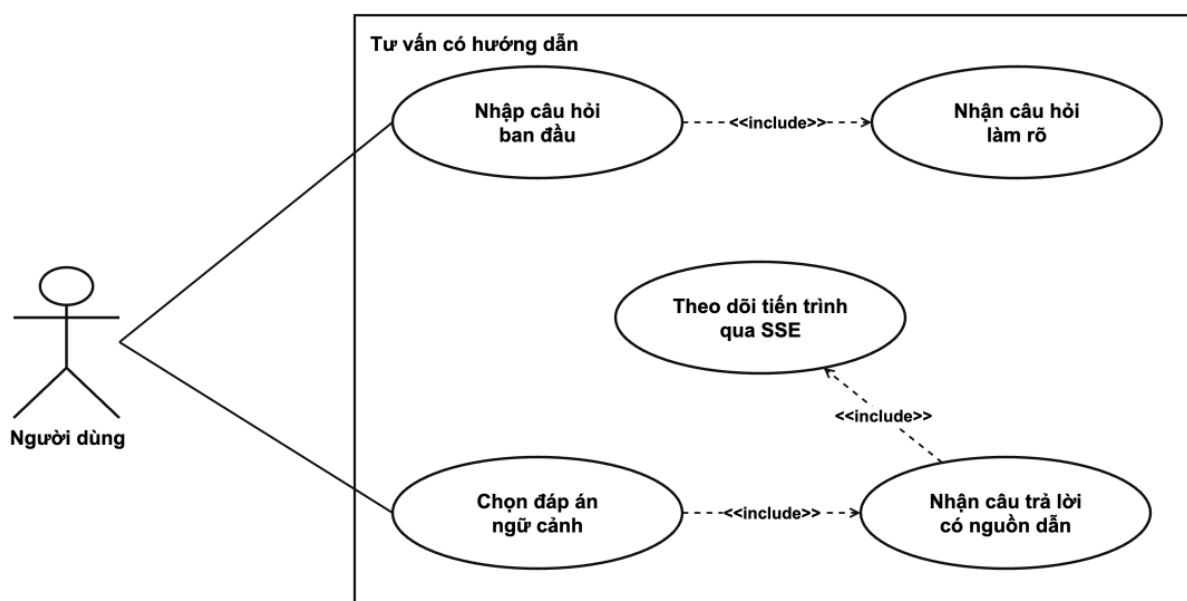
Bảng 3.7. Đặc tả ca sử dụng gửi câu hỏi pháp luật

Tên ca sử dụng	UC – Gửi câu hỏi pháp luật
Tác nhân	Người dùng
Mô tả	Ca sử dụng mô tả quá trình người dùng gửi câu hỏi pháp luật bằng ngôn ngữ tự nhiên và nhận câu trả lời có kèm nguồn tham chiếu từ hệ thống.
Luồng chính	1. Người dùng mở màn hình hội thoại và nhập câu hỏi pháp luật. 2. Giao diện kiểm tra nội dung nhập và gửi yêu cầu đến hệ thống. 3. Hệ thống tạo mới hoặc sử dụng hội thoại hiện có, lưu câu hỏi của người dùng.

	4. Hệ thống truy hồi dữ liệu pháp luật liên quan và xử lý câu trả lời. 5. Trong quá trình xử lý, giao diện hiển thị tiến trình để người dùng biết yêu cầu đang được thực hiện. 6. Hệ thống trả về câu trả lời, nguồn tham chiếu và lưu lại tin nhắn phản hồi. 7. Giao diện hiển thị câu trả lời và có thể hiển thị thêm các câu hỏi gợi ý liên quan.
Luồng thay thế	Nếu câu hỏi rỗng, hội thoại không hợp lệ hoặc hệ thống không xử lý được yêu cầu, giao diện hiển thị thông báo lỗi phù hợp.
Kết quả	Người dùng nhận được câu trả lời pháp luật, nguồn tham chiếu và lịch sử hội thoại được cập nhật.
Yêu cầu đặc biệt	Câu trả lời cần ưu tiên căn cứ từ dữ liệu pháp luật đã lưu và phải hiển thị nguồn để người dùng kiểm chứng.

3.2.2.4. Ca sử dụng tư vấn có hướng dẫn

Tư vấn có hướng dẫn được tách khỏi hỏi đáp thông thường vì luồng sử dụng có thêm bước làm rõ thông tin. Cách tổ chức này phản ánh những tình huống pháp lý cần biết thêm ngữ cảnh trước khi hệ thống có thể đưa ra câu trả lời phù hợp.



Hình 3.5. Ca sử dụng tư vấn có hướng dẫn

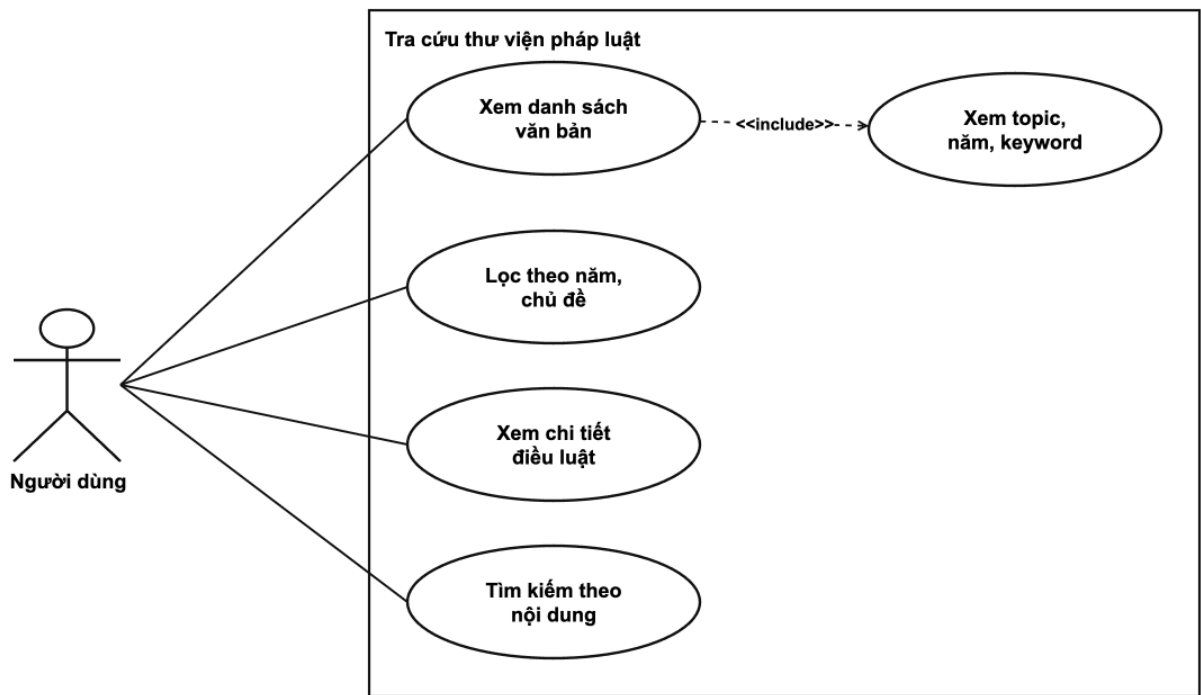
Bảng 3.8. Đặc tả ca sử dụng tư vấn có hướng dẫn

Tên ca sử dụng	UC – Tư vấn có hướng dẫn
Tác nhân	Người dùng

Mô tả	Ca sử dụng mô tả quá trình hệ thống hỗ trợ người dùng làm rõ tình huống pháp lý trước khi đưa ra nội dung tư vấn cuối cùng.
Luồng chính	1. Người dùng mở chức năng tư vấn có hướng dẫn và nhập vấn đề cần tư vấn. 2. Hệ thống phân tích câu hỏi ban đầu để xác định chủ đề và mức độ thiếu thông tin. 3. Nếu cần thêm ngữ cảnh, hệ thống tạo danh sách câu hỏi làm rõ. 4. Người dùng chọn phương án trả lời hoặc bổ sung thêm thông tin. 5. Hệ thống sử dụng câu hỏi ban đầu và các thông tin làm rõ để truy hồi căn cứ pháp luật phù hợp. 6. Hệ thống tạo câu trả lời tư vấn cuối cùng, kèm nguồn tham chiếu. 7. Giao diện hiển thị kết quả tư vấn cho người dùng.
Luồng thay thế	Nếu vấn đề không thuộc phạm vi pháp luật, thiếu thông tin nghiêm trọng hoặc phát sinh lỗi xử lý, hệ thống thông báo lý do và yêu cầu người dùng điều chỉnh thông tin.
Kết quả	Người dùng nhận được câu trả lời tư vấn cụ thể hơn so với hỏi đáp trực tiếp.
Yêu cầu đặc biệt	Các câu hỏi làm rõ cần ngắn gọn, dễ chọn và liên quan trực tiếp đến tình huống pháp lý người dùng đưa ra.

3.2.2.5. Ca sử dụng tra cứu thư viện pháp luật

Tra cứu thư viện pháp luật phục vụ nhu cầu đọc và đối chiếu văn bản theo cấu trúc có sẵn. Ca sử dụng này khác với hỏi đáp ở chỗ người dùng chủ động lọc, mở danh sách văn bản và xem nội dung điều luật thay vì yêu cầu hệ thống tổng hợp câu trả lời.



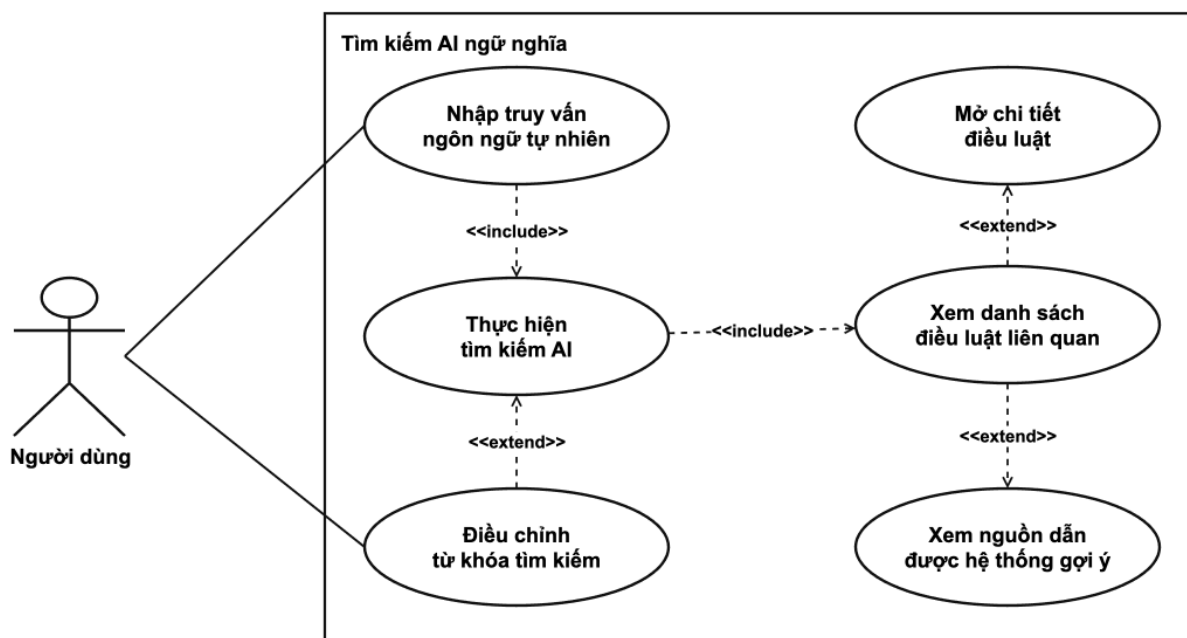
Hình 3.6. Ca sử dụng tra cứu thư viện pháp luật

Bảng 3.9. Đặc tả ca sử dụng tra cứu thư viện pháp luật

Tên ca sử dụng	UC – Tra cứu thư viện pháp luật
Tác nhân	Người dùng
Mô tả	Ca sử dụng mô tả quá trình người dùng xem danh sách văn bản pháp luật, lọc dữ liệu và mở chi tiết văn bản hoặc điều luật trong thư viện.
Luồng chính	1. Người dùng mở màn hình thư viện pháp luật. 2. Hệ thống trả về danh sách văn bản đã được lưu trong CSDL. 3. Người dùng nhập từ khoá hoặc chọn bộ lọc theo năm, chủ đề, mã văn bản. 4. Hệ thống truy vấn dữ liệu phù hợp với điều kiện lọc và trả danh sách kết quả. 5. Người dùng chọn một văn bản hoặc điều luật muốn xem. 6. Hệ thống trả nội dung chi tiết của văn bản hoặc điều luật. 7. Giao diện hiển thị nội dung để người dùng đọc và đối chiếu.
Luồng thay thế	Nếu không tìm thấy dữ liệu phù hợp, hệ thống hiển thị trạng thái rỗng và cho phép người dùng thay đổi điều kiện tìm kiếm.
Kết quả	Người dùng tra cứu được văn bản hoặc điều luật cần xem trong thư viện pháp luật.
Yêu cầu đặc biệt	Dữ liệu hiển thị cần giữ nguyên nội dung pháp luật và phân tách rõ văn bản, điều luật, chủ đề, năm ban hành.

3.2.2.6. Ca sử dụng tìm kiếm ngữ nghĩa

Tìm kiếm ngữ nghĩa mở rộng chức năng thư viện bằng cách cho phép nhập truy vấn tự nhiên. Đặc tả tập trung vào tương tác người dùng với danh sách kết quả, trong khi cơ chế embedding và truy hồi vector được trình bày ở phần thiết kế và biểu đồ tuần tự.



Hình 3.7. Ca sử dụng tìm kiếm ngữ nghĩa

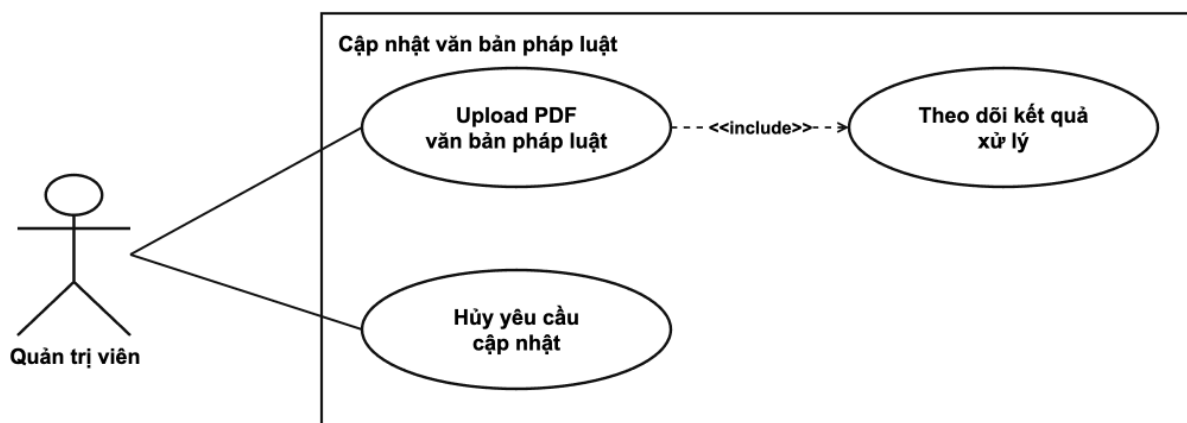
Bảng 3.10. Đặc tả ca sử dụng tìm kiếm ngữ nghĩa

Tên ca sử dụng	UC – Tìm kiếm ngữ nghĩa văn bản pháp luật
Tác nhân	Người dùng
Mô tả	Ca sử dụng mô tả quá trình người dùng tìm điều luật liên quan bằng truy vấn tự nhiên thay vì chỉ tìm theo từ khóa chính xác.
Luồng chính	1. Người dùng mở thư viện pháp luật và bật chế độ tìm kiếm ngữ nghĩa. 2. Người dùng nhập truy vấn bằng ngôn ngữ tự nhiên. 3. Hệ thống phân tích truy vấn, truy hồi dữ liệu từ CSDL Vector và xếp hạng kết quả. 4. Giao diện hiển thị danh sách điều luật liên quan cùng điểm phù hợp hoặc thông tin nguồn. 5. Người dùng chọn một kết quả để xem chi tiết. 6. Hệ thống trả nội dung điều luật tương ứng và giao diện hiển thị cho người dùng.

Luồng thay thế	Nếu truy vấn quá ngắn, không có kết quả phù hợp hoặc hệ thống truy hồi thất bại, giao diện hiển thị thông báo và gợi ý người dùng thay đổi truy vấn.
Kết quả	Người dùng nhận được danh sách điều luật liên quan đến nội dung truy vấn tự nhiên.
Yêu cầu đặc biệt	Kết quả tìm kiếm cần ưu tiên điều luật có độ liên quan cao và cho phép người dùng mở nguồn chi tiết để kiểm chứng.

3.2.2.7. Ca sử dụng cập nhật văn bản pháp luật

Cập nhật văn bản pháp luật là nghiệp vụ quản trị làm thay đổi trực tiếp kho tri thức của hệ thống. Vì vậy, phần đặc tả nêu rõ các điều kiện kiểm tra, kết quả xử lý và yêu cầu đồng bộ dữ liệu sau khi quản trị viên tải lên tệp PDF.



Hình 3.8. Ca sử dụng cập nhật văn bản pháp luật

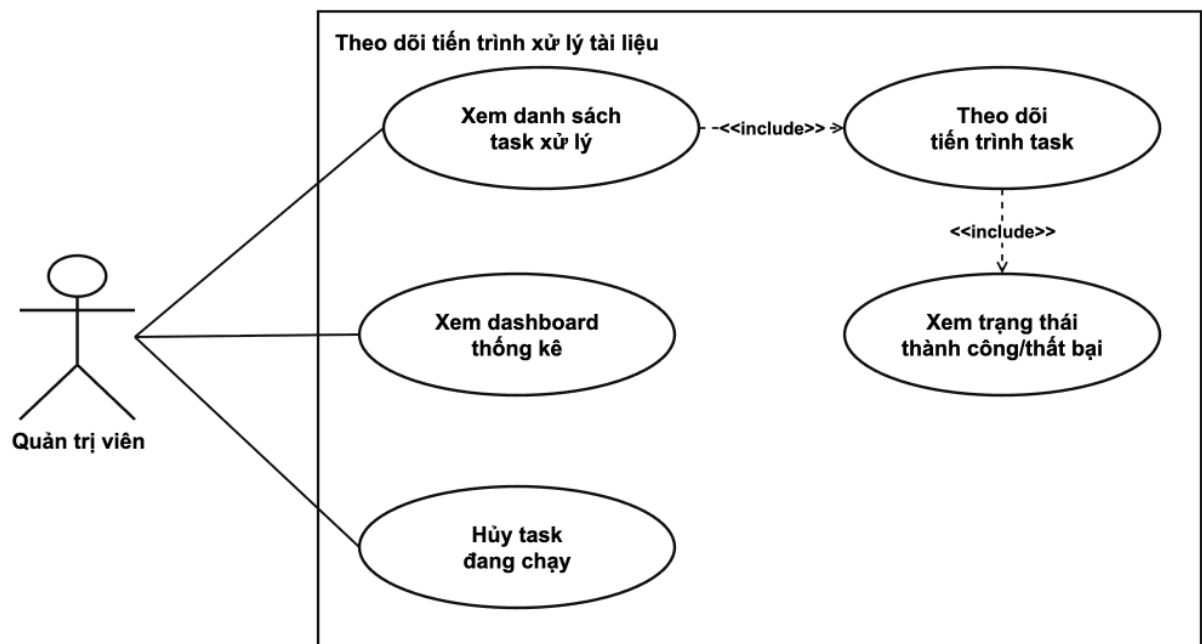
Bảng 3.11. Đặc tả ca sử dụng cập nhật văn bản pháp luật

Tên ca sử dụng	UC – Cập nhật văn bản pháp luật
Tác nhân	Quản trị viên
Mô tả	Ca sử dụng mô tả quá trình quản trị viên tải lên tệp PDF văn bản pháp luật để hệ thống trích xuất, kiểm tra và cập nhật vào kho dữ liệu.
Luồng chính	1. Quản trị viên đăng nhập vào trang quản trị. 2. Quản trị viên mở chức năng cập nhật văn bản pháp luật và chọn tệp PDF. 3. Giao diện kiểm tra định dạng, kích thước tệp và gửi yêu cầu tải lên. 4. Hệ thống tạo tác vụ xử lý và thông báo trạng thái tiếp nhận. 5. Hệ thống kiểm tra trùng lặp, trích xuất nội dung, chuẩn hoá thông tin văn bản và điều luật. 6. Hệ thống lưu dữ liệu vào CSDL văn bản, tạo embedding và cập

	nhật CSDL Vector. 7. Khi hoàn tất, hệ thống cập nhật trạng thái tác vụ và giao diện hiển thị kết quả xử lý.
Luồng thay thế	Nếu tệp không hợp lệ, văn bản đã tồn tại, quá trình trích xuất lỗi hoặc lưu dữ liệu thất bại, hệ thống cập nhật trạng thái lỗi và thông báo nguyên nhân cho quản trị viên. Nếu quản trị viên hủy tác vụ, hệ thống chuyển tác vụ sang trạng thái đã hủy.
Kết quả	Văn bản pháp luật mới được cập nhật vào hệ thống hoặc tác vụ xử lý được ghi nhận với trạng thái lỗi/hủy rõ ràng.
Yêu cầu đặc biệt	Quá trình cập nhật phải bảo đảm đồng bộ giữa CSDL văn bản, dữ liệu hiển thị và CSDL Vector; khi lỗi cần tránh để dữ liệu bị lệch giữa các kho.

3.2.2.8. Ca sử dụng theo dõi tiến trình xử lý tài liệu

Theo dõi tiến trình xử lý tài liệu hỗ trợ quản trị viên quan sát các tác vụ đang chạy và xử lý các tình huống lỗi. Nhóm ca sử dụng này bổ sung cho luồng cập nhật văn bản bằng cách mô tả phần giám sát, hủy tác vụ và quản lý lịch sử xử lý.



Hình 3.9. Ca sử dụng theo dõi tiến trình xử lý tài liệu

Bảng 3.12. Đặc tả ca sử dụng theo dõi tiến trình xử lý tài liệu

Tên ca sử dụng	UC – Theo dõi tiến trình xử lý tài liệu
Tác nhân	Quản trị viên

Mô tả	Ca sử dụng mô tả quá trình quản trị viên theo dõi trạng thái các tác vụ xử lý văn bản pháp luật trên trang quản trị.
Luồng chính	1. Quản trị viên đăng nhập vào trang quản trị. 2. Quản trị viên mở trang thống kê hoặc trang theo dõi tiến trình xử lý tài liệu. 3. Hệ thống trả về danh sách tác vụ gần đây, trạng thái, tiến độ, tên tệp, mã văn bản và số điều luật nếu có. 4. Giao diện thiết lập kênh nhận cập nhật theo thời gian thực cho các tác vụ đang chạy. 5. Khi tác vụ thay đổi trạng thái, hệ thống gửi cập nhật để giao diện làm mới tiến trình. 6. Quản trị viên có thể mở tác vụ để xem lỗi, quay lại trang tải lên, hủy tác vụ đang chạy hoặc xóa nhật ký tác vụ không cần thiết.
Luồng thay thế	Nếu không có tác vụ nào, giao diện hiển thị trạng thái rỗng. Nếu tác vụ đã hoàn tất hoặc thất bại, hệ thống không cho phép hủy mà chỉ cho phép xem hoặc xóa nhật ký.
Kết quả	Quản trị viên nắm được tình trạng xử lý tài liệu và có thể can thiệp kịp thời khi tác vụ lỗi hoặc chạy quá lâu.
Yêu cầu đặc biệt	Các cập nhật theo thời gian thực cần phản ánh đúng trạng thái tác vụ, tránh hiển thị sai tiến trình khi có nhiều tab hoặc nhiều tác vụ cùng chạy.

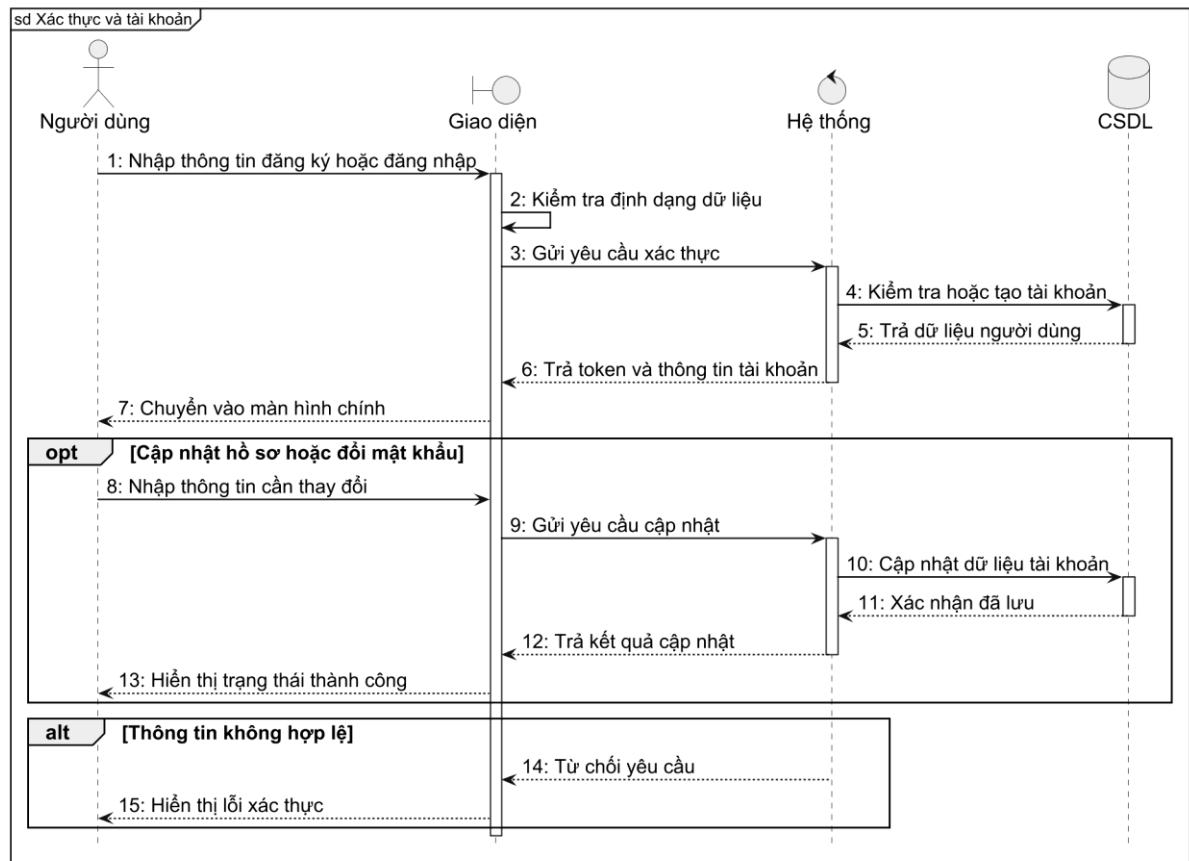
3.2.3. Biểu đồ tuần tự

Biểu đồ tuần tự được sử dụng để mô tả thứ tự trao đổi giữa tác nhân, giao diện, hệ thống và lớp dữ liệu trong các nghiệp vụ chính. Ở chương này, biểu đồ không trình bày chi tiết đến mức tên lớp hoặc tên tệp mã nguồn, mà tập trung vào trình tự nghiệp vụ: tác nhân thực hiện thao tác gì, giao diện chuyển yêu cầu như thế nào, hệ thống truy cập dữ liệu ở thời điểm nào và kết quả được trả lại theo trình tự ra sao.

Các luồng được lựa chọn bao phủ phạm vi sản phẩm hiện tại, gồm xác thực và tài khoản, quản lý hội thoại, hỏi đáp pháp luật, tư vấn có hướng dẫn, tra cứu thư viện pháp luật, tìm kiếm ngữ nghĩa, thống kê quản trị, cập nhật văn bản và quản lý văn bản pháp luật. Ở những luồng có đọc/ghi dữ liệu quan trọng, biểu đồ bổ sung thêm lifeline CSDL hoặc CSDL Vector để làm rõ quan hệ giữa thao tác nghiệp vụ và dữ liệu nền. Cách tách này giúp phân phân tích bám sát ca sử dụng đã trình bày, đồng thời vẫn đủ tổng quát để không phụ thuộc vào chi tiết triển khai cụ thể.

3.2.3.1. Biểu đồ tuần tự luồng xác thực và quản lý tài khoản

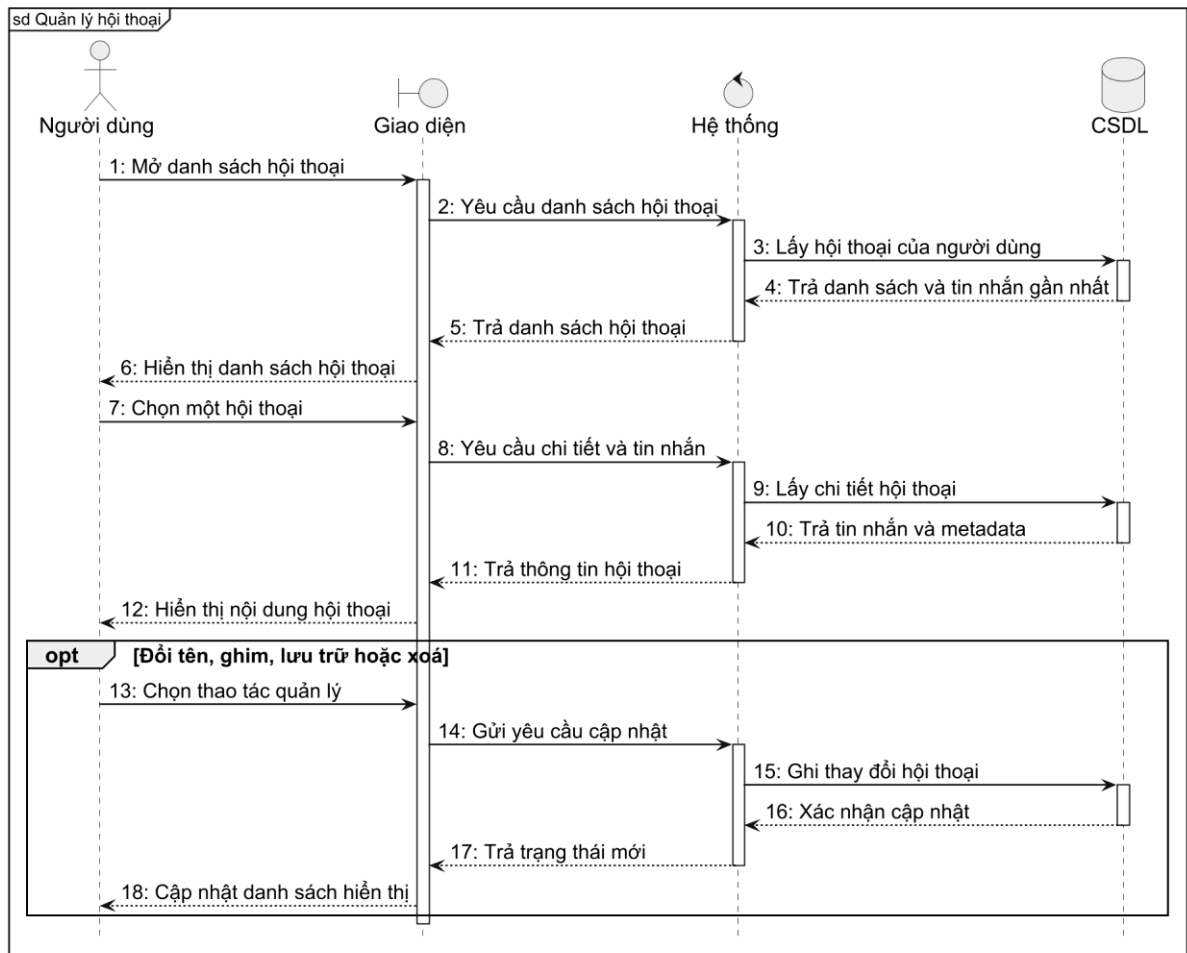
Luồng xác thực bắt đầu khi người dùng nhập thông tin đăng ký hoặc đăng nhập. Giao diện chịu trách nhiệm kiểm tra định dạng dữ liệu ban đầu trước khi gửi yêu cầu đến hệ thống. Hệ thống truy vấn CSDL để kiểm tra tài khoản hoặc tạo tài khoản mới, sau đó trả về thông tin phiên đăng nhập cho giao diện. Với thao tác cập nhật hồ sơ hoặc đổi mật khẩu, dữ liệu người dùng tiếp tục được ghi lại vào CSDL, bảo đảm thông tin hiển thị ở các lần truy cập sau luôn nhất quán.



Hình 3.10. Biểu đồ tuần tự luồng xác thực và quản lý tài khoản

3.2.3.2. Biểu đồ tuần tự luồng quản lý hội thoại

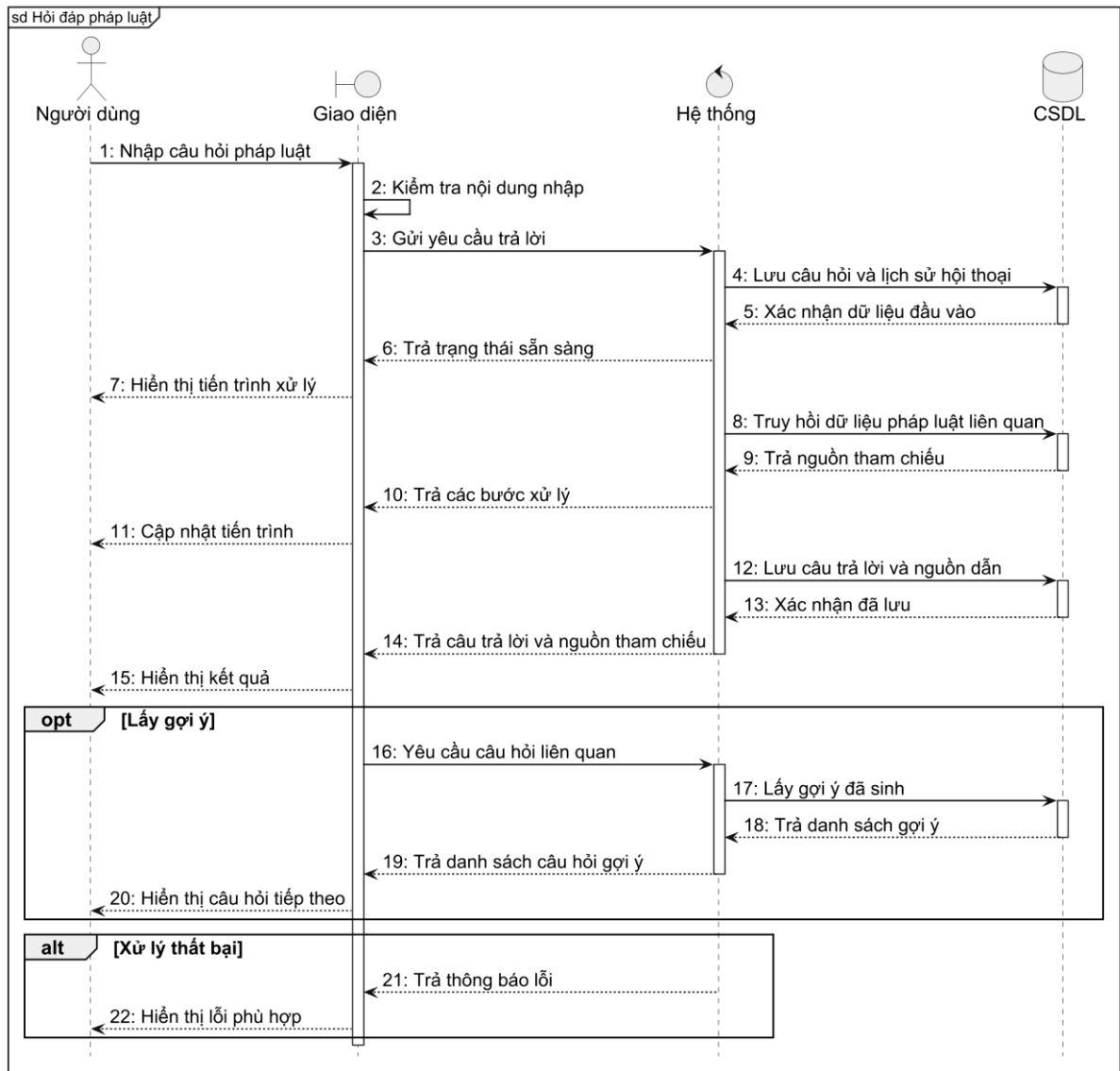
Luồng quản lý hội thoại thể hiện cách ứng dụng tải danh sách hội thoại, mở chi tiết một hội thoại và cập nhật trạng thái hội thoại. CSDL được bổ sung vào biểu đồ để làm rõ rằng danh sách hội thoại, tin nhắn gần nhất và các thông tin bổ sung như ghim/lưu trữ đều được đọc hoặc ghi thông qua hệ thống. Khi người dùng chọn đổi tên, ghim, lưu trữ hoặc xóa, hệ thống kiểm tra quyền sở hữu trước khi ghi thay đổi và trả lại trạng thái mới cho giao diện.



Hình 3.11. Biểu đồ tuần tự luồng quản lý hội thoại

3.2.3.3. Biểu đồ tuần tự luồng hỏi đáp pháp luật

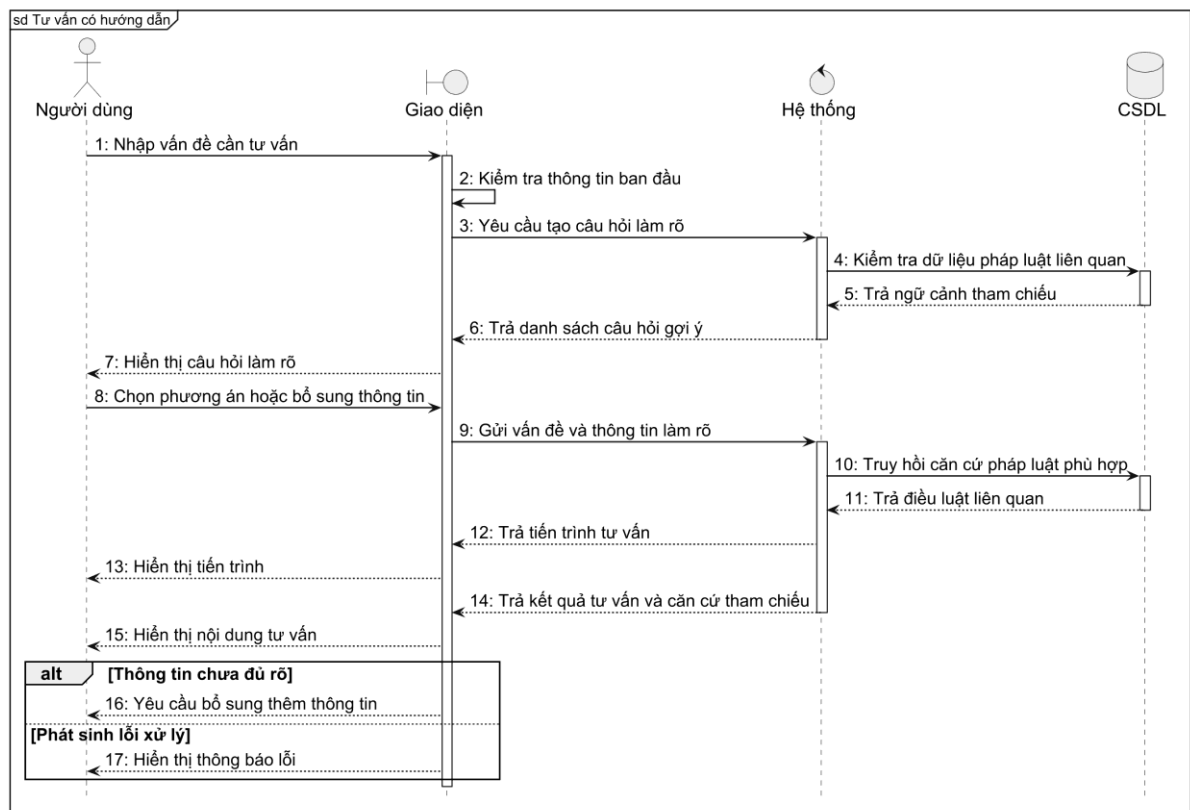
Luồng hỏi đáp là nghiệp vụ trọng tâm của ứng dụng. Sau khi người dùng gửi câu hỏi, hệ thống lưu câu hỏi và lịch sử hội thoại, truy hồi dữ liệu pháp luật liên quan, sau đó trả tiến trình xử lý cho giao diện. Khi câu trả lời hoàn tất, nội dung trả lời và nguồn tham chiếu được lưu lại để người dùng có thể mở lại hội thoại sau này. Việc biểu diễn CSDL trong luồng này giúp làm rõ rằng chức năng hỏi đáp không chỉ trả lời tức thời mà còn duy trì lịch sử, nguồn dẫn và câu hỏi gợi ý.



Hình 3.12. Biểu đồ tuần tự luồng hỏi đáp pháp luật

3.2.3.4. Biểu đồ tuần tự luồng tư vấn có hướng dẫn

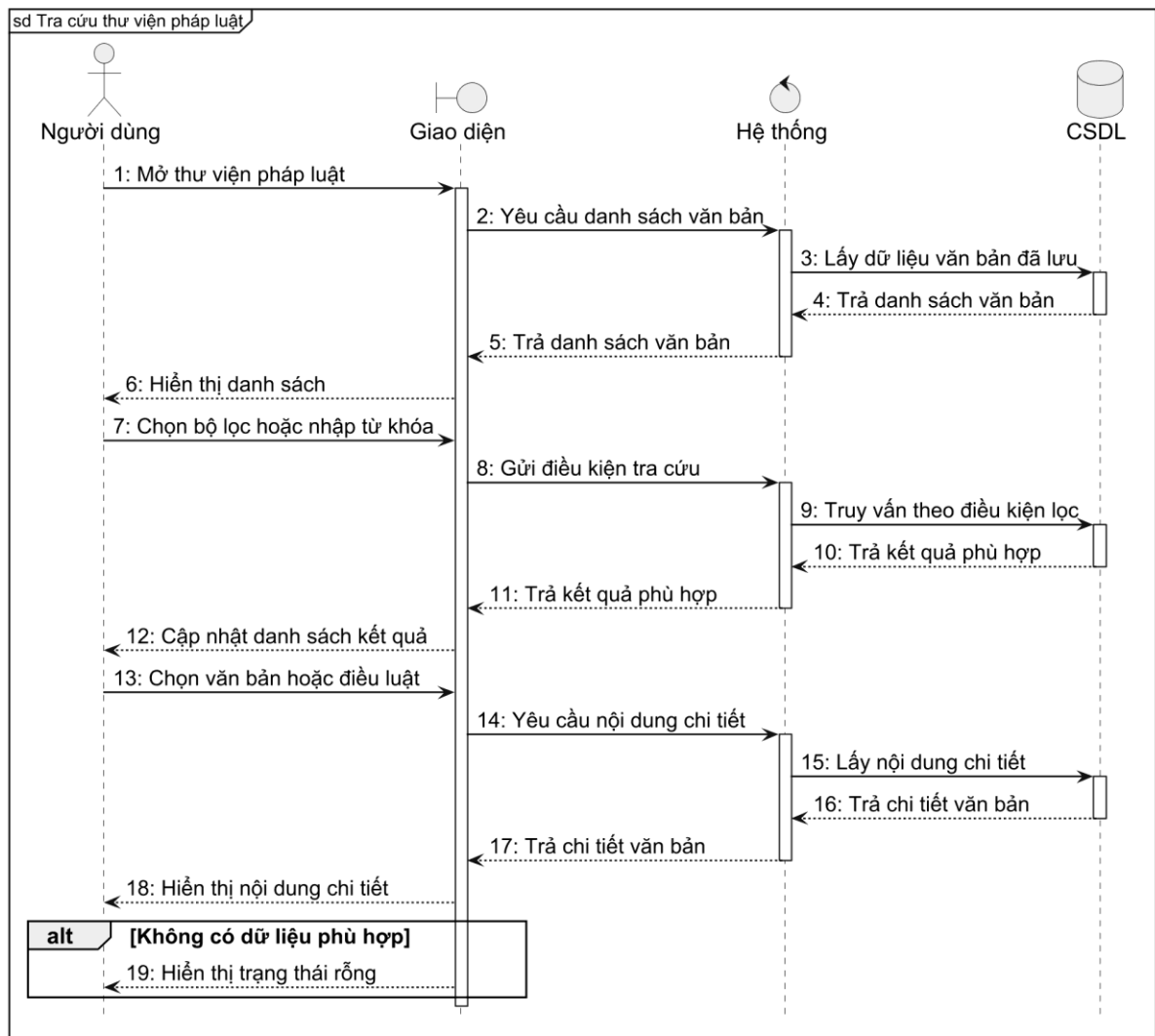
Luồng tư vấn có hướng dẫn được thiết kế cho các tình huống câu hỏi ban đầu còn thiếu ngữ cảnh. Hệ thống trước hết kiểm tra dữ liệu pháp luật liên quan để tạo câu hỏi làm rõ phù hợp. Sau khi người dùng bổ sung thông tin, hệ thống tiếp tục truy hỏi căn cứ pháp luật rồi mới tạo kết quả tư vấn cuối cùng. Cách xử lý này giúp câu trả lời cụ thể hơn, đặc biệt với những vấn đề pháp lý phụ thuộc nhiều vào hoàn cảnh thực tế.



Hình 3.13. Biểu đồ tuần tự luồng tư vấn có hướng dẫn

3.2.3.5. Biểu đồ tuần tự luồng tra cứu thư viện pháp luật

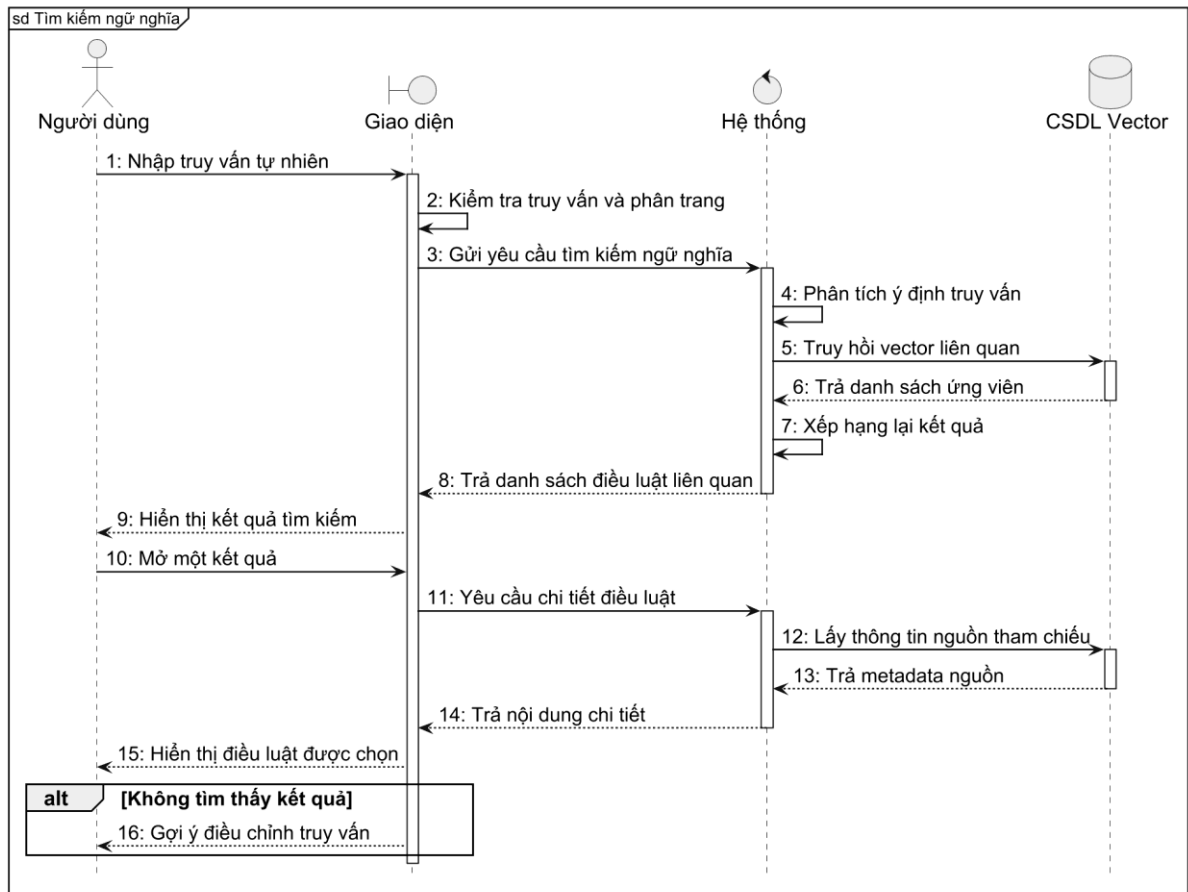
Luồng tra cứu thư viện pháp luật phục vụ nhu cầu xem văn bản và điều luật theo hướng có cấu trúc. Người dùng có thể mở danh sách văn bản, lọc theo năm/chủ đề/từ khoá, sau đó chọn một văn bản hoặc điều luật để xem chi tiết. CSDL được đặt thành một thành phần riêng trong biểu đồ để thể hiện rõ dữ liệu trả về là dữ liệu pháp luật đã được lưu trữ, khác với luồng hỏi đáp nơi hệ thống cần tổng hợp câu trả lời mới.



Hình 3.14. Biểu đồ tuần tự luồng tra cứu thư viện pháp luật

3.2.3.6. Biểu đồ tuần tự luồng tìm kiếm ngữ nghĩa

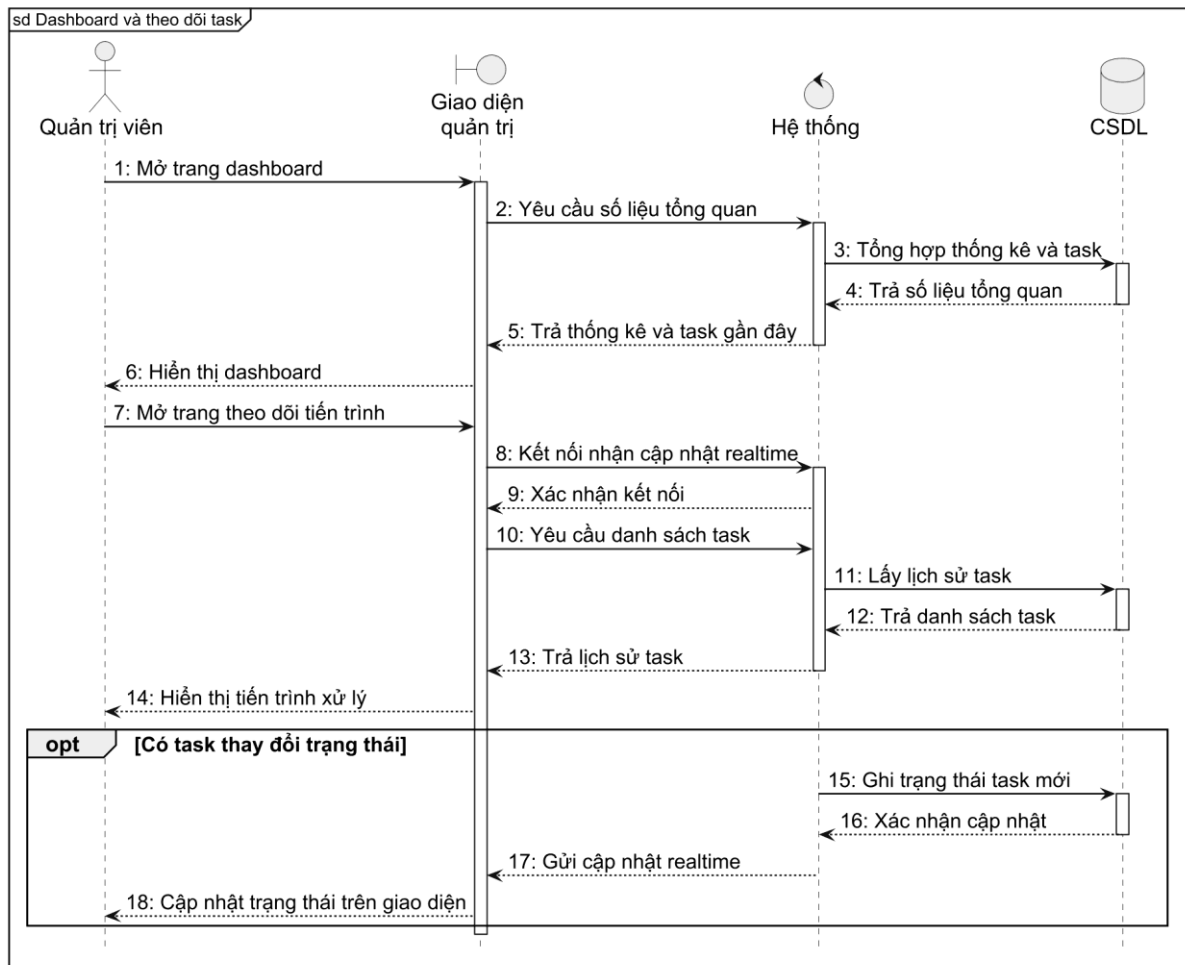
Luồng tìm kiếm ngữ nghĩa cho phép người dùng nhập truy vấn tự nhiên thay vì chỉ tìm theo từ khóa chính xác. Hệ thống phân tích truy vấn, truy hồi dữ liệu từ CSDL Vector, xếp hạng lại kết quả rồi trả danh sách điều luật liên quan cho giao diện. Việc tách CSDL Vector trong biểu đồ giúp nhấn mạnh khác biệt giữa tra cứu thông thường và tìm kiếm ngữ nghĩa: dữ liệu được tìm theo độ tương đồng nội dung thay vì chỉ theo bộ lọc có cấu trúc.



Hình 3.15. Biểu đồ tuần tự luồng tìm kiếm ngữ nghĩa

3.2.3.7. Biểu đồ tuần tự luồng thống kê và theo dõi tác vụ

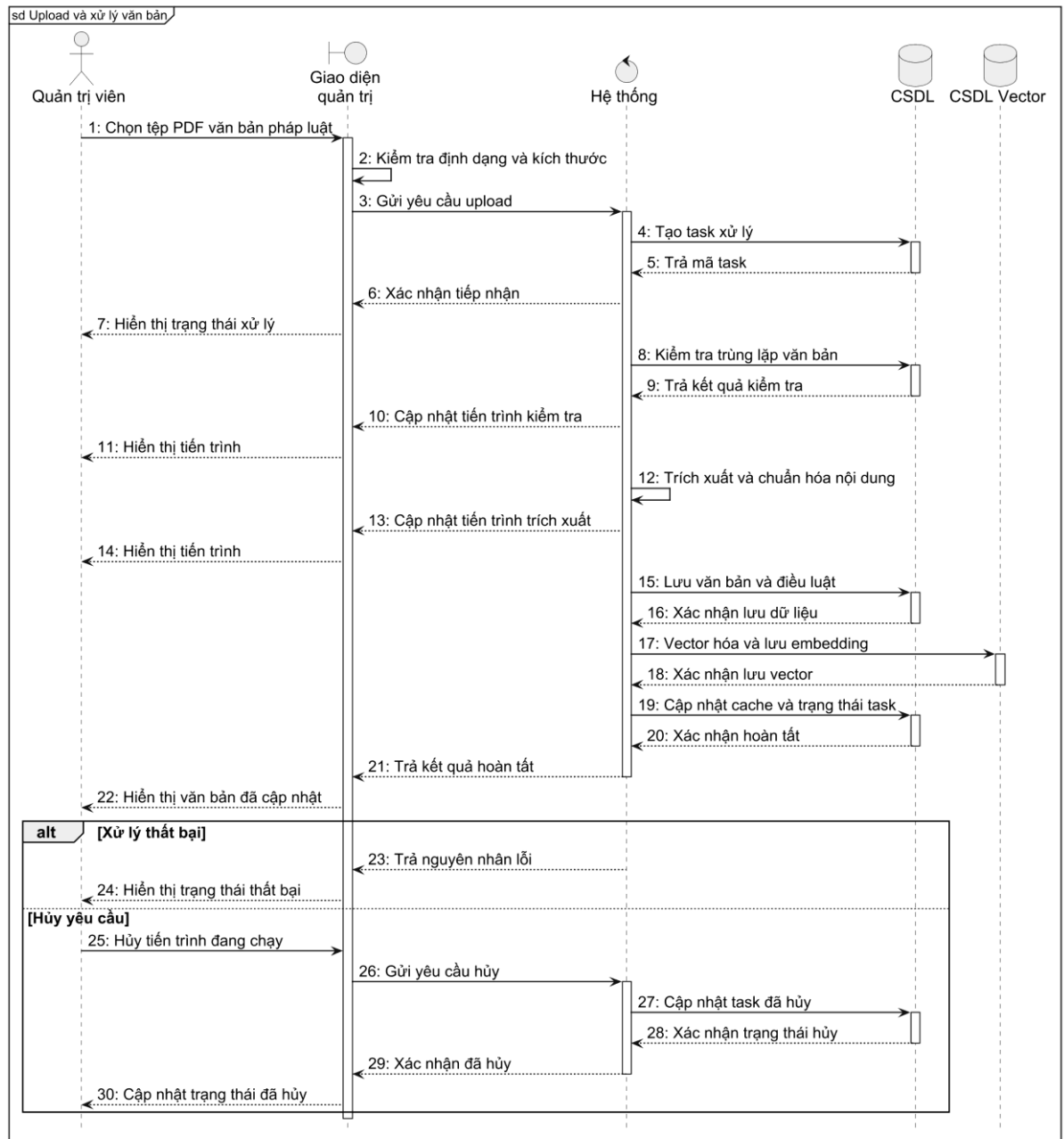
Luồng thống kê giúp quản trị viên nắm nhanh tình trạng dữ liệu và các tác vụ xử lý gần đây. Khi mở trang thống kê, hệ thống tổng hợp số liệu từ CSDL để trả về các chỉ số chính. Khi chuyển sang trang theo dõi tiến trình, giao diện thiết lập kênh nhận cập nhật theo thời gian thực và tải danh sách tác vụ hiện có. Mỗi khi tác vụ thay đổi trạng thái, hệ thống ghi trạng thái mới và gửi cập nhật để giao diện phản ánh ngay tiến trình xử lý.



Hình 3.16. Biểu đồ tuần tự luồng thống kê và theo dõi tác vụ

3.2.3.8. Biểu đồ tuần tự luồng tải lên và xử lý văn bản pháp luật

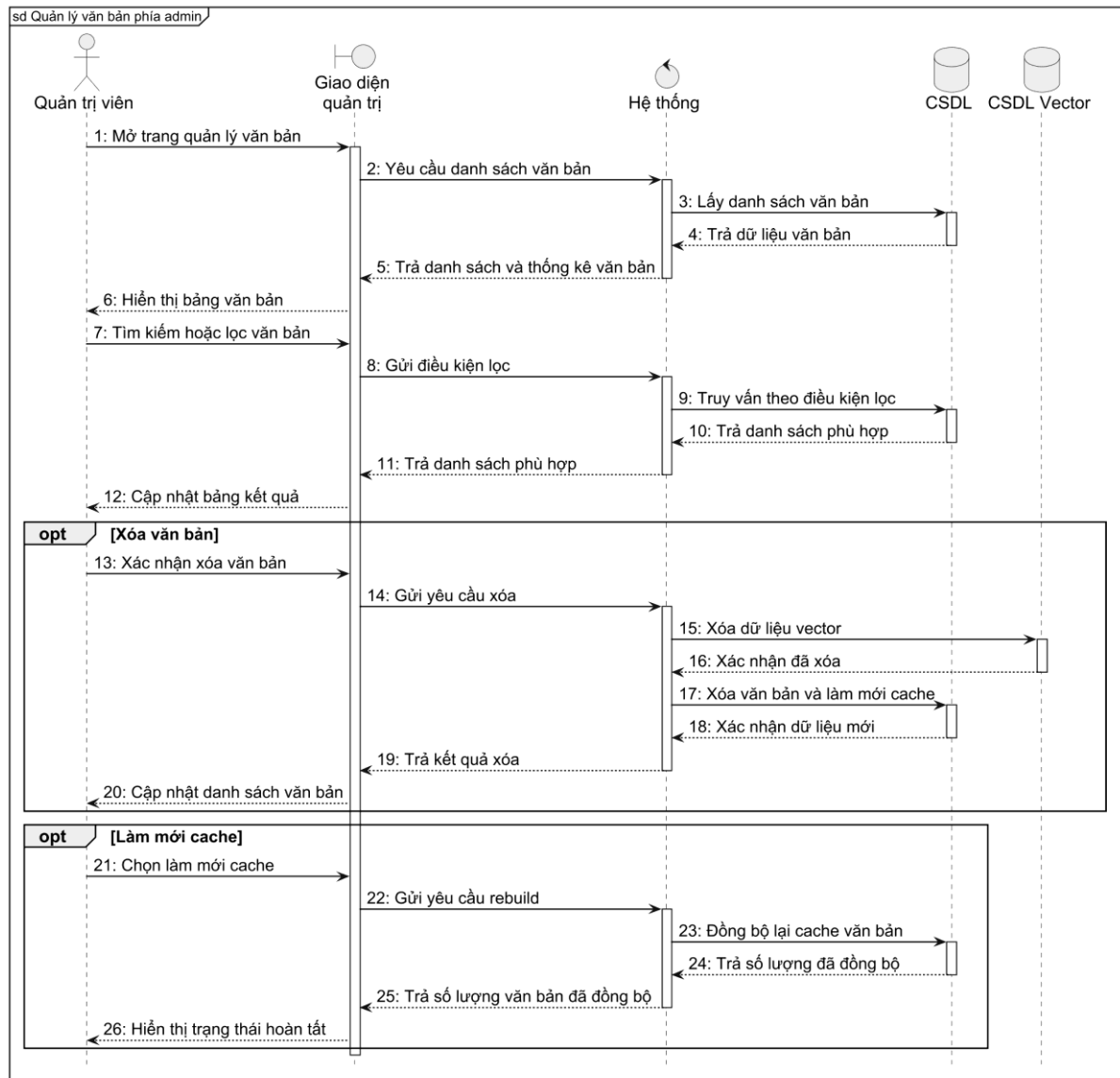
Luồng tải lên văn bản là nghiệp vụ quản trị quan trọng vì làm thay đổi kho dữ liệu phục vụ hỏi đáp và tra cứu. Giao diện kiểm tra định dạng tệp trước khi gửi lên hệ thống. Hệ thống tạo tác vụ xử lý trong CSDL, kiểm tra trùng lặp, trích xuất nội dung, lưu văn bản và điều luật, sau đó vector hoá dữ liệu vào CSDL Vector. Nếu tệp không hợp lệ, xử lý thất bại hoặc quản trị viên hủy yêu cầu, trạng thái tác vụ được cập nhật để quản trị viên có thể theo dõi và xử lý tiếp.



Hình 3.17. Biểu đồ tuần tự luồng tải lên và xử lý văn bản pháp luật

3.2.3.9. Biểu đồ tuần tự luồng quản lý văn bản phía quản trị

Sau khi văn bản đã được đưa vào hệ thống, quản trị viên cần có luồng quản lý riêng để tìm kiếm, lọc, xóa văn bản hoặc làm mới dữ liệu hiển thị. CSDL lưu nội dung văn bản và thông tin quản trị, trong khi CSDL Vector lưu dữ liệu phục vụ tìm kiếm ngữ nghĩa. Khi xóa văn bản, hệ thống phải xử lý đồng thời hai lớp dữ liệu này để tránh tình trạng giao diện còn hiển thị văn bản không còn khả dụng hoặc kết quả tìm kiếm vẫn trỏ đến dữ liệu đã bị xóa.

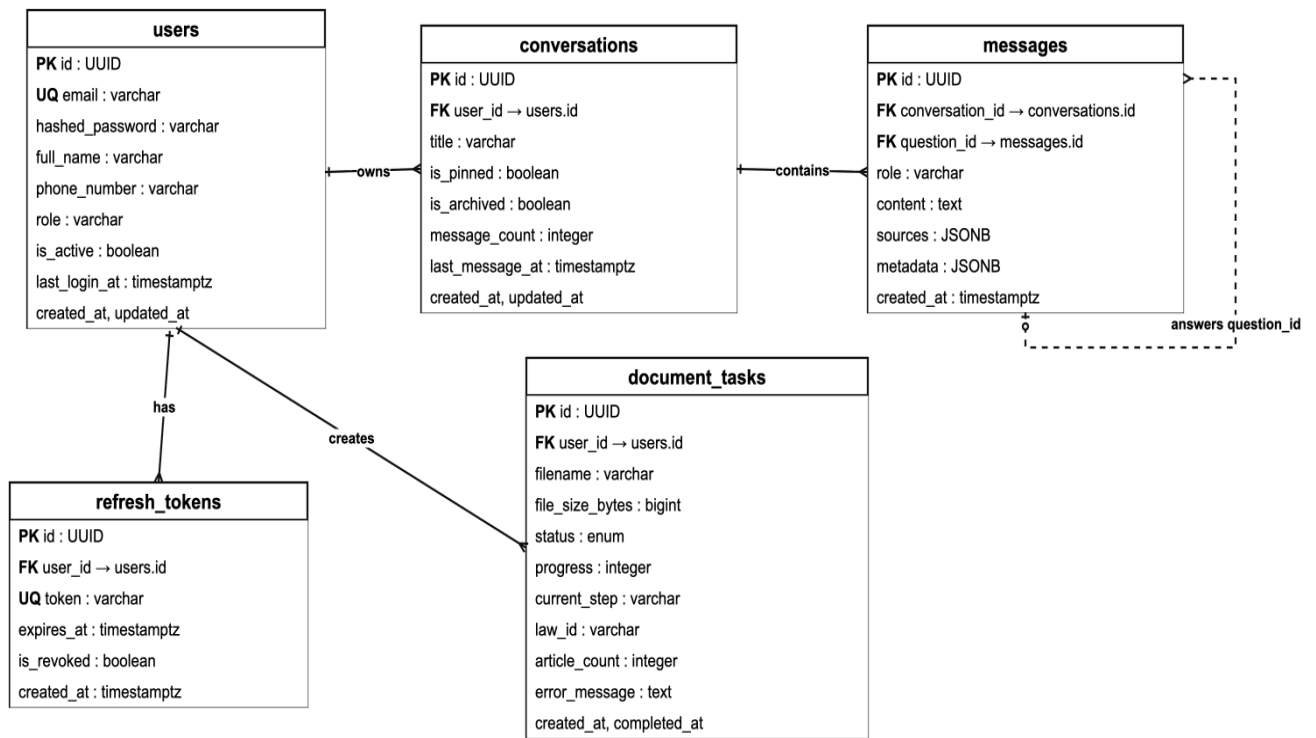


Hình 3.18. Biểu đồ tuần tự luồng quản lý văn bản phía quản trị

3.3. Thiết kế hệ thống

3.3.1. Thiết kế cơ sở dữ liệu

Dữ liệu của hệ thống được tổ chức theo đặc trưng sử dụng của từng nhóm nghiệp vụ. PostgreSQL lưu các dữ liệu có quan hệ chặt chẽ và cần đảm bảo tính toàn vẹn, chẳng hạn tài khoản, hội thoại, tin nhắn và tác vụ xử lý tài liệu. MongoDB lưu nội dung văn bản pháp luật ở dạng document linh hoạt, phù hợp với cấu trúc văn bản có nhiều thuộc tính mở rộng. ChromaDB lưu vector embedding của các đoạn văn bản để phục vụ truy hồi ngữ nghĩa trong các chức năng hỏi đáp và tìm kiếm.



Hình 3.19. Sơ đồ quan hệ dữ liệu trong PostgreSQL

Trong phạm vi PostgreSQL, các bảng được thiết kế xoay quanh ba nhóm dữ liệu chính. Nhóm thứ nhất là dữ liệu người dùng và phiên đăng nhập, phục vụ xác thực và phân quyền. Nhóm thứ hai là dữ liệu hội thoại, dùng để lưu lịch sử trao đổi giữa người dùng và hệ thống. Nhóm thứ ba là dữ liệu tác vụ xử lý tài liệu, dùng cho quá trình quản trị văn bản pháp luật. Cách tổ chức này giúp tách rõ dữ liệu phục vụ người dùng cuối với dữ liệu phục vụ vận hành hệ thống.

Bảng 3.13. Bảng users lưu thông tin người dùng

Trường	Kiểu	Mô tả
id	UUID	Khoá chính của người dùng.
email	String	Email đăng nhập, duy nhất trong hệ thống.
hashed_password	String	Mật khẩu đã được băm trước khi lưu.
full_name	String	Họ tên người dùng.
phone_number	String	Số điện thoại, có thể rỗng.
role	String	Phân biệt người dùng thường và quản trị viên.
is_active	Boolean	Trạng thái hoạt động của tài khoản.
last_login_at	DateTime	Thời điểm đăng nhập gần nhất.

Bảng 3.14. Bảng conversations lưu hội thoại

Trường	Kiểu	Mô tả
id	UUID	Khoá chính của hội thoại.
user_id	UUID	Khoá ngoại liên kết tới users.
title	String	Tiêu đề hội thoại, có thể sinh từ chủ đề câu hỏi đầu tiên.
is_pinned	Boolean	Đánh dấu hội thoại được ghim.
is_archived	Boolean	Đánh dấu hội thoại đã lưu trữ.
message_count	Integer	Số lượng tin nhắn trong hội thoại.
last_message_at	DateTime	Thời điểm có tin nhắn cuối.

Bảng 3.15. Bảng messages lưu tin nhắn

Trường	Kiểu	Mô tả
id	UUID	Khoá chính của tin nhắn.
conversation_id	UUID	Khoá ngoại liên kết tới conversations.
question_id	UUID	Tham chiếu tới tin nhắn câu hỏi khi cần ghép cặp hỏi - đáp.
role	String	Vai trò của tin nhắn: user hoặc assistant.
content	Text	Nội dung tin nhắn.
sources	JSONB	Danh sách nguồn pháp luật được dùng trong câu trả lời.
metadata	JSONB	Thông tin bổ sung như phân tích truy vấn, thời gian xử lý hoặc câu hỏi gợi ý.

Bảng 3.16. Bảng document_tasks lưu tiến trình xử lý tài liệu

Trường	Kiểu	Mô tả
id	UUID	Khoá chính của tác vụ xử lý tài liệu.
user_id	UUID	Quản trị viên tạo tác vụ.
filename	String	Tên tệp PDF gốc.
file_size_bytes	BigInteger	Kích thước tệp được tải lên.

status	Enum	Trạng thái pending, processing, completed, failed hoặc cancelled.
progress	Integer	Phần trăm xử lý.
current_step	String	Bước xử lý hiện tại để hiển thị trên Admin Web.
law_id	String	Mã văn bản pháp luật sau khi phân tách dữ liệu.
article_count	Integer	Số điều luật được trích xuất.
error_message	Text	Thông tin lỗi nếu tác vụ thất bại.

Bảng 3.17. Bảng refresh_tokens lưu phiên đăng nhập

Trường	Kiểu	Mô tả
id	UUID	Khoá chính của refresh token.
user_id	UUID	Khoá ngoại liên kết tới users.
token	String	Refresh token được lưu để kiểm tra phiên đăng nhập.
expires_at	DateTime	Thời điểm hết hạn.
is_revoked	Boolean	Đánh dấu token đã bị thu hồi.

Bảng 3.18. Collection VietnamLawDB.articles trong MongoDB

Trường	Mô tả
_id	Khoá duy nhất của điều luật, thường được ghép từ law_id và article_id.
law_id	Mã văn bản pháp luật.
article_id	Số hoặc mã điều luật trong văn bản.
title	Tiêu đề điều luật.
text	Nội dung nguyên văn của điều luật.
source_url	Đường dẫn tệp PDF gốc nếu có.
full_content_search	Nội dung ghép phục vụ tìm kiếm toàn văn.
metadata	Thông tin chủ đề, từ khoá, tóm tắt và năm ban hành.

Đối với dữ liệu văn bản pháp luật, MongoDB và ChromaDB được sử dụng bổ trợ cho nhau. MongoDB giữ nội dung có thể đọc và hiển thị lại cho người dùng,

trong khi ChromaDB giữ các đoạn văn bản đã được vector hoá để phục vụ truy hỏi theo ngữ nghĩa. Việc tách hai lớp lưu trữ này giúp hệ thống vừa đáp ứng nhu cầu tra cứu văn bản nguyên bản, vừa hỗ trợ các chức năng hỏi đáp dựa trên nội dung liên quan.

Bảng 3.19. Collection vietnamese_law trong ChromaDB

Trường	Mô tả
id	Mã đoạn văn bản theo dạng law_id_article_id_chunkN.
document	Nội dung đoạn văn bản gồm tiêu đề điều luật và phần văn bản.
embedding	Vector biểu diễn ngữ nghĩa của đoạn văn bản.
metadata.law_id	Mã văn bản pháp luật.
metadata.article_id	Mã điều luật.
metadata.chunk_index	Thứ tự đoạn trong điều luật.
metadata.total_chunks	Tổng số đoạn của điều luật.
metadata.year	Năm ban hành dùng cho lọc hoặc ưu tiên văn bản mới.
metadata.topics	Danh sách chủ đề được lưu dạng chuỗi JSON.
metadata.keywords	Danh sách từ khoá được lưu dạng chuỗi JSON.

3.3.2. Thiết kế các API

Các API của hệ thống được chia thành hai nhóm theo vai trò sử dụng. Main Service cung cấp API cho ứng dụng di động và Admin Web, đảm nhiệm xác thực, quản lý hội thoại, quản lý văn bản và điều phối các yêu cầu nghiệp vụ. RAG Service chủ yếu đóng vai trò dịch vụ nội bộ, nhận yêu cầu từ Main Service để thực hiện truy hỏi, sinh câu trả lời, tư vấn có hướng dẫn và cập nhật dữ liệu vector vào ChromaDB.

Cách phân chia API như trên giúp giao diện người dùng không phải làm việc trực tiếp với dịch vụ xử lý AI hoặc CSDL Vector. Các yêu cầu từ phía giao diện được tập trung qua Main Service, sau đó Main Service mới điều phối sang RAG Service khi nghiệp vụ cần truy hỏi hoặc sinh câu trả lời. Thiết kế này giúp kiểm soát xác thực, phân quyền và ghi nhận lịch sử xử lý ở một điểm thống nhất.

Bảng 3.20. Nhóm API xác thực

API	Phương thức	Chức năng
/api/v1/auth/signup	POST	Đăng ký tài khoản mới.
/api/v1/auth/login	POST	Đăng nhập và nhận token.

/api/v1/auth/refresh	POST	Làm mới access token.
/api/v1/auth/logout	POST	Đăng xuất một phiên.
/api/v1/auth/me	GET/PUT	Xem hoặc cập nhật thông tin người dùng.
/api/v1/auth/change-password	PUT	Đổi mật khẩu.

Bảng 3.21. Nhóm API hội thoại và hỏi đáp

API	Phương thức	Chức năng
/api/v1/chat/conversations	GET	Lấy danh sách hội thoại.
/api/v1/chat/conversations/{id}	GET/PUT/DELETE	Xem, cập nhật hoặc xóa hội thoại.
/api/v1/chat/conversations/{id}/messages	GET	Lấy tin nhắn của hội thoại.
/api/v1/chat/messages	POST	Gửi tin nhắn và nhận câu trả lời dạng thường.
/api/v1/chat/messages/stream	POST	Gửi tin nhắn và nhận tiến trình/câu trả lời qua SSE.
/api/v1/chat/messages/{message_id}/suggested-questions	GET	Lấy câu hỏi gợi ý cho tin nhắn assistant.

Bảng 3.22. Nhóm API tư vấn có hướng dẫn

API	Phương thức	Chức năng
/api/v1/guided/clarify	POST	Sinh câu hỏi làm rõ từ câu hỏi ban đầu.
/api/v1/guided/answer	POST	Tạo câu trả lời cuối cùng sau khi người dùng chọn đáp án.
/api/v1/guided/answer/stream	POST	Tạo câu trả lời cuối cùng và trả tiến trình qua SSE.

Bảng 3.23. Nhóm API tra cứu pháp luật

API	Phương thức	Chức năng
/api/v1/laws	GET	Lấy danh sách văn bản pháp luật.

/api/v1/laws/search	GET/POST	Tìm kiếm văn bản theo nội dung, mã văn bản, điều, năm, chủ đề hoặc từ khoá.
/api/v1/laws/ai-search	POST	Tìm kiếm ngữ nghĩa bằng RAG Service.
/api/v1/laws/topics	GET	Lấy danh sách chủ đề.
/api/v1/laws/years	GET	Lấy danh sách năm ban hành.
/api/v1/laws/detail	GET	Lấy chi tiết một điều luật.

Bảng 3.24. Nhóm API quản trị tài liệu

API	Phương thức	Chức năng
/api/v1/documents/ws	WebSocket	Theo dõi tiến trình xử lý tài liệu theo thời gian thực.
/api/v1/documents/upload-v2	POST	Tải lên và xử lý PDF văn bản pháp luật.
/api/v1/documents/tasks	GET	Lấy lịch sử tác vụ xử lý tài liệu.
/api/v1/documents/tasks/{task_id}/cancel	POST	Hủy tác vụ đang chạy hoặc đang treo.
/api/v1/documents/tasks/{task_id}	DELETE	Xoá nhật ký tác vụ.
/api/v1/laws/{law_id}	DELETE	Xoá văn bản khỏi MongoDB, ChromaDB và dữ liệu hiển thị.

Bảng 3.25. Nhóm API nội bộ của RAG Service

API	Phương thức	Chức năng
/api/v1/rag/agent-search/stream	POST	Chạy luồng Agentic RAG và trả tiến trình qua SSE.
/api/v1/rag/guided-clarify	POST	Phân tích câu hỏi và sinh câu hỏi làm rõ.
/api/v1/rag/guided-answer/stream	POST	Chạy luồng tư vấn có hướng dẫn và trả tiến trình qua SSE.
/api/v1/rag/semantic-search	POST	Tìm kiếm ngữ nghĩa trong ChromaDB.

/api/v1/ingest/articles	POST	Chia đoạn, tạo embedding và lưu điều luật vào ChromaDB.
/api/v1/ingest/articles/{law_id}	DELETE	Xoá các đoạn vector của một văn bản khi hoàn tác hoặc xoá văn bản.

3.4. Tổng kết chương 3

Chương 3 đã trình bày quá trình phân tích và thiết kế hệ thống Vietnam Law Chatbot theo các nhóm nội dung chính: yêu cầu hệ thống, ca sử dụng, luồng tuần tự, cơ sở dữ liệu và API. Thông qua phân phân tích, hệ thống được xác định có hai nhóm tác nhân chính là người dùng cuối và quản trị viên. Các chức năng trọng tâm tập trung vào hỏi đáp pháp luật, tư vấn có hướng dẫn, tra cứu thư viện văn bản, tìm kiếm ngữ nghĩa và cập nhật dữ liệu pháp luật.

Kết quả thiết kế cho thấy hệ thống được tổ chức theo mô hình nhiều thành phần, có sự phân tách rõ giữa xử lý nghiệp vụ, lưu trữ dữ liệu và xử lý tri thức pháp luật. PostgreSQL đảm nhiệm dữ liệu tài khoản, hội thoại và tác vụ xử lý; MongoDB lưu nội dung văn bản pháp luật ở dạng linh hoạt; ChromaDB phục vụ truy hồi vector cho các chức năng dựa trên ngữ nghĩa. Các API cùng cơ chế cập nhật tiến trình theo thời gian thực là cơ sở để triển khai giao diện, kiểm thử chức năng và đánh giá hệ thống trong chương tiếp theo.

CHƯƠNG 4: TRIỂN KHAI VÀ THỰC NGHIỆM

Chương này trình bày kết quả triển khai thực tế của hệ thống Vietnam Law Chatbot, bao gồm môi trường vận hành, giao diện người dùng trên hai nền tảng Mobile và Web, cùng với quá trình thực nghiệm và đánh giá định lượng hiệu quả của giải pháp Agentic RAG đề xuất. Trên cơ sở đó, chương đưa ra các nhận xét về ưu điểm, hạn chế và hướng phát triển tiếp theo của hệ thống.

4.1. Môi trường triển khai

4.1.1. Môi trường phát triển

Hệ thống được phát triển và kiểm thử trên môi trường sau:

Bảng 4.1. Cấu hình môi trường phát triển

Thành phần	Phiên bản / Cấu hình
Hệ điều hành	Windows 11 Pro (Build 22631, 64-bit)
CPU	Intel Core i5-12500H (12 nhân / 16 luồng, 2.5 GHz)
RAM	16 GB DDR4 3200 MHz (2x8 GB)
Python	3.12.2
JDK	21.0.8 LTS
Node.js	v24.13.0
IDE	Android Studio Meerkat / VS Code
Docker Desktop	29.1.3

4.1.2. Kiến trúc triển khai

Hệ thống gồm bốn thành phần chính chạy song song: Main Service (port 8000) đảm nhận xử lý API công khai và xác thực người dùng; RAG Service (port 8001) phụ trách luồng Agentic RAG, embedding và nạp dữ liệu; ChromaDB được ánh xạ ra host qua port 4000 và lắng nghe port 8000 trong Docker network; PostgreSQL (port 5432) quản lý dữ liệu người dùng và phiên làm việc. Ngoài ra, MongoDB chạy ở port 27017 để lưu trữ 528.620 điều khoản pháp luật.

Trong giai đoạn phát triển, ba cơ sở dữ liệu (PostgreSQL, ChromaDB, MongoDB) được container hóa bằng Docker Compose, trong khi hai dịch vụ

backend chạy trực tiếp trên máy host để thuận tiện cho việc gỡ lỗi và tải lại khi thay đổi mã nguồn. Cấu hình Docker Compose cho lớp dữ liệu như sau:

Bảng 4.2. Cấu hình Docker Compose lớp dữ liệu

Container	Image	Port	Volume
postgres	postgres:16-alpine	5432:5432	postgres_data
chromadb	chromadb/chroma:latest	4000:8000	chroma_data
mongodb	mongo:7	27017:27017	mongo_data

Các dịch vụ backend giao tiếp với nhau qua hai lớp xác thực: JWT Bearer token cho luồng Client → Main Service, và X-API-Key nội bộ cho luồng Main Service → RAG Service. Cách tổ chức này bảo đảm RAG Service không tiếp xúc trực tiếp với Internet.

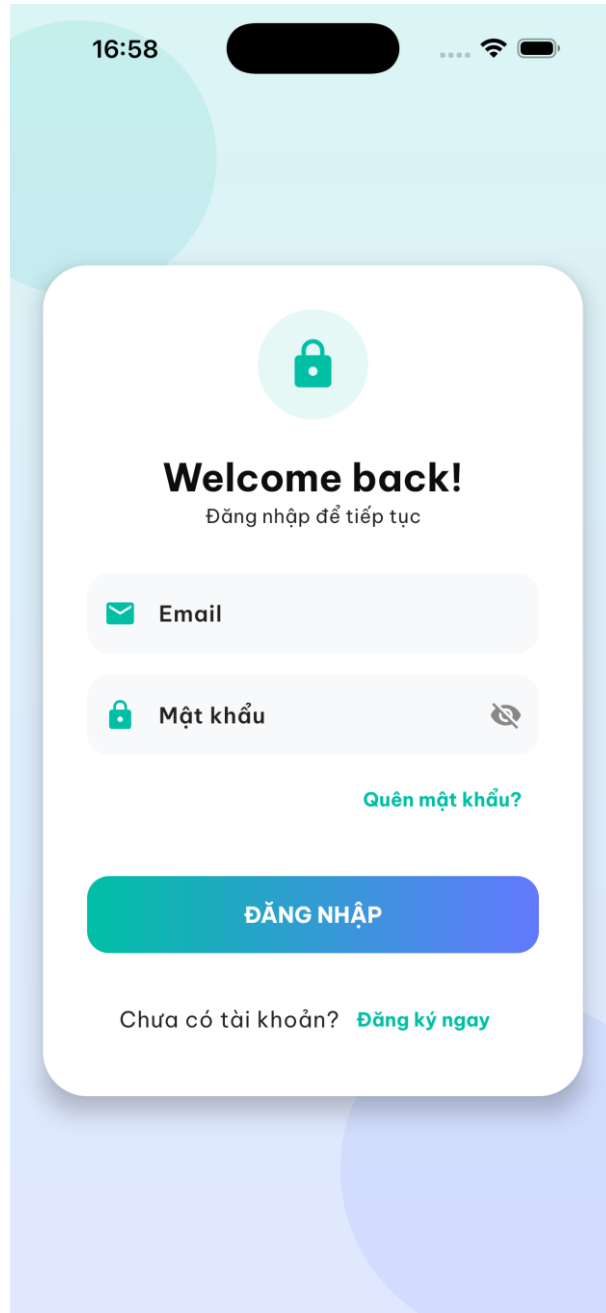
4.1.3. Biến môi trường và bảo mật

Mỗi dịch vụ được cấu hình qua tệp .env riêng biệt và không đưa vào hệ thống quản lý mã nguồn. Các biến môi trường quan trọng bao gồm khóa API của mô hình ngôn ngữ lớn, TAVILY_API_KEY, JWT_SECRET_KEY, X_API_KEY_SECRET và chuỗi kết nối cho từng cơ sở dữ liệu. Việc tách cấu hình khỏi mã nguồn giúp hạn chế rò rỉ thông tin nhạy cảm và thuận tiện hơn khi triển khai ở các môi trường khác nhau.

4.2. Giao diện ứng dụng Mobile

Ứng dụng Mobile được phát triển theo kiến trúc Kotlin Multiplatform (KMP) với Compose Multiplatform, cho phép triển khai đồng thời trên Android và iOS từ một mã nguồn chung. Giao diện tuân theo Material Design 3, được tối ưu cho màn hình điện thoại với kích thước từ 5 đến 6,7 inch. Các màn hình chính gồm có Đăng nhập, Thư viện pháp luật, Chi tiết văn bản, Chat Agentic RAG, AI-Powered Search, Guided Consultation và Quản lý hội thoại.

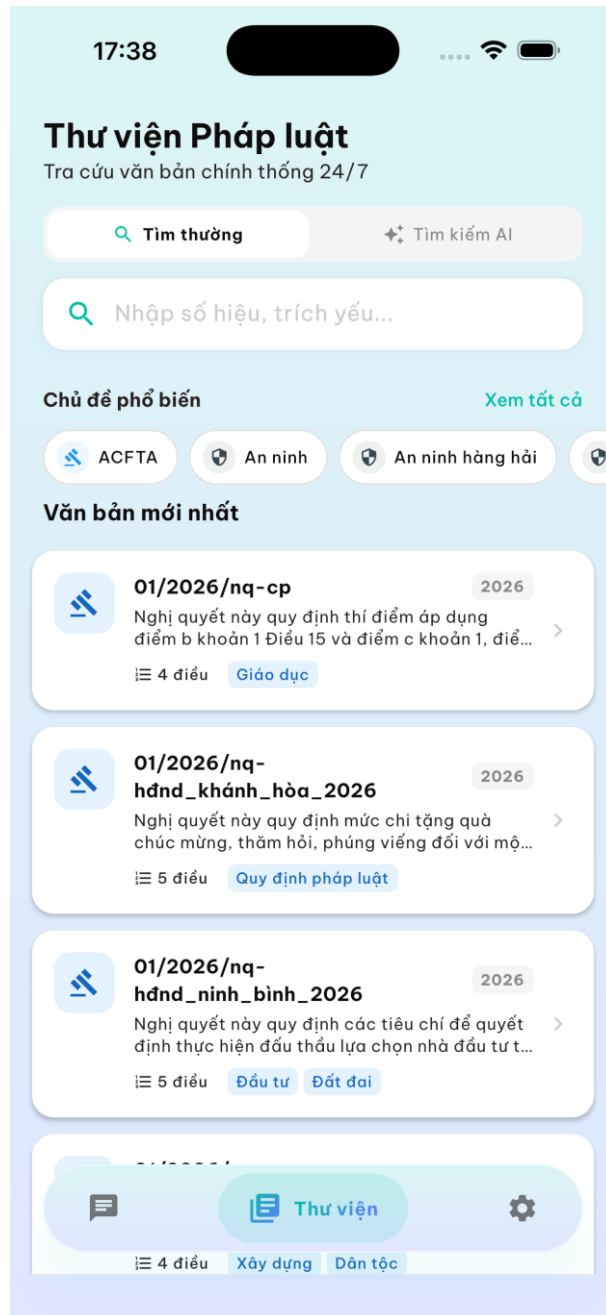
4.2.1. Đăng nhập và xác thực



Hình 4.1. Màn hình đăng nhập ứng dụng Mobile

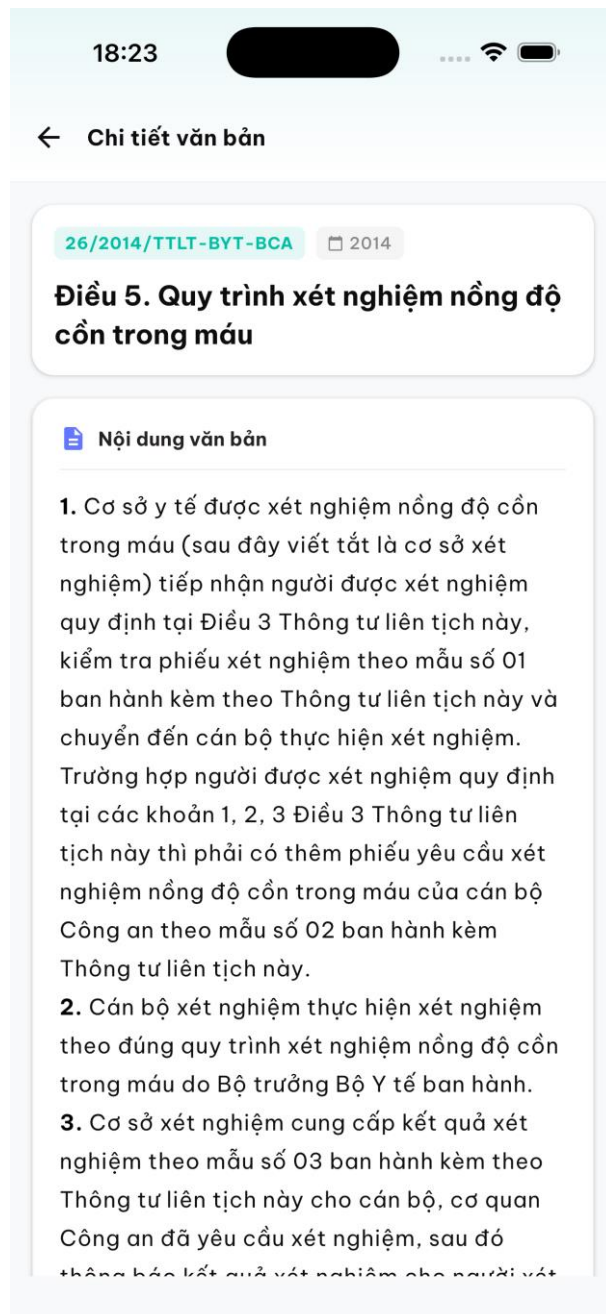
Màn hình đăng nhập cung cấp biểu mẫu xác thực bằng email và mật khẩu, đồng thời kiểm tra hợp lệ theo thời gian thực. Sau khi xác thực thành công, JWT access token và refresh token được lưu trữ mã hóa trong KSafe, sử dụng Android Keystore trên Android và iOS Keychain trên iOS. Ktor HTTP client được cấu hình cơ chế tự động làm mới token hết hạn để người dùng không phải đăng nhập lại trong quá trình sử dụng.

4.2.2. Thư viện văn bản pháp luật



Hình 4.2. Màn hình thư viện văn bản pháp luật

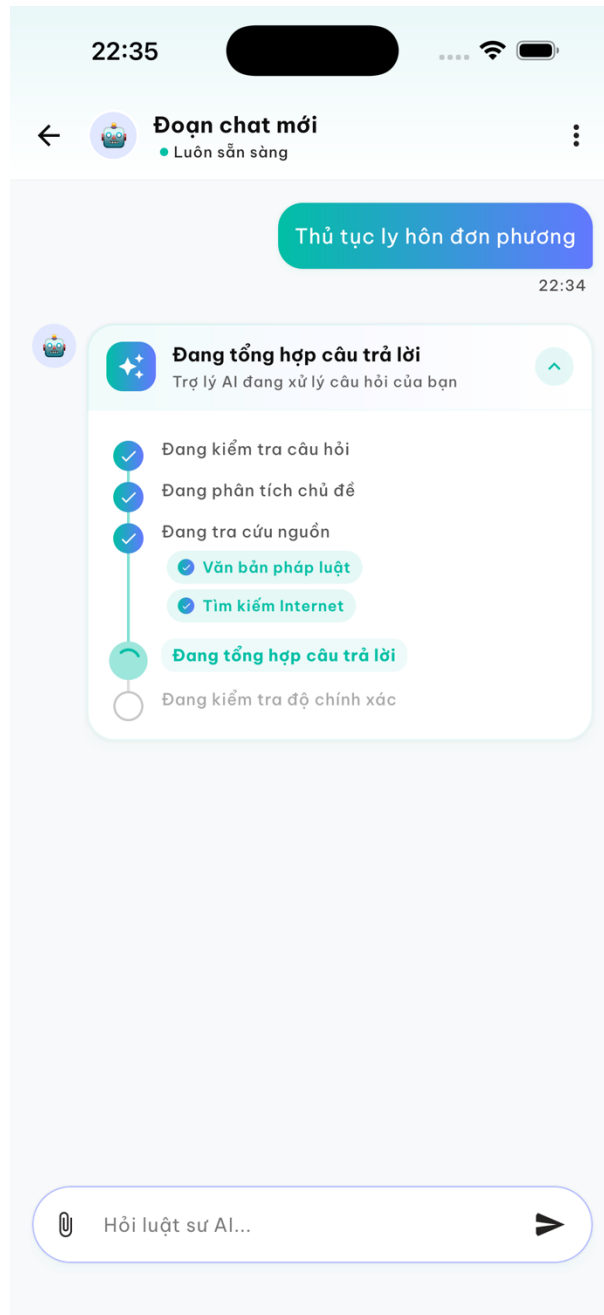
Màn hình Thư viện cho phép người dùng duyệt và tìm kiếm trong kho 46.047 văn bản pháp luật, với các bộ lọc theo chủ đề, năm ban hành và cơ quan ban hành. Danh sách điều khoản được tải theo phân trang để tối ưu hiệu năng, kết hợp cơ chế lazy loading có sẵn trong Compose Multiplatform.



Hình 4.3. Màn hình chi tiết văn bản pháp luật

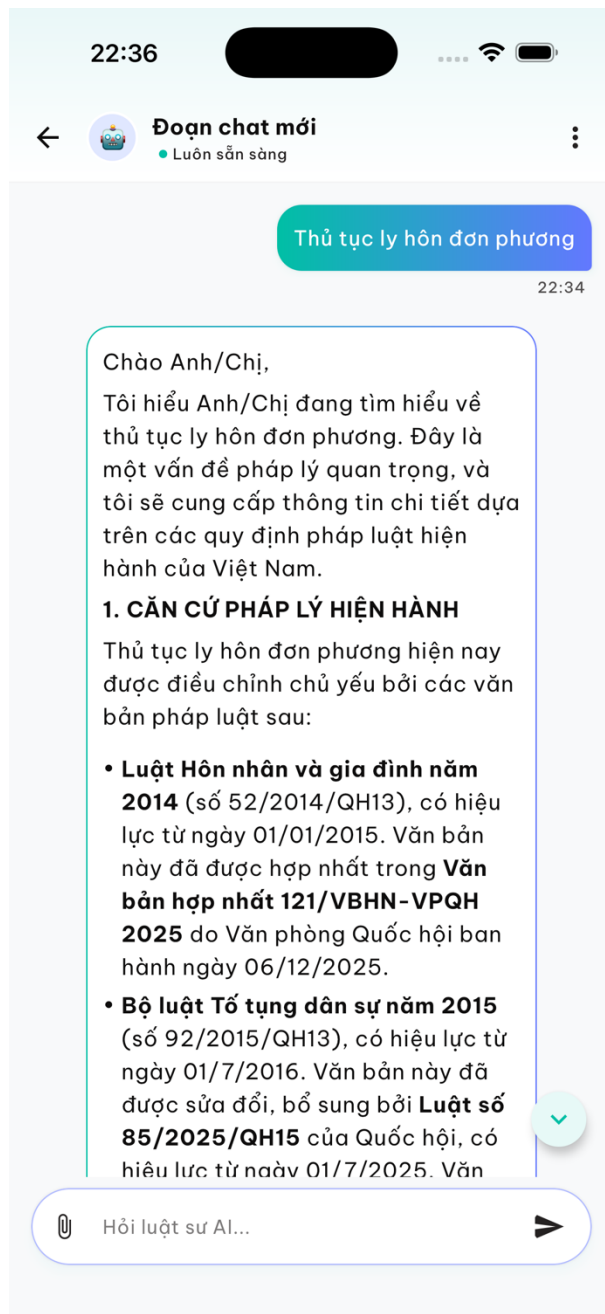
Khi chọn một văn bản, người dùng chuyển sang màn hình Chi tiết để xem toàn bộ nội dung điều khoản theo cấu trúc phân cấp (chương → điều → khoản → điểm). Nội dung được hiển thị ở định dạng rich text, hỗ trợ cuộn dài và tìm kiếm trong văn bản, giúp người dùng tra cứu trực tiếp nội dung pháp luật gốc mà không cần rời ứng dụng.

4.2.3. Chat Agentic RAG và ThinkingPanel



Hình 4.4. Màn hình Chat với ThinkingPanel đang hiển thị các bước xử lý

Đây là tính năng trọng tâm của ứng dụng. Giao diện Chat sử dụng SSE (Server-Sent Events) để hiển thị tiến trình xử lý theo thời gian thực thay vì để người dùng chờ phản hồi tĩnh. Ứng dụng nhận các sự kiện ready, progress, done và error: ready xác nhận tin nhắn người dùng đã được lưu, progress cập nhật ThinkingPanel, còn done trả về câu trả lời hoàn chỉnh cùng nguồn tham chiếu sau khi backend đã lưu tin nhắn trợ lý. Điểm nổi bật là ThinkingPanel, thành phần hiển thị tiến trình xử lý của luồng AI theo 5 bước tuần tự: Guardrail, Query Analysis, Agent, Tool Calls và Verifier.



Hình 4.5. Câu trả lời Agentic RAG



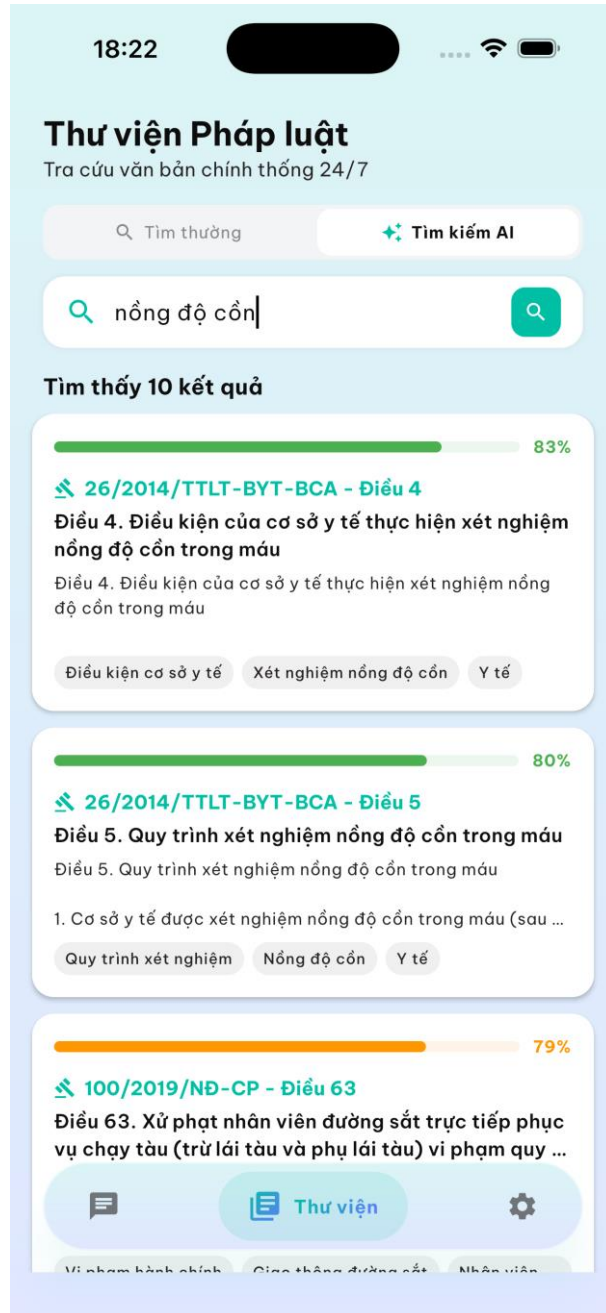
Hình 4.6. Câu trả lời Agentic RAG - gợi ý câu hỏi tiếp theo



Hình 4.7. Câu trả lời Agentic RAG - danh sách nguồn tham chiếu

Sau khi quy trình xử lý hoàn tất, câu trả lời được hiển thị dưới dạng Markdown với danh sách nguồn tham chiếu rõ ràng, bao gồm tên văn bản pháp luật, số điều khoản và năm ban hành.

4.2.4. AI-Powered Search (Tìm kiếm theo ngữ nghĩa)



Hình 4.8. Giao diện tìm kiếm văn bản pháp luật theo ngữ nghĩa

Tính năng AI-Powered Search cho phép người dùng tìm kiếm văn bản pháp luật bằng ngôn ngữ tự nhiên thay vì chỉ dựa vào từ khóa chính xác. Truy vấn của người dùng được embedding bằng mô hình `bkai-foundation-models/vietnamese-bi-encoder` (768 chiều), sau đó ChromaDB thực hiện truy vấn cosine similarity để tìm các điều khoản có nội dung gần nhất về mặt ngữ nghĩa. Kết quả trả về được sắp xếp theo độ liên quan, kèm tên văn bản, số điều khoản và đoạn trích ngữ cảnh, giúp người dùng định vị nhanh văn bản pháp lý cần tham chiếu mà không cần biết chính xác tên văn bản hay số điều.

4.2.5. Guided Consultation (Tư vấn có hướng dẫn)

18:15

← **Tư vấn có hướng dẫn**

Vượt đèn đỏ bị phạt bao nhiêu tiền?

Tiếp tục

Chưa biết hỏi gì?
Chọn chủ đề để xem câu hỏi mẫu

Giao thông Đất đai - Nhà ở Lao động
Doanh nghiệp Hình sự
Hôn nhân - Gia đình Y tế - Bảo hiểm
Giáo dục Tiêu dùng Công nghệ - Mạng
Hành chính Môi trường

Giao thông

- Vượt đèn đỏ bị phạt bao nhiêu tiền? →
- Nồng độ cồn bao nhiêu thì bị tước bằng lái? →
- Không đội mũ bảo hiểm bị phạt như thế nào? →
- Đi ngược chiều bị xử phạt ra sao? →

Hình 4.9. Guided Consultation - câu hỏi gốc và gợi ý câu hỏi làm rõ ngữ cảnh

Tính năng Guided Consultation được thiết kế cho các câu hỏi pháp lý thiếu ngữ cảnh. Ở bước đầu tiên, người dùng nhập câu hỏi và hệ thống hiển thị đồng thời các gợi ý câu hỏi bổ trợ (multiple-choice) nhằm làm rõ hoàn cảnh cụ thể - ví dụ: loại hợp đồng lao động, lĩnh vực doanh nghiệp, vùng địa lý.

18:17

← **Tư vấn có hướng dẫn** ↻

Lĩnh vực: Vi phạm vượt đèn đỏ – xử phạt hành chính theo Nghị định 168/2024

1 Anh/Chị điều khiển loại phương tiện nào khi vi phạm?

☐ Ô tô

☐ Xe máy / mô tô

☐ Xe máy điện / xe đạp điện

☒ **Xe đạp / phương tiện thô sơ**

☐ Không rõ / Không áp dụng

2 Hành vi vượt đèn đỏ có gây ra hậu quả gì không?

☐ Không gây tai nạn, chỉ vi phạm thuần túy

☐ Gây va chạm nhẹ, không có thương tích

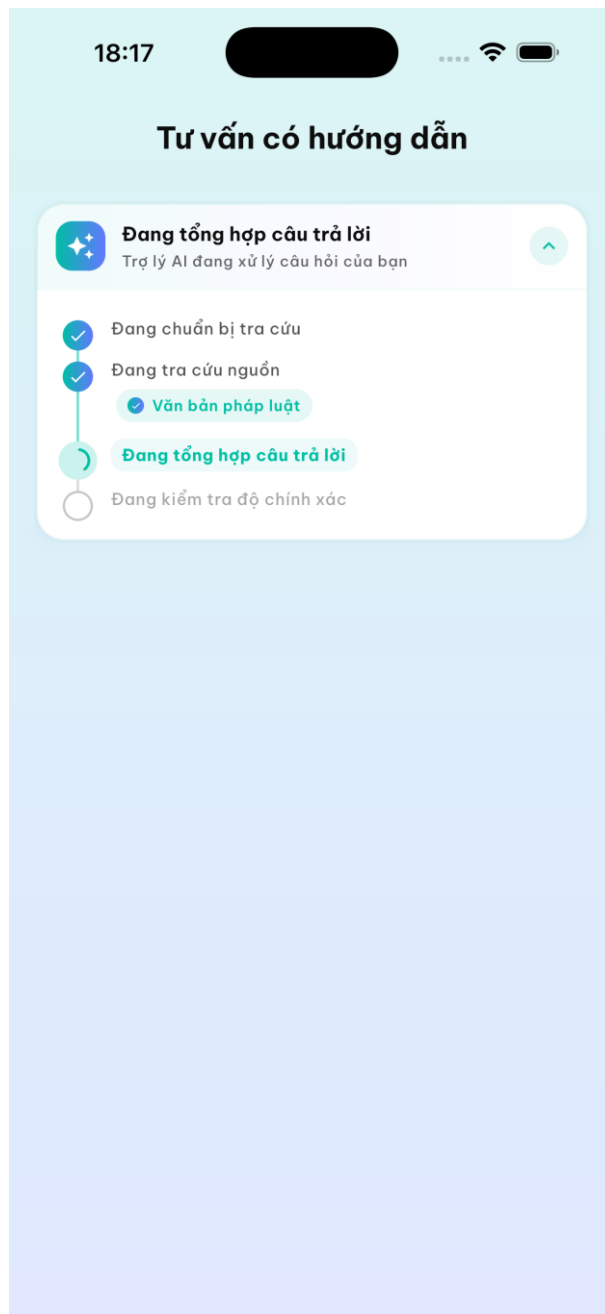
☒ **Gây tai nạn có thương tích / thiệt hại tài sản**

☐ Không rõ / Không áp dụng

Xem câu trả lời

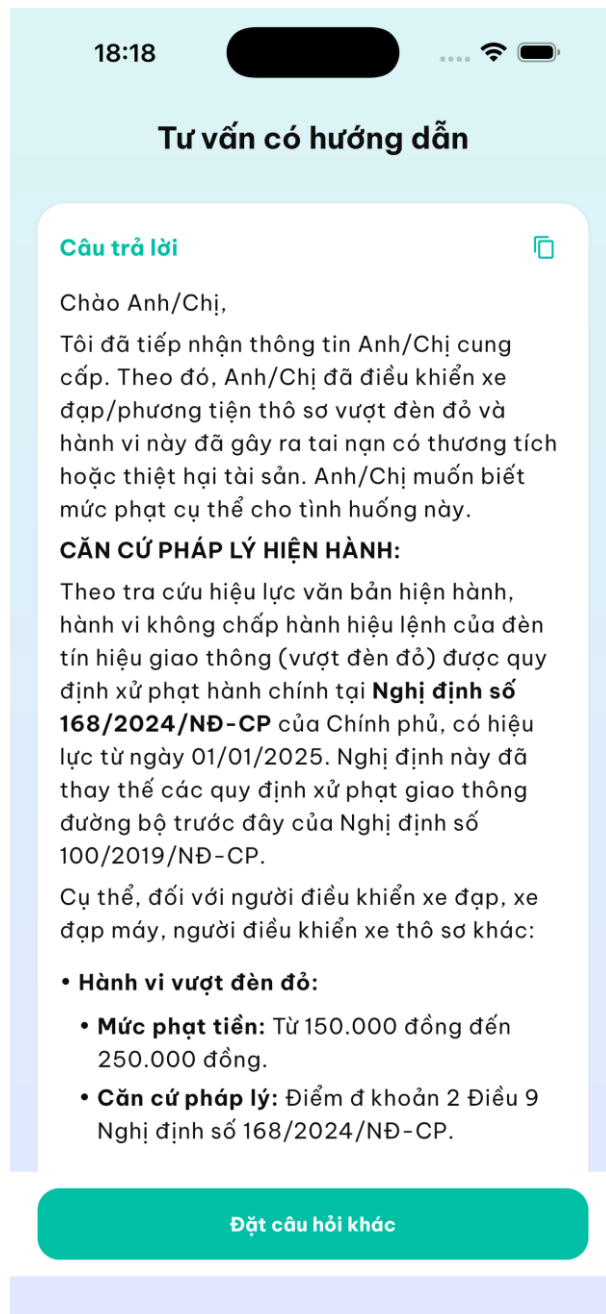
Hình 4.10. Guided Consultation - câu hỏi thu thập thông tin chi tiết

Ở bước làm rõ, hệ thống thu thập thêm thông tin chi tiết từ người dùng thông qua các câu hỏi trắc nghiệm có cấu trúc. Các câu hỏi này được sinh ở endpoint `guided-clarify` bằng LLMs dựa trên câu hỏi ban đầu, chủ đề pháp lý phát hiện được và những dữ kiện còn thiếu. Sau khi người dùng chọn phương án, Guided Graph sử dụng các thông tin này trong Planning Node để tạo truy vấn nội bộ và truy vấn web cụ thể hơn trước khi chạy Agent và Verifier.



Hình 4.11. Guided Consultation - hệ thống đang suy nghĩ và tổng hợp

Sau khi thu thập đủ ngữ cảnh, hệ thống chạy Guided Graph (START → planning → agent ↔ tools → verifier → END) để tổng hợp câu trả lời. Trạng thái đang tổng hợp câu trả lời được hiển thị tương tự ThinkingPanel, phản ánh quá trình Agent gọi công cụ truy hỏi và Verifier kiểm chứng kết quả.



Hình 4.12. Guided Consultation - kết quả tư vấn chuyên sâu theo ngữ cảnh

Bước cuối trả về câu trả lời chuyên sâu được tùy chỉnh hoàn toàn theo tình huống cụ thể của người dùng, kèm trích dẫn điều khoản và tên văn bản pháp lý liên quan. Nhờ thu thập đủ ngữ cảnh trước khi truy vấn, kết quả chính xác hơn so với câu hỏi mở thông thường, đặc biệt với các tình huống pháp lý có nhiều ngoại lệ theo từng đối tượng hoặc ngành nghề.

4.2.6. Quản lý hội thoại



Hình 4.13. Màn hình danh sách hội thoại

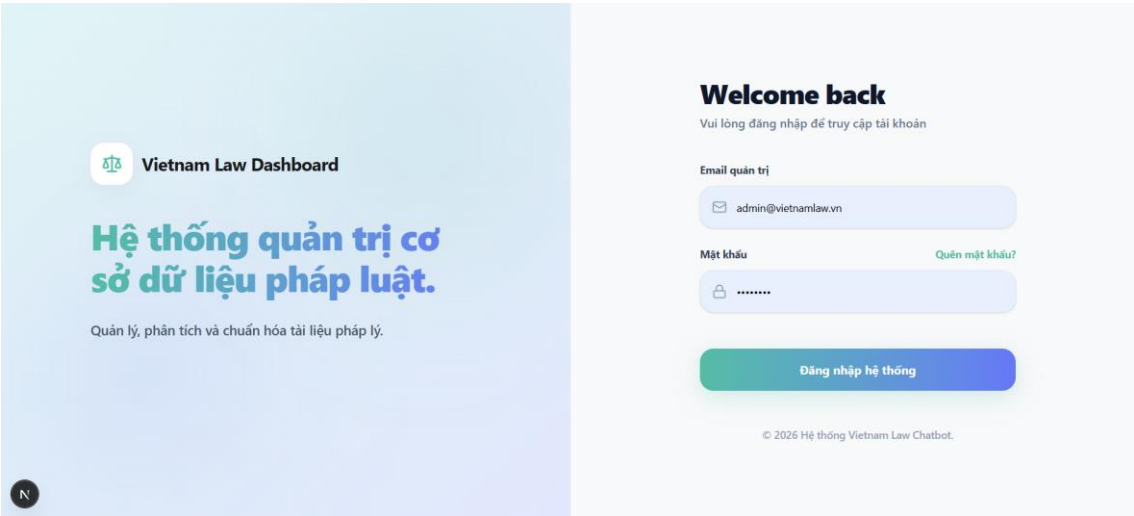
Ứng dụng lưu trữ toàn bộ lịch sử hội thoại theo tài khoản người dùng. Tiêu đề mỗi cuộc hội thoại được tự động tạo bởi LLM từ câu hỏi đầu tiên. Người dùng có thể ghim (pin) các cuộc hội thoại quan trọng, lưu trữ (archive) hoặc xóa các cuộc hội thoại không cần thiết.

4.3. Giao diện Admin Web

Admin Web được xây dựng bằng Next.js 16.1.6 với App Router, React 19.2.3, TailwindCSS 4 và thư viện UI shadcn/ui 3.8. Đây là giao diện quản trị nội bộ dành

cho người quản lý hệ thống, cung cấp các chức năng theo dõi và vận hành nội dung pháp luật trong hệ thống.

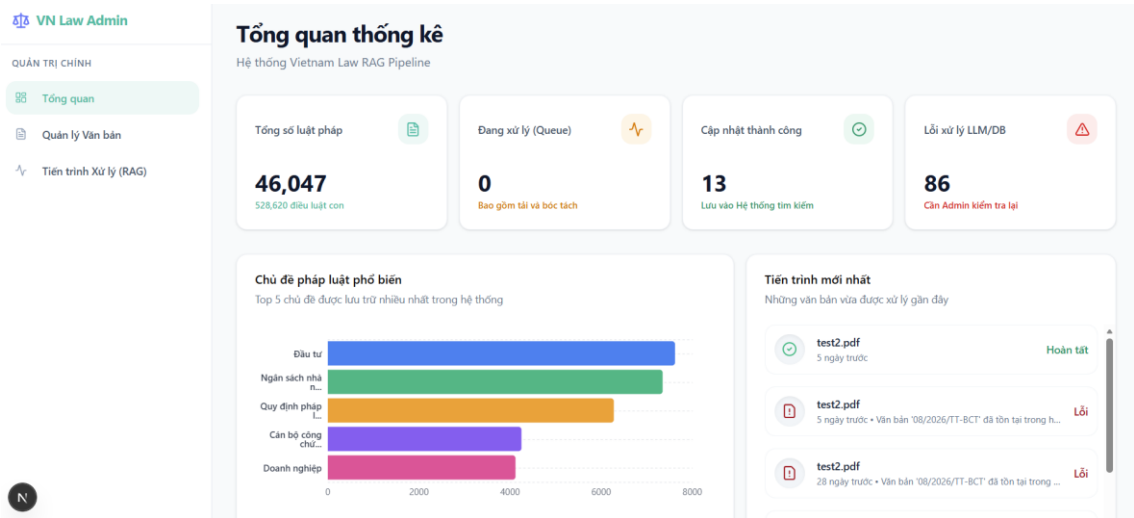
4.3.1. Đăng nhập quản trị



Hình 4.14. Trang đăng nhập Admin Web

Trang đăng nhập quản trị yêu cầu tài khoản có quyền role=admin được cấp từ hệ thống. Sau khi đăng nhập thành công, Admin Web lưu JWT access token trong cookie phía client tên admin_token và gắn token này vào header Authorization dạng Bearer khi gọi API. Cách triển khai hiện tại phù hợp cho môi trường demo của đồ án; khi triển khai sản xuất có thể nâng cấp sang cơ chế phiên bảo mật hơn ở phía server.

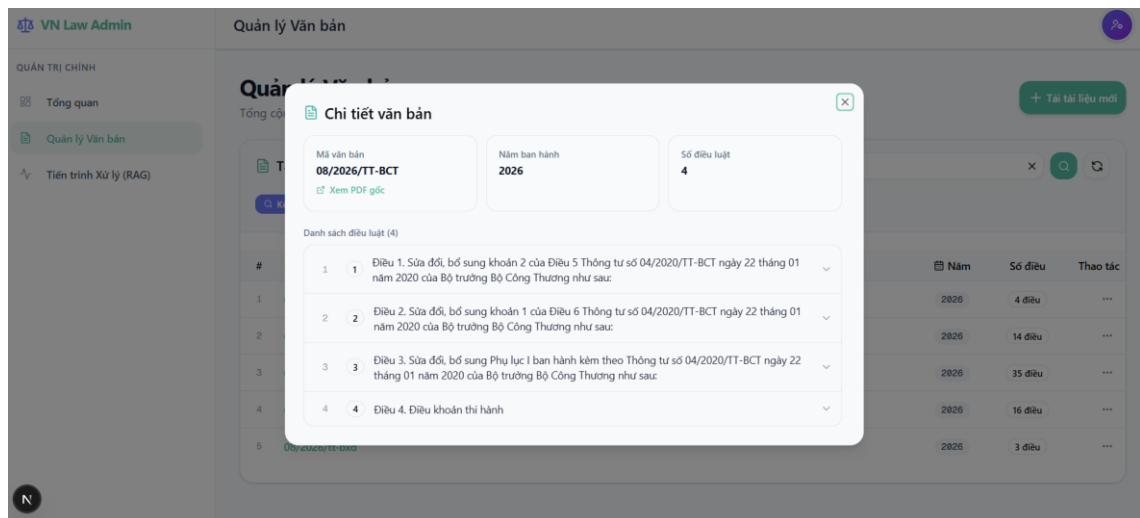
4.3.2. Dashboard tổng quan



Hình 4.15. Dashboard tổng quan với BarChart và các card thống kê

Dashboard hiển thị tổng quan trạng thái hệ thống thông qua API thống kê và cập nhật trạng thái tác vụ mới nhất bằng WebSocket khi có tiến trình xử lý tài liệu. Giao diện gồm các thẻ thống kê nhanh như tổng số văn bản, tổng số điều luật, số tác vụ đang xử lý, số tác vụ thành công và số tác vụ lỗi. Biểu đồ cột ngang sử dụng Recharts để thể hiện Top 5 chủ đề pháp luật phổ biến nhất trong kho dữ liệu, giúp quản trị viên nắm nhanh cấu trúc dữ liệu hiện có.

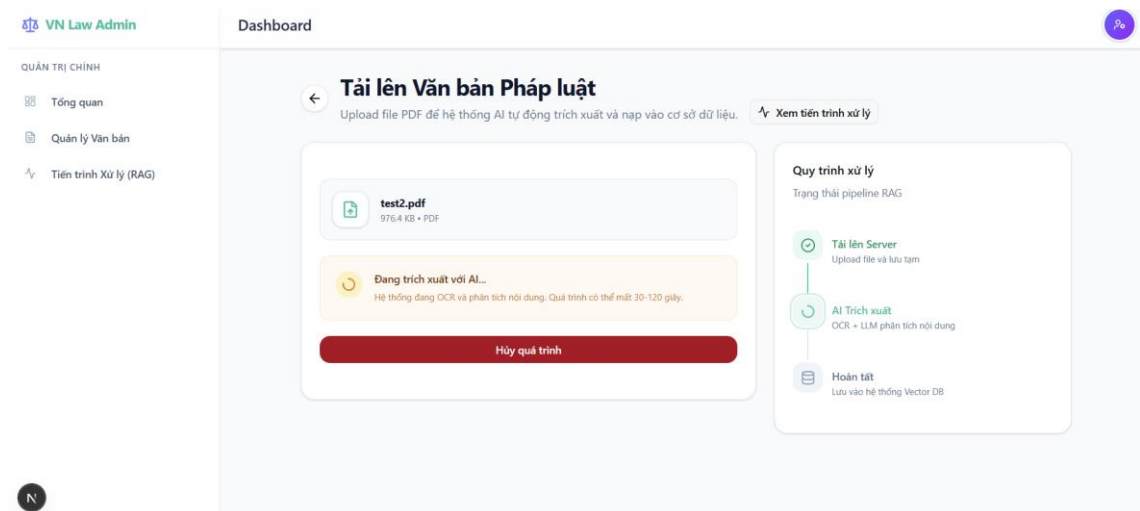
4.3.3. Quản lý tài liệu



Hình 4.16. Chi tiết tài liệu - danh sách điều khoản đã được trích xuất

Trang Documents hiển thị danh sách tất cả tác vụ xử lý tài liệu (DocumentTask), bao gồm các thông tin: tên tệp PDF, mã văn bản pháp luật (law_id) được trích xuất tự động, số điều khoản đã phân tách. Khi xem chi tiết từng tác vụ, quản trị viên có thể kiểm tra toàn bộ danh sách điều khoản đã được trích xuất, bao gồm số điều, tiêu đề và nội dung, để đối chiếu với văn bản PDF gốc và phát hiện các trường hợp phân tách sai hoặc thiếu.

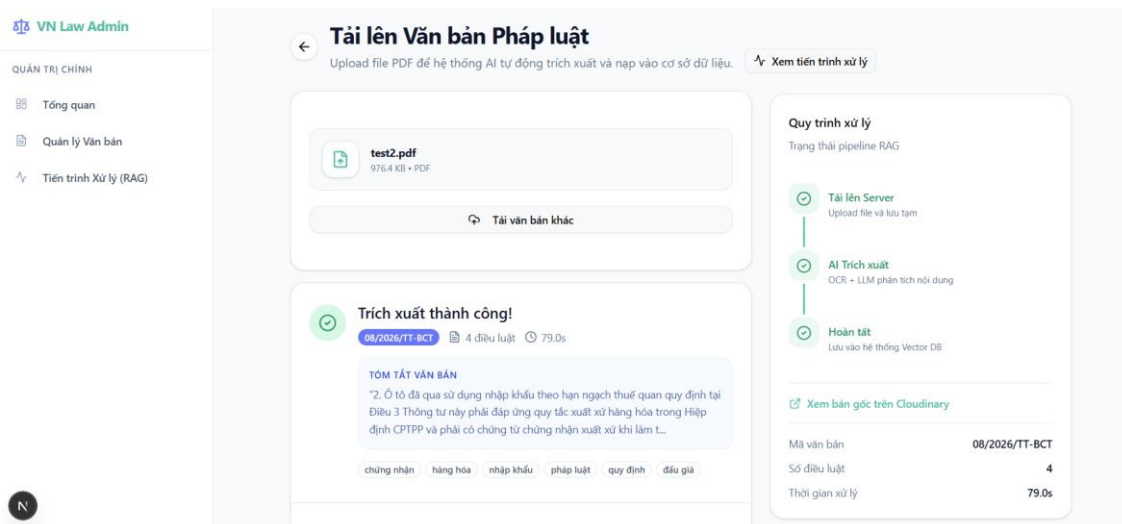
4.3.4. Tải lên và theo dõi tiến trình



Hình 4.17. Giao diện tải lên PDF đang trong quá trình xử lý

Trang Upload hỗ trợ kéo-thả tệp PDF hoặc chọn tệp theo cách thông thường. Sau khi tải lên, hệ thống tạo tác vụ xử lý, tiến kiểm trùng lặp, chạy OCR tự host để trích xuất lớp chữ hoặc nhận dạng ảnh quét, sau đó dùng LLMs để chuẩn hóa nội dung thành cấu trúc văn bản pháp luật. Việc tải tệp gốc lên Cloudinary được thực hiện song song với bước OCR. Nếu đường OCR/structuring không xử lý được tài liệu, hệ thống mới chuyển sang cơ chế đọc trực tiếp bằng mô hình thị giác-ngôn ngữ như một nhánh dự phòng. Kết quả hợp lệ được lưu vào MongoDB, tạo embedding và đồng bộ vào ChromaDB; tiến trình được phản hồi theo thời gian thực qua WebSocket.

Khả năng khôi phục sau khi tải lại trang được triển khai theo hướng lưu tên tệp đang xử lý trong localStorage. Khi người dùng quay lại trang Upload, giao diện gọi GET /documents/tasks để lấy danh sách tác vụ của quản trị viên hiện tại, tìm tác vụ đang chạy hoặc tác vụ đã hoàn tất tương ứng với tệp đang chờ, rồi khôi phục thành tiến trình hoặc hiển thị kết quả xử lý. Cách làm này giúp hạn chế mất trạng thái khi trình duyệt bị tải lại trong lúc pipeline vẫn đang chạy ở backend.



Hình 4.18. Trạng thái hoàn thành sau khi xử lý tài liệu thành công

4.4. Đánh giá hệ thống

4.4.1. Thiết kế thực nghiệm

Để đánh giá hiệu quả của giải pháp Agentic RAG đề xuất, phần thực nghiệm được thiết kế tập trung vào hai mục tiêu: (1) đánh giá chất lượng câu trả lời trên bộ dữ liệu kiểm thử đa dạng theo các chỉ số định lượng, và (2) đo lường hiệu năng của quy trình xử lý để xác định tính khả dụng trong điều kiện thực tế.

Bộ dữ liệu kiểm thử

Bộ dữ liệu kiểm thử được xây dựng thành hai tập câu hỏi độc lập với tổng cộng 60 câu, phủ rộng các lĩnh vực pháp luật thực tế bao gồm giao thông đường bộ, lao động, doanh nghiệp, đất đai & nhà ở và hình sự & dân sự:

Bảng 4.3. Cấu trúc bộ dữ liệu kiểm thử

Tập	Loại	Số câu	Mô tả
N1	Factual	30	Câu hỏi có đáp án xác định (số liệu, điều khoản cụ thể). Đánh giá tự động bằng so khớp kết quả; trong đó 3 câu chuyên biệt kiểm tra Temporal Conflict (NĐ 100/2019 vs NĐ 168/2024)
N2	Open/Reasoning	30	Câu hỏi mở, tình huống thực tế, cần tổng hợp nhiều nguồn. Đánh giá bằng ChatGPT (LLM-judge) theo thang 1–5

Câu hỏi tập N1 được tham chiếu với các văn bản pháp luật cụ thể (điều khoản và số văn bản) để có thể so khớp tự động; câu hỏi tập N2 đánh giá khả năng suy luận và tổng hợp của hệ thống trên các tình huống pháp lý phức tạp.

4.4.2. Các chỉ số đánh giá

Hệ thống được đánh giá bằng năm chỉ số, được phân nhóm theo mục tiêu đo lường:

Bảng 4.4. Bộ chỉ số đánh giá hệ thống

Chỉ số	Áp dụng	Mô tả	Phương pháp đo
Accuracy@1	N1	Tỷ lệ câu trả lời chứa đúng thông tin factual (số liệu, điều khoản)	So khớp tự động với expected_ref
Answer Quality	N2	Chất lượng tổng thể câu trả lời (độ đầy đủ, chính xác, có trích dẫn)	ChatGPT-5.5, thang điểm 1–5
Citation Accuracy	N1	Tỷ lệ nguồn được trích dẫn khớp với văn bản thực sự chứa đáp án	So khớp tự động
Temporal Conflict Detection Rate	Subset 3 câu N1	Tỷ lệ phát hiện đúng cặp xung đột pháp luật cũ/mới	So khớp tự động

Ngoài các chỉ số về chất lượng, phần thực nghiệm cũng đo độ trễ phản hồi, tính từ thời điểm gửi câu hỏi đến khi quá trình trả lời hoàn tất, theo phân vị P50 và P95 trên 60 lượt truy vấn. Đây là chỉ số quan trọng trong bối cảnh ứng dụng thực tế, khi người dùng kỳ vọng hệ thống phản hồi trong thời gian chấp nhận được.

4.4.3. Kết quả thực nghiệm

Thực nghiệm được tiến hành trên bộ 60 câu hỏi (30 N1 + 30 N2) chạy tự động qua API chatbot. Toàn bộ 60 câu hoàn thành không có lỗi. Điểm N1 được chấm theo phương pháp so khớp từ khóa tự động: mỗi câu hỏi được gán tập từ khóa factual trích từ văn bản pháp luật gốc, câu trả lời được tính đúng khi chứa ít nhất một từ khóa trong tập đó. Điểm N2 được chấm bởi ChatGPT theo thang 1–5 với tiêu chí: 5 = đầy đủ, chính xác, có trích dẫn điều khoản; 4 = đúng hướng, thiếu vài điểm phụ; 3 = đúng phần lớn nhưng thiếu thông tin quan trọng; 2 = lẫn thông tin đúng và sai; 1 = sai hoàn toàn.

Bảng 4.5. Kết quả đánh giá hệ thống Agentic RAG

Chỉ số	Kết quả	Ghi chú
Accuracy@1 (N1, 30 câu)	90,0% (27/30)	3 câu sai do thiếu số liệu điều khoản cụ thể trong câu trả lời
Citation Accuracy (N1)	56,7% (17/30)	Phản ánh việc NĐ 168/2024 chưa được lập chỉ mục đầy đủ trong ChromaDB
Temporal Conflict OK (3 câu ★)	3/3 (100%)	Hệ thống xác định đúng NĐ 168/2024 thay thế NĐ 100/2019
Answer Quality (N2, LLM-judge)	3,67/5,0	Phân bố: 5★×4, 4★×16, 3★×6, 2★×4, 1★×0
Độ trễ P50 (toàn bộ phản hồi)	65,5s	Bao gồm toàn bộ quy trình: Guardrail → Query Analysis → Agent → Verifier
Độ trễ P95 (toàn bộ phản hồi)	100,6s	Trường hợp câu hỏi phức tạp, Agent thực hiện nhiều vòng lặp ReAct

Nhận xét tổng hợp

Kết quả thực nghiệm cho thấy Agentic RAG đạt chất lượng câu trả lời tốt với Accuracy@1 = 90,0% và Answer Quality = 3,67/5,0, khẳng định khả năng tư vấn pháp lý thực tế của hệ thống. Hai phát hiện nổi bật được phân tích dưới đây.

Thứ nhất, Temporal Conflict Detection đạt 100% (3/3). Đây là tính năng đặc thù của hệ thống, không có ở các chatbot RAG thông thường. Trong cả ba câu hỏi kiểm tra xung đột thời gian giữa NĐ 100/2019 và NĐ 168/2024, hệ thống xác định đúng văn bản đang có hiệu lực, trình bày so sánh rõ ràng và không trích dẫn sai văn bản đã hết hiệu lực. Kết quả này chứng minh logic Temporal Conflict dựa trên năm trong số hiệu văn bản, loại văn bản và metadata chủ đề hoạt động đúng trong các tình huống kiểm thử.

Thứ hai, khoảng cách giữa Accuracy@1 (90,0%) và Citation Accuracy (56,7%) là một phát hiện quan trọng. Hệ thống trả lời đúng nhiều hơn so với tỷ lệ truy hồi đúng văn bản nguồn, cho thấy mô hình ngôn ngữ lớn có thể bổ sung kiến thức nội tại từ quá trình huấn luyện khi quá trình truy hồi không tìm được đúng văn bản, đặc biệt với NĐ 168/2024 là văn bản mới chưa được lập chỉ mục đầy đủ. Đây

vừa là điểm mạnh vì hệ thống vẫn trả lời đúng, vừa là hạn chế vì câu trả lời thiếu nguồn trích dẫn từ kho dữ liệu nội bộ nên khó kiểm chứng.

Về *Answer Quality (N2)*, điểm 3,67/5,0 phản ánh hệ thống trả lời đúng hướng với căn cứ pháp lý rõ ràng ở hầu hết câu hỏi mở. Một số câu bị trừ điểm do thiếu chi tiết về chế tài, quy trình khiếu nại hoặc chưa phân biệt đủ các trường hợp ngoại lệ - đây là giới hạn tự nhiên khi câu hỏi mở yêu cầu phân tích tình huống phức tạp.

Về độ trễ, $P50 = 65,5$ giây và $P95 = 100,6$ giây là thời gian phản hồi cao hơn kỳ vọng ban đầu (ước tính 10–15 giây). Nguyên nhân chủ yếu đến từ: (1) mỗi câu hỏi thực hiện 4–6 lần gọi LLM tuần tự, (2) API của mô hình ngôn ngữ lớn có độ trễ nền khoảng 10–15 giây/lần gọi trong điều kiện thực tế, và (3) truy hồi hai giai đoạn với top-60 $\rightarrow$ top-20 bổ sung khoảng 500 ms. Trong bối cảnh tư vấn pháp lý, nơi độ chính xác quan trọng hơn tốc độ, đây là đánh đổi có thể chấp nhận được, nhưng cần cải thiện trong phiên bản sản xuất thực tế.

4.4.4. Phân tích tình huống nghiên cứu

Để minh họa cụ thể hiệu quả của hệ thống, mục này phân tích một tình huống nghiên cứu điển hình thể hiện tính năng Temporal Conflict Resolution.

Tình huống nghiên cứu: Phát hiện xung đột mức phạt vi phạm nồng độ cồn

Câu hỏi: "Hiện nay lái xe ô tô có nồng độ cồn 0,35 mg/L khí thở bị phạt bao nhiêu?"

Đây là câu hỏi có xung đột thời gian điển hình: hệ thống CSDL chứa cả NB 100/2019/NĐ-CP (quy định mức phạt cũ 30–40 triệu đồng, tước GPLX 16–18 tháng) và NB 168/2024/NĐ-CP (thay thế một phần NB 100/2019 từ ngày 01/01/2025, quy định mức phạt 30–40 triệu đồng, tước GPLX 22–24 tháng với mức vi phạm tương đương).

Nếu dùng phương pháp RAG không có Temporal Conflict Detection, quá trình truy hồi đoạn dữ liệu có thể trả về thông tin từ văn bản cũ hoặc mới mà không phân biệt, dẫn đến câu trả lời mơ hồ hoặc sai. Agentic RAG xử lý theo luồng:

1. Query Analysis xác định đây là câu hỏi về mức xử phạt giao thông, cần tìm kiếm theo từ khóa "nồng độ cồn" và "xe ô tô"; sau khi truy hồi, hệ thống ưu tiên văn bản mới hơn bằng trường year trong metadata và dùng năm trong law_id để phát hiện cặp văn bản có khả năng xung đột.
2. Công cụ retrieve_internal_law thực hiện Two-Stage Retrieval; Temporal Conflict Detection phát hiện hai văn bản có cùng loại văn bản và chủ đề chính nhưng khác năm ban hành trong số hiệu văn bản $\rightarrow$ đánh dấu cặp xung đột.

3. Agent nhận kết quả quan sát có cặp xung đột, lấy cả hai nguồn vào ngữ cảnh kèm chú thích.

4. **Verifier** kiểm tra và xác nhận câu trả lời chỉ trích dẫn NĐ 168/2024 là văn bản đang có hiệu lực, đồng thời giải thích rằng NĐ 100/2019 đã được thay thế.

Kết quả: câu trả lời chính xác, minh bạch, hiển thị cả hai nguồn để người dùng tham khảo.

4.4.5. Đánh giá hiệu năng hệ thống

Bảng 4.6. Chỉ số hiệu năng hệ thống

Chỉ số	Giá trị đo được
Truy vấn ChromaDB top-60 (cosine, HNSW)	< 200ms
Xếp hạng lại hai giai đoạn (top-60 → top-20)	< 500ms
Phản hồi đầy đủ Agentic RAG (P50)	65,5 giây (đo thực nghiệm, 60 câu)
Phản hồi đầy đủ Agentic RAG (P95)	100,6 giây (đo thực nghiệm)
Quy trình nạp dữ liệu (PDF 50 điều, khoảng 30 trang)	3–5 phút
Tạo embedding theo lô 100 đoạn	< 10 giây

Về mặt tài nguyên, RAG Service tiêu thụ khoảng 500–800 MB RAM trong điều kiện bình thường, tăng lên 1–1,5 GB khi xử lý lô embedding lớn. ChromaDB với 690.360 vector chiều 768 chiếm khoảng 3,5 GB dung lượng lưu trữ trên đĩa. Các thông số này nằm trong giới hạn hoạt động bình thường của một máy chủ phổ thông với 8 GB RAM.

4.5. Tổng kết chương 4

Chương 4 đã trình bày toàn diện kết quả triển khai hệ thống Vietnam Law Chatbot trên ba phương diện: môi trường vận hành thực tế, giao diện người dùng trực quan trên hai nền tảng Mobile (KMP) và Admin Web (Next.js), cùng với kết quả thực nghiệm định lượng đánh giá hiệu quả phương pháp Agentic RAG đề xuất.

Kết quả thực nghiệm cho thấy việc bổ sung các cơ chế Guardrail, Two-Stage Reranking, ReAct Agent Loop và Verifier giúp cải thiện chất lượng câu trả lời, đặc biệt trong việc hạn chế hiện tượng sinh thông tin không có căn cứ và xử lý xung đột pháp luật theo thời gian, đây là hai vấn đề đặc thù của lĩnh vực pháp lý Việt Nam. Hạn chế chính của quy trình hiện tại là độ trễ cao (P50 = 65,5 giây, P95 = 100,6

giây), tuy nhiên mức đánh đổi này phù hợp hơn với bối cảnh tư vấn pháp lý, nơi độ chính xác được ưu tiên hơn tốc độ.

KẾT LUẬN

Sau quá trình nghiên cứu cơ sở lý thuyết, phân tích yêu cầu, thiết kế và triển khai hệ thống, đồ án đã xây dựng được hệ thống Vietnam Law Chatbot nhằm hỗ trợ người dùng tra cứu, hỏi đáp và tiếp cận thông tin pháp luật Việt Nam theo hướng thuận tiện hơn. Hệ thống được phát triển trên nền tảng kết hợp mô hình ngôn ngữ lớn với phương pháp Agentic RAG, qua đó vừa khai thác khả năng sinh ngôn ngữ tự nhiên, vừa tăng cường khả năng truy hồi căn cứ từ kho văn bản pháp luật.

- **Kết quả đạt được**

- Trình bày được cơ sở lý thuyết liên quan đến LLMs, RAG, AI Agent, Agentic RAG, LangGraph, ChromaDB và các công nghệ được sử dụng trong quá trình xây dựng hệ thống.
- Phân tích và thiết kế được kiến trúc hệ thống gồm ứng dụng di động, trang quản trị, dịch vụ xử lý chính, dịch vụ RAG và các lớp lưu trữ dữ liệu phục vụ cho bài toán tư vấn pháp luật.
- Xây dựng được các chức năng trọng tâm như hỏi đáp pháp luật, tư vấn có hướng dẫn, tra cứu thư viện văn bản, tìm kiếm ngữ nghĩa, quản lý hội thoại, cập nhật văn bản pháp luật và theo dõi tiến trình xử lý dữ liệu.
- Thực nghiệm hệ thống trên bộ câu hỏi kiểm thử, từ đó đánh giá được khả năng trả lời, khả năng trích dẫn nguồn và khả năng xử lý một số trường hợp có yếu tố xung đột pháp luật theo thời gian.

- **Hạn chế**

- Độ trễ phản hồi còn tương đối cao do quy trình Agentic RAG gồm nhiều bước xử lý, nhiều lần truy hồi và nhiều lượt gọi mô hình ngôn ngữ lớn.
- Độ chính xác của phân trích dẫn nguồn vẫn phụ thuộc vào mức độ đầy đủ và chất lượng của kho dữ liệu đã được xử lý, nhúng và lập chỉ mục trong cơ sở dữ liệu vector.
- Một số câu hỏi pháp lý mở hoặc có tình huống phức tạp vẫn cần cải thiện thêm về khả năng lập luận, tổng hợp căn cứ và diễn giải kết quả theo ngữ cảnh người dùng.

- **Hướng phát triển**

- Tối ưu tốc độ xử lý bằng cách giảm số lượt gọi mô hình, cải thiện cơ chế truy hồi, bổ sung cache cho các truy vấn phổ biến và hoàn thiện luồng phản hồi dạng streaming.

- Mở rộng quy trình cập nhật dữ liệu pháp luật theo định kỳ, bổ sung kiểm tra hiệu lực văn bản và mở rộng phạm vi dữ liệu theo nhiều lĩnh vực pháp luật hơn.
- Bổ sung bộ thực nghiệm có quy mô lớn hơn, kết hợp đánh giá từ chuyên gia pháp lý và tiếp tục hoàn thiện giao diện để hệ thống phù hợp hơn với nhu cầu sử dụng thực tế.

TÀI LIỆU THAM KHẢO

- [1] P. Lewis, E. Perez, A. Piktus và cộng sự, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", NeurIPS, 2020.
- [2] S. Yao, J. Zhao, D. Yu và cộng sự, "ReAct: Synergizing Reasoning and Acting in Language Models", ICLR, 2023.
- [3] Y. Gao, Y. Xiong, X. Gao và cộng sự, "Retrieval-Augmented Generation for Large Language Models: A Survey", arXiv:2312.10997, 2023.
- [4] LangChain, "LangGraph Documentation", Online: <https://langchain-ai.github.io/langgraph/>.
- [5] Chroma, "Chroma Documentation", Online: <https://docs.trychroma.com/>.
- [6] PostgreSQL Global Development Group, "PostgreSQL Documentation", Online: <https://www.postgresql.org/docs/>.
- [7] MongoDB, "MongoDB Manual", Online: <https://www.mongodb.com/docs/manual/>.
- [8] JetBrains, "Kotlin Multiplatform Documentation", Online: <https://www.jetbrains.com/help/kotlin-multiplatform-dev/>.
- [9] Android Developers, "Jetpack Compose Documentation", Online: <https://developer.android.com/compose>.
- [10] Vercel, "Next.js Documentation", Online: <https://nextjs.org/docs>.
- [11] Cơ sở dữ liệu quốc gia về văn bản pháp luật, "Hệ thống văn bản quy phạm pháp luật Việt Nam", Online: <https://vbpl.vn/>.